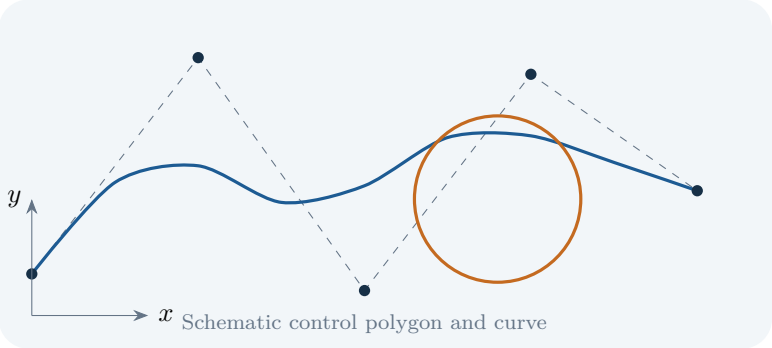
LIBRARY MANUAL / FIRST EDITION

nurbspath

An illustrated C++20 reference

Entities, construction, NURBS paths,
numerical queries, and output



Schematic control polygon and curve

Library version 0.4.3

Source revision `6ee464320b96`

27 October 2026

Standalone experimental manual. All figures are TikZ/PGFPlots drawings; the sampled reference curves and query results are calculated by nurbspath. This document is built independently of the project CMake configuration.

Contents

1 Reading and using the library **1**

1.1 Scope and a first working program 1

1.2 Two independent Cartesian worlds 2

1.3 Units, parameters, and tolerances 2

1.4 Values, shared ownership, and checked editing 2

1.5 How the worked figures were made 3

2 Entities and elementary geometry **4**

2.1 Points: positions and translations 4

2.2 Vectors: directions and their components 5

2.3 Rotation about a 3D origin axis 8

2.4 Rays and their native parameter 8

2.5 Circle and sphere surfaces 9

2.6 Planes: an orthonormal local frame 11

2.7 Reading the contracts consistently 12

3 Construction and shared ownership **13**

3.1 Point and vector factory catalogue 13

3.2 Ray, circle, and sphere creation 14

3.3 Five ways to allocate a plane 14

3.4 Ordinary spline constructors and factories 15

3.5 Array-backed spline construction 17

3.6 Conversions and definition updates 18

4 NURBS curves and interpolation **19**

4.1 One curve definition, two separate worlds 19

4.2 Construction from a complete definition 19

4.3 Weights and exact conic geometry 21

4.4 Evaluation and analytic differential geometry 22

4.5 Global interpolation through sample positions 24

4.6 Two different notions of length 26

4.7 Checked editing and flattened scalar parameters . . . 26
4.8 Array-backed curves for direct numeric work . . . 28

5 Intersections, distances, and numerical tools 31
5.1 Choose a query and its controls . . . 31
5.2 Read an intersection result . . . 32
5.3 Rays: forward domains and direction scales . . . 33
5.4 Splines: crossings, tangencies, and continuous contact . . . 35
5.5 Distances return geometry, not just a number . . . 35
5.6 Closest points on NURBS splines . . . 38
5.7 Reusable scalar algorithms . . . 39
5.8 Linear systems and small helpers . . . 41

6 Rendering, persistence, and diagnostics 43
6.1 A flat native 2D canvas . . . 43
6.2 World-space cameras in 3D . . . 43
6.3 Diagnostic glyphs and convenience rendering . . . 44
6.4 Coordinate streams, CSV/TSV, and binary records . . . 45
6.5 Tagged records and independent presentation rows . . . 46
6.6 Advisory application and positioned messages . . . 47
6.7 Configuration and the hand-authored extension point . . . 48

A Public API catalogue 49
A.1 nurbspath::application_info . . . 49
A.2 nurbspath::circle2 . . . 49
A.3 nurbspath::diagnostic_message3 . . . 50
A.4 nurbspath::intersection2 . . . 51
A.5 nurbspath::intersection3 . . . 51
A.6 nurbspath::intersection_result2 . . . 51
A.7 nurbspath::intersection_result3 . . . 51
A.8 nurbspath::line_color . . . 52
A.9 nurbspath::line_font . . . 52
A.10 nurbspath::numerical_settings . . . 52
A.11 nurbspath::nurbs_arr_spline2 . . . 53
A.12 nurbspath::nurbs_arr_spline3 . . . 56
A.13 nurbspath::nurbs_spline2 . . . 60
A.14 nurbspath::nurbs_spline3 . . . 63
A.15 nurbspath::plane3 . . . 66
A.16 nurbspath::point2 . . . 67
A.17 nurbspath::point3 . . . 68

A.18 nurbspath::point_circle_distance2 . . . 69
A.19 nurbspath::point_spline_distance2 . . . 70
A.20 nurbspath::point_spline_distance3 . . . 70
A.21 nurbspath::point_surface_distance3 . . . 70
A.22 nurbspath::ray2 . . . 70
A.23 nurbspath::ray3 . . . 71
A.24 nurbspath::sphere3 . . . 72
A.25 nurbspath::spline_derivatives2 . . . 73
A.26 nurbspath::spline_derivatives3 . . . 73
A.27 nurbspath::svg_canvas2 . . . 73
A.28 nurbspath::svg_document3 . . . 74
A.29 nurbspath::svg_graphics_options2 . . . 74
A.30 nurbspath::svg_graphics_options3 . . . 74
A.31 nurbspath::svg_view2 . . . 75
A.32 nurbspath::svg_view3 . . . 75
A.33 nurbspath::tagged_entity3 . . . 76
A.34 nurbspath::tagged_read_result . . . 76
A.35 nurbspath::tagged_record . . . 76
A.36 nurbspath::text_label . . . 76
A.37 nurbspath::vector2 . . . 77
A.38 nurbspath::vector3 . . . 79
A.39 Namespace functions, aliases, and result enums . . . 82

B Rebuilding and extending this manual **95**
B.1 A standalone local build . . . 95
B.2 Source layout . . . 96
B.3 Your introductory and closing material . . . 96
B.4 Refreshing the API snapshot . . . 96
B.5 Checking numerical and visual changes . . . 97

List of Figures

2.1 Positions are anchored in a Cartesian world. The difference $B - A$ is a displacement; adding that displacement to A recovers B. 5

2.2 The 2D perpendicular operation and the 3D cross product have different return types. Both retain orientation information. 7

2.3 Projection and rejection decompose a vector. Reflection across the horizontal line reverses the normal component and retains the parallel component. 7

2.4 Positive rotation about $+z$ appears counterclockwise when viewed from the positive-Z side. The public rotation call is for a 3D point. 8

2.5 Uniform increments in s follow the unnormalized direction. The dashed continuation behind the origin is outside the ray domain. 9

2.6 A circle's parameter is an angle, while its outward normal is a unit radial vector. . 10

2.7 Sphere coordinates use longitude around the Z axis and latitude above the XY plane. This is a schematic 3D view. 10

2.8 The orthonormal frame gives world-distance plane coordinates. Orthogonal projection removes the signed height h along the normal. 11

3.1 Three-point plane construction viewed in the plane itself. The third point chooses the positive-v side; its along-u component is discarded when the frame is built. . . 15

3.2 The standard factory places control points along a segment. A closed flag validates a seam; the supplied definition determines the shape. 17

4.1 Control points steer the curve; they are generally not samples on it. The blue path is evaluated by the library. The clamped endpoint blocks in this example make the evaluated endpoints coincide with P_0 and P_4. 20

4.2 Stored knots and the active domain. Endpoint multiplicities describe clamping; they are not extra finite intervals to sample. 21

4.3 A quadratic rational curve lies on the unit circle, even though its control polygon follows the two sides of a square. Its curvature is one in inverse world units. The plotted blue points are library evaluations. 22

4.4 Analytic first-derivative components and curvature for the running cubic curve. Speed and turning vary through the native domain. The plotted values come from `first_derivative` and `curvature`. 23

4.5 A curve through measured positions. Each annotation is the original native station, not a newly normalized coordinate. The library supplies the evaluated curve and sample positions used in this TikZ picture. 25

4.6 Arc-length sampling joins evaluated points with straight chords. Uniform native-parameter steps generally produce unequal world-space chord lengths. The coarse orange polyline illustrates the approximation, not an adaptive accuracy guarantee. 27

4.7 Flattened numeric ordering for an ordinary spline's indexed parameters and an array-backed spline's public `parameters` storage. Typed metadata is stored separately. 27

5.1 Crossing and tangent ray–circle contacts. The black points are computed by the library; TikZ constructs the unit circles and rays. The tangent contact has no residual sign change. 33

5.2 The three ray–plane classifications. The drawn patch represents an infinite plane; its finite outline is only a diagram aid. 34

5.3 Closed-seam deduplication and sampled continuous-contact detection. The left-hand surface is shown by its trace through the curve's drawing plane. The right-hand dots illustrate residual checks rather than isolated intersection records. 36

5.4 Analytic closest points for radial surfaces and planes. These distances need no sampling. Interior radial queries still measure to the surface, rather than returning zero throughout the filled region. 37

5.5 A sampled and refined 2D closest point. The curve samples and the two endpoints of the red distance segment come from the library reference-data generator in `manual/pictures/`. The 3D distance overload follows the same search strategy in its independent world. 38

5.6 Why closest-point searches refine more than the single best sample. This schematic squared-distance graph illustrates a minimum near the beginning of the domain and another basin near its end. It is not an additional library test curve. 39

5.7 The scalar residual $f(x) = (x+0.8)(x-0.3)^2(x-1.2)$. Its sampled graph is generated by C++ using the library's scalar helper; TikZ marks the known reference roots. Sign changes alone would miss the contact at $x = 0.3$. 40

5.8 A root refiner and a minimum refiner answer different questions. The green segments show one possible retained interval. The diagrams are schematic; neither refiner reports a continuous global-search certificate. 41

6.1 A flat drawing needs no plane embedding. Uniform scale preserves shape; the image coordinate direction is a display convention. 44

6.2 Camera projection is a display operation on the same 3D geometry. The perspective diagram is a 2D cross-section of the viewing cone. 44

6.3 Near-plane clipping prevents an invisible endpoint from producing an unbounded perspective line. The clipped endpoint is an intermediate rendering point, not a new library entity. 45

6.4 Reader calls skip advisory rows, then consume at most one required geometry or presentation row. Malformed required data still fails strictly. 47

Chapter 1

Reading and using the library

1.1 Scope and a first working program

`nurbspath` is a dependency-free, header-only C++20 library. Its geometry types use a scalar template parameter constrained by `std::floating_point`; `float`, `double`, and `long double` are valid choices. Include `nurbspath/nurbspath.hpp` for the complete API or a focused public header for a smaller translation unit.

This manual describes the source snapshot printed on the title page. Chapters 2–6 explain the public operations by purpose; Appendix A records individual overloads, defaults, result types, and public data members extracted from those headers. Internal `nurbspath::detail` helpers are implementation, not a public contract.

```
#include <nurbspath/config.hpp>
#include <nurbspath/nurbspath.hpp>
#include <iomanip>
#include <iostream>

int main() {
    namespace np = nurbspath;
    using REAL = double;
    const np::circle2<REAL> circle({0.0, 0.0}, 1.0);
    const np::ray2<REAL> ray({-3.0, 0.0}, {2.0, 0.0});
    const auto result = np::intersect(ray, circle);
    std::cout << std::scientific << std::setprecision(4);
    for (const auto& hit : result.points) {
        std::cout << "s=" << hit.s << " point=" << hit.point << '\n';
    }
}
```

```
s=1.0000e+00 point=-1.0000e+00 0.0000e+00
s=2.0000e+00 point=1.0000e+00 0.0000e+00
```

The ray direction has length two, so the native parameter is half the travelled world distance. Compile this program directly with the source include directory:

```
c++ -std=c++20 -I../include first_query.cpp -o first_query
```

This command is shown relative to the manual directory. The manual's own reproducible reference programs and build instructions are in Appendix B.

1.2 Two independent Cartesian worlds

Concept	Independent 2D world	Shared 3D world
Position	`point2<REAL>`	`point3<REAL>`
Offset or direction	`vector2<REAL>`	`vector3<REAL>`
Forward line	`ray2<REAL>`	`ray3<REAL>`
Round surface	`circle2<REAL>`	`sphere3<REAL>`
Rational path	`nurbs_spline2<REAL>`	`nurbs_spline3<REAL>`
Array-backed path	`nurbs_arr_spline2 <REAL>`	`nurbs_arr_spline3 <REAL>`
Plane	No 2D plane entity	`plane3<REAL>`

A point represents a location; a vector represents an offset or direction. Point subtraction gives a vector, point plus vector gives a point, and vector arithmetic remains vector arithmetic. A 2D entity does not implicitly become a 3D entity. The explicit bridge is `project(plane, entity2)`, described in Chapter 2. Queries on 2D entities compute entirely in 2D.

1.3 Units, parameters, and tolerances

The library does not prescribe metres, millimetres, or another length unit. Choose one consistent world unit. Angles are in radians. Display widths and text sizes are separate consumer-selected display units.

Entity	Parameter domain	Meaning
Ray	$s \geq 0$ for forward queries	$O + s\boldsymbol{d}$; direction is unnormalised.
Spline	$[t_p, t_C]$	Active knot interval; C is the number of controls.
Circle	$u \in [0, 2\pi)$	Counterclockwise angle from positive x.
Sphere	$u \in [0, 2\pi)$, $v \in [-\pi/2, \pi/2]$	Longitude and latitude.
Plane	$(u, v) \in \mathbb{R}^2$	Orthonormal local coordinates.

The ray's evaluation formula can be evaluated for negative s, but intersection queries treat it as a forward half-line. Spline parameters remain native: interpolation preserves the supplied stations, and plane embedding preserves the same s values. A spline parameter is not automatically a physical arc length everywhere between interpolation stations.

What a tolerance controls

Definition tolerances validate entity construction and parameter boundaries. `numerical_settings<REAL>` separately controls function residuals, parameter intervals, iteration counts, and sampling density. These quantities have different units. Sampling-based intersections, spline distances, and arc length estimates depend on their chosen sample counts; a finite sampling run does not prove a continuous global result for every possible curve.

1.4 Values, shared ownership, and checked editing

Construct entities as ordinary values for simple ownership. The `make_*` functions in `creators.hpp` mirror geometry constructors and return `std::shared_ptr` objects. They preserve constructor validation and exceptions. They do not introduce a common geometry base class.

Ordinary spline definitions are private and use checked, atomic setters. Failed edits leave the old definition intact. Array-backed alternatives deliberately expose a `std::valarray<REAL>`; direct replacement is allowed, so geometry operations validate the current storage and do not rely on stale endpoint caches. References returned by getters follow their documented invalidation rules.

1.5 How the worked figures were made

Simple vector, plane, and circle constructions are drawn directly in TikZ. Curves, analytic derivatives, interpolation points, intersections, and closest points use data generated by `pictures/generate_reference.cpp`, which includes the library itself. The generator checks its results before writing the coordinate files. The stored plot coordinates and displayed output use scientific notation with at most four decimal places. The calculations are performed in full `double` precision before display rounding.

The figures distinguish a mathematical curve from its drawn approximation: PGFPlots connects library-evaluated points. Its smooth appearance is not an extra numerical guarantee. This manual requires no image downloads, external plotting service, shell escape, or TikZ externalisation.

Chapter 2

Entities and elementary geometry

An entity carries both a geometric meaning and a dimension. A `point2` is a position in the native 2D world; a `vector2` is a displacement in that world. Their 3D counterparts have the same distinction. All geometry templates take a floating-point scalar type, written `REAL`; the examples in this chapter use `double`.

Table 2.1: Entity families and their native parameters.

Entity	Meaning	Parameters
`point2`, `point3`	Cartesian position	coordinates
`vector2`, `vector3`	Offset or direction	components
`ray2`, `ray3`	Forward half-line	$s \geq 0$
`circle2`	Circle in the native 2D world	angle u
`sphere3`	Spherical surface in 3D	longitude u, latitude v
`plane3`	Infinite oriented plane in 3D	local distances u, v
`nurbs_spline2`, `nurbs_spline3`	Rational B-spline curve	native s
`nurbs_arr_spline2`, `nurbs_arr_spline3`	Array-backed curve alternative	native s

There is no `plane2`. A 2D point is not implicitly a 3D point with a zero Z coordinate. The explicit bridge `project(plane, entity2)` embeds a 2D entity using a specified `plane3`; its geometric meaning is described in Chapter 5. Spline definitions and their analytic derivatives are developed in Chapter 4.

2.1 Points: positions and translations

`point2<REAL>` and `point3<REAL>` are aggregates with public, zero-initialized coordinates `x`, `y`, and, for 3D, `z`. Aggregate initialization and `origin()` provide their creation calls:

```
using namespace nurbspath;
point2<double> a{1.0, 2.0};
point3<double> b{1.0, 2.0, 3.0};
const auto o2 = point2<double>::origin();
const auto o3 = point3<double>::origin();
```

`operator[]` gives mutable or constant coordinate access in X/Y/Z order, starting at index zero; an invalid index throws `std::out_of_range`. `operator==` compares coordinates exactly. Use

`approximately_equal(other, tolerance)` when a Euclidean position tolerance is intended.

Subtracting two points gives the vector from the right operand to the left. Adding or subtracting a vector translates a point; `+=` and `-=` are the in-place forms. Both `point + vector` and `vector + point` are available. Two points cannot be added, and a point cannot be scaled as though it were a displacement.

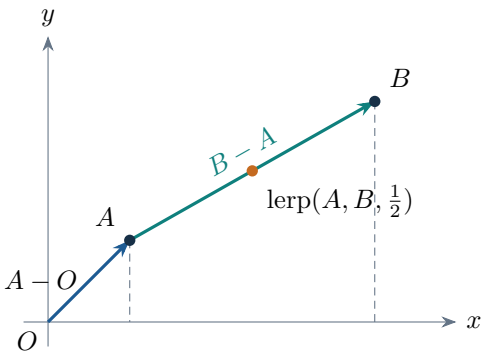


Figure 2.1: Positions are anchored in a Cartesian world. The difference $B - A$ is a displacement; adding that displacement to A recovers B.

```
const point3<double> first{1.0, 1.0, 0.0};
const point3<double> second{4.0, 3.0, 0.0};
const vector3<double> offset = second - first;
const point3<double> translated = first + offset;
const point3<double> middle = lerp(first, second, 0.5);
const double separation = distance(first, second);
```

`lerp(first, second, fraction)` evaluates $P_0 + t(P_1 - P_0)$ in either world. It permits extrapolation: fractions below zero or above one continue along the same line. `distance` returns Euclidean distance and `distance_squared` returns its square without a square root. These free functions take two points from the same world.

The point members `magnitude()` and `manhattan_distance()` measure distance from the Cartesian origin, respectively $\sqrt{x^2 + y^2 + z^2}$ and $|x| + |y| + |z|$, with the Z term omitted in 2D. They do not measure distance from a user-selected local origin.

Explicit component copying

A same-world vector may be assigned to a point. For example, `b = vector3<double>{1, 2, 3}` replaces the point coordinates with those components. This is a coordinate-copy operation, not a translation. Point and vector types nevertheless remain distinct, and no dimension conversion is introduced by this assignment.

2.2 Vectors: directions and their components

Vectors have public, zero-initialized components and the same checked indexed access and exact equality as points. Their constructors are:

```
vector2<double> zero2;
vector2<double> v2{3.0, 4.0};
vector3<double> zero3;
vector3<double> v3{1.0, 2.0, 3.0};
vector3<double> from_position{point3<double>{1.0, 2.0, 3.0}};
```

The constructors from entities are explicit. A `vector2` constructed from a `point2`, `circle2`, or `ray2` copies, respectively, the point coordinates, circle center, or ray origin. A `vector3` constructed from a `point3`, `sphere3`, `plane3`, or `ray3` copies the point, center, plane origin, or ray origin. In particular, constructing a vector from a ray does not extract its direction; use `direction()` for that purpose.

2.2.1 Algebra and length

For both vector types, unary plus and minus, vector addition and subtraction, scalar multiplication in either order, and scalar division are available. Their compound forms are `+=`, `-=`, `*=`, and `/=`. Division by an exactly zero scalar throws `std::domain_error`.

Table 2.2: Vector operations common to 2D and 3D. Here a and b have the same vector type and t is a scalar.

Call	Result or use
`a.dot(b)`, `dot(a, b)`	Scalar inner product $a \cdot b$.
`a.cross(b)`, `cross(a, b)`	Signed scalar area in 2D; right-handed perpendicular vector in 3D.
`a.length_squared()`	$a \cdot a$.
`a.length()`	Euclidean length, evaluated with `std::hypot`.
`a.is_near_zero(tolerance)`	Whether squared length is at most the squared tolerance.
`a.normalized(tolerance)`	New unit vector; throws for a length at or below the tolerance.
`a.normalize()`	Normalize in place using the default tolerance; unchanged if it fails.
`a.component_product(b)`	Product of corresponding components.
`a.min_component()`, `a.max_component()`	Smallest or largest signed component.
`a.approximately_equal(b, tolerance)`	Euclidean component-space difference within tolerance.
`lerp(a, b, t)`	$(1 - t)a + tb$; extrapolation is permitted.

`zero()`, `unit_x()`, and `unit_y()` are static vector constructors; `vector3` also has `unit_z()`. `default_tolerance()` is 64 times the machine epsilon of `REAL`. This absolute tolerance is a convenient arithmetic default; applications should supply a meaningful length tolerance when their scale requires it.

In 2D, $a \times b = a_x b_y - a_y b_x$ is positive when b lies counterclockwise from a through an angle below π. `perpendicular_left()` returns $(-a_y, a_x)$ and `perpendicular_right()` returns $(a_y, -a_x)$; neither normalizes. In 3D, the cross product follows the right-hand rule.

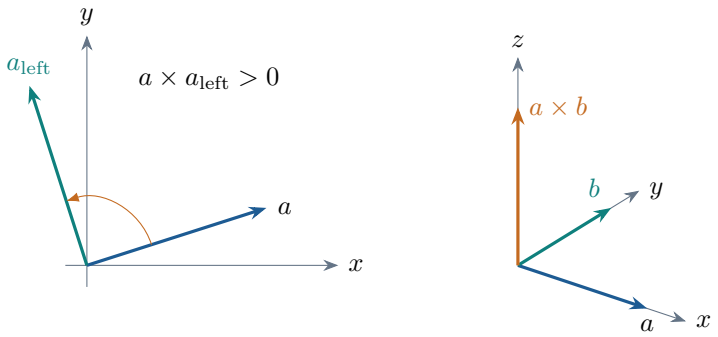


Figure 2.2: The 2D perpendicular operation and the 3D cross product have different return types. Both retain orientation information.

2.2.2 Projection, rejection, reflection, and angles

For a nonzero axis b, the member `a.projected_onto(b, tolerance)` computes

$$a_{\parallel} = \frac{a \cdot b}{b \cdot b}\, b, \qquad a_{\perp} = a - a_{\parallel}.$$

`a.rejected_from(b, tolerance)` returns $a_{\perp}$. The axis need not be normalized. Both calls reject an axis whose squared length is at or below the squared tolerance.

`a.reflected(normal, tolerance)` computes $a - 2(a \cdot \hat{n})\hat{n}$. Its argument is the normal to the reflecting line in 2D or plane in 3D. Passing the direction of the reflecting line would describe a different reflection. A near-zero normal throws `std::domain_error`.

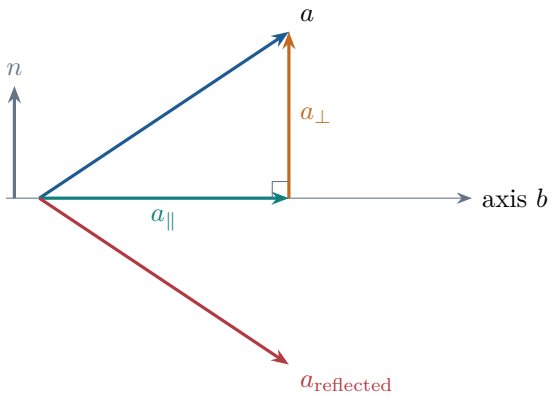


Figure 2.3: Projection and rejection decompose a vector. Reflection across the horizontal line reverses the normal component and retains the parallel component.

`angle_to(other, tolerance)` returns an unsigned angle in radians between zero and π, using a clamped cosine. Its degeneracy test compares the product of the lengths to the square of the supplied tolerance. `vector2::signed_angle_to(other, tolerance)` instead returns the counterclockwise signed angle in $[-\pi, \pi]$, rejecting either individual direction when it is too small. There is no signed 3D angle without an additional orientation convention.

```
const vector2<double> velocity{3.0, 2.0};
const auto along = velocity.projected_onto(vector2<double>{1.0, 0.0});
const auto across = velocity.rejected_from(vector2<double>{1.0, 0.0});
const auto bounced = velocity.reflected(vector2<double>{0.0, 1.0});
const double turn = vector2<double>::unit_x().signed_angle_to(velocity);
```

2.3 Rotation about a 3D origin axis

The free function `rotate(axis, angle, point)` rotates a `point3<REAL>` about a direction through the Cartesian world origin. The axis need not be normalized and the signed angle is in radians, following the right-hand rule. An exactly zero axis throws `std::domain_error`. Finite components, point coordinates, and angle are required for a finite result. The function uses Rodrigues' formula:

$$R(p) = \cos\theta\, p + \sin\theta\, (\hat{a} \times p) + (1 - \cos\theta)(\hat{a} \cdot p)\hat{a}.$$

For rotation about an axis through a different point C, explicitly shift the position to the origin, rotate it, and shift it back:

```
const point3<double> center{2.0, 1.0, 0.0};
const point3<double> position{3.0, 1.0, 0.0};
const vector3<double> offset = position - center;
const point3<double> relative{offset.x, offset.y, offset.z};
const auto rotated_relative = rotate(
    vector3<double>::unit_z(), std::numbers::pi / 2.0, relative);
const auto rotated = center + vector3<double>{rotated_relative};
```

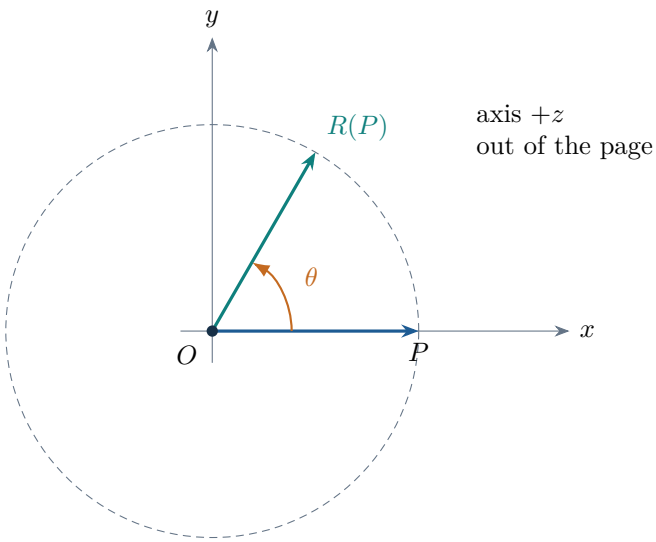


Figure 2.4: Positive rotation about $+z$ appears counterclockwise when viewed from the positive-Z side. The public rotation call is for a 3D point.

2.4 Rays and their native parameter

Construct a ray with an origin, direction, and optional minimum direction length. The `ray2` and `ray3` APIs have the same forms:

```
ray2<double> ray_2d(point2<double>{1.0, 1.0},
                    vector2<double>{2.0, 0.5});
ray3<double> ray_3d(point3<double>{1.0, 1.0, 0.0},
                    vector3<double>{2.0, 0.5, 0.0});
const auto at_two = ray_3d.point_at(2.0);
const auto same_point = ray_3d.evaluate(2.0);
const auto unit_tangent = ray_3d.tangent();
```

The native parameterization is

$$r(s) = O + sD, \qquad s \geq 0.$$

Construction preserves D exactly rather than normalizing it. A parameter change Δs therefore moves through a world distance $|\Delta s|\,\|D\|$; s is itself a distance only for a unit direction. `point_at` and its alias `evaluate` evaluate the formula even for a negative argument, but ray intersection and distance queries use the forward half-line. `tangent()` returns the normalized positive direction without changing the stored ray.

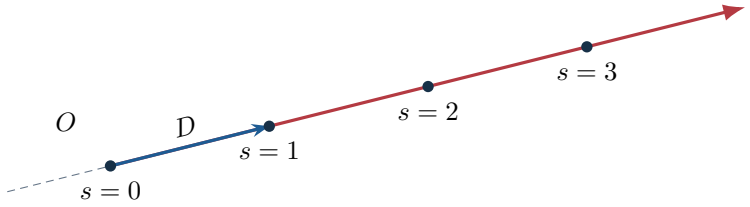


Figure 2.5: Uniform increments in s follow the unnormalized direction. The dashed continuation behind the origin is outside the ray domain.

`origin()` and `direction()` return constant references; `get_origin()` and `get_direction()` are equivalent getters. Use `set_origin`, `set_direction`, or `set_origin_and_direction` to edit the definition. The direction and bulk setters validate before committing, leaving the ray unchanged when the new direction is rejected. The constructor and these setters throw `std::invalid_argument` when the direction is within the supplied tolerance of zero. Origin setters also accept a same-world vector and copy its components as coordinates.

2.5 Circle and sphere surfaces

`circle2(center, radius)` and `sphere3(center, radius)` require a finite, strictly positive radius. Both expose `center()` and `radius()`, with equivalent `get_center()` and `get_radius()` forms. A circle remains a native 2D entity; a sphere is a 3D surface.

2.5.1 A circle's angular coordinate

For center C and radius R, `circle2::point_at(u)` evaluates

$$C + R(\cos u, \sin u).$$

The angle is in radians and increases counterclockwise from positive X. `parameter_of(point, tolerance)` obtains $u \in [0, 2\pi)$ by radial projection, so the input need not lie on the circle. It throws `std::domain_error` within the supplied center-distance tolerance and `std::invalid_argument` for a negative tolerance. `normal_at(point)` returns the normalized radial vector and rejects the exactly central point.

```
circle2<double> boundary(point2<double>{1.0, 0.0}, 2.0);
const auto p = boundary.point_at(std::numbers::pi / 3.0);
const double u = boundary.parameter_of(p);
const auto outward = boundary.normal_at(p);
```

2.5.2 Longitude and latitude on a sphere

`sphere3::point_at(u, v)` evaluates

$$C + R(\cos v \cos u,\ \cos v \sin u,\ \sin v).$$

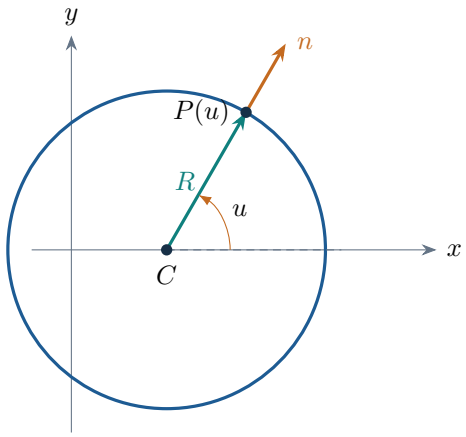


Figure 2.6: A circle's parameter is an angle, while its outward normal is a unit radial vector.

The reported longitude lies in $[0, 2\pi)$ and latitude in $[-\pi/2, \pi/2]$. Longitude is not geometrically unique at the poles. `parameters_of(point, tolerance)` obtains the pair by radial projection, with the same center rejection and negative-tolerance exceptions as the circle. `normal_at(point)` is the outward unit radial direction and throws at the center.

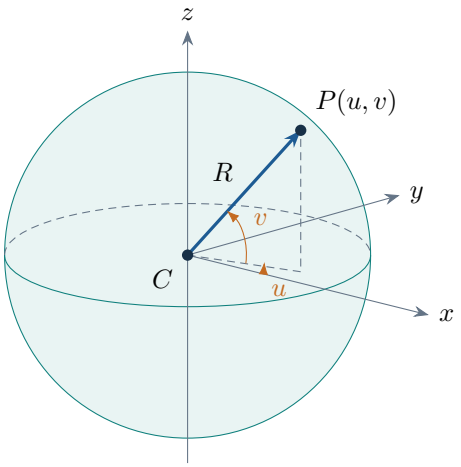


Figure 2.7: Sphere coordinates use longitude around the Z axis and latitude above the XY plane. This is a schematic 3D view.

```
sphere3<double> ball(point3<double>{0.0, 0.0, 0.0}, 2.0);
const auto surface_point = ball.point_at(
    std::numbers::pi / 4.0, std::numbers::pi / 6.0);
const auto [longitude, latitude] = ball.parameters_of(surface_point);
const auto normal = ball.normal_at(surface_point);
```

For either type, `set_center` preserves the radius, `set_radius` validates the new radius, and `set_center_and_radius` validates before updating either member. Failed radius validation leaves the object unchanged. Center setters and the bulk setter accept a same-world vector as a component source; assignment from such a vector also replaces the center while preserving the radius. Center-coordinate copying does not itself validate finiteness. Supply finite geometric coordinates when a later operation requires a finite result.

2.6 Planes: an orthonormal local frame

A `plane3` stores a world-space parameter origin O, a unit normal $\hat{n}$, and orthonormal directions $\hat{u}, \hat{v}$ with $\hat{u} \times \hat{v} = \hat{n}$. Its creation forms are:

```
const auto z = vector3<double>::unit_z();
plane3<double> hessian(z, 2.0);
plane3<double> positioned(point3<double>{0.0, 0.0, 2.0}, z);
plane3<double> oriented(point3<double>{0.0, 0.0, 2.0}, z,
                        vector3<double>::unit_x());
```

The Hessian form `plane3(normal, signed_distance, tolerance)` first normalizes the supplied normal. Its equation is $\hat{n} \cdot x = d$, and its parameter origin is $d\hat{n}$. The distance is signed in world units; it is not scaled by the magnitude of the original normal. A non-finite distance throws `std::invalid_argument`.

The origin-and-normal form places $(u, v) = (0, 0)$ at the given world point and chooses a stable in-plane frame. The third form adds a preferred `u_hint`: the constructor rejects its normal component and normalizes what remains. The normal and accepted in-plane hint must exceed the supplied tolerance, otherwise `std::domain_error` is thrown. Both forms retain the supplied parameter origin rather than moving it to the Hessian closest point.

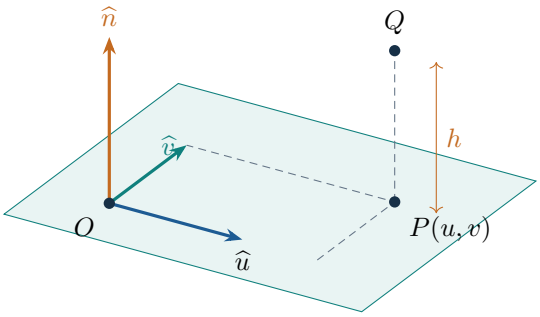


Figure 2.8: The orthonormal frame gives world-distance plane coordinates. Orthogonal projection removes the signed height h along the normal.

`point_at(u, v)` evaluates $O + u\hat{u} + v\hat{v}$. For any world point Q, `signed_distance_to(Q)` returns $h = (Q-O)\cdot\hat{n}$, `project(Q)` returns $Q - h\hat{n}$, and `parameters_of(Q)` returns $((Q-O)\cdot\hat{u}, (Q-O)\cdot\hat{v})$. Thus coordinates may be recovered from an off-plane point without first calling `project`. The member `project(point3)` is an orthogonal point projection within the 3D world. The free `project(plane3, entity2)` is the separate, explicit embedding operation.

```
const point3<double> query{1.0, 3.0, 5.0};
const double height = oriented.signed_distance_to(query);
const auto foot = oriented.project(query);
const auto [u, v] = oriented.parameters_of(query);
const auto reconstructed = oriented.point_at(u, v);
```

`origin()`, `normal()`, `u_direction()`, and `v_direction()` are constant-reference getters. Their `get_...` forms are equivalent. Two further equivalent getters report $\hat{n} \cdot O$:

- `signed_distance_from_origin()`;
- `get_signed_distance_from_origin()`.

`set_origin` moves the parameter origin and retains the frame. `set_normal` retains the origin

and generates a new stable frame; `set_u_direction` retains the origin and normal and rebuilds the two in-plane directions from a hint. `set_definition` has Hessian, origin-and-normal, and origin-normal-hint forms matching construction. These frame-changing setters build a complete candidate before committing it, so a rejected normal or hint leaves the original plane unchanged. Origin-based setters also accept a `vector3` whose components become coordinates. The normal and in-plane basis directions remain read-only through getters.

2.7 Reading the contracts consistently

Coordinate access and simple arithmetic are deliberately lightweight. They do not impose a universal finite-value check on every assignment. In contrast, radius validation, direction degeneracy checks, frame construction, NURBS definition validation, and record encoding have specific documented checks. A zero vector is a valid stored vector but cannot define a unit direction or plane normal. A point at a sphere or circle center is a valid position but has no radial surface coordinate.

Point and vector CSV, whitespace, binary, and selected 3D tagged I/O members are described with the complete output protocols later in this manual. Those methods do not change the geometric distinction between positions and displacements.

Chapter 3

Construction and shared ownership

The public constructors create values. The allocation helpers in `nurbspath/creators.hpp` create the corresponding values under `std::shared_ptr` ownership. They retain the constructors' geometric validation and exceptions. Both styles are useful: local calculations often use values, while a collection with shared lifetime may use factory results. The umbrella header includes these helpers.

```
#include <nurbspath/config.hpp>
#include <nurbspath/nurbspath.hpp>

using namespace nurbspath;
point3<double> value{1.0, 2.0, 3.0};
auto shared = make_point3<double>(1.0, 2.0, 3.0);
const double separation = distance(value, *shared);
```

The suffix identifies the Cartesian world. A 2D factory never silently allocates a 3D entity. Calls below use `double`; replacing it with another standard floating-point type changes the scalar type throughout. Allocation can throw `std::bad_alloc` in addition to the validation exceptions noted for each entity.

3.1 Point and vector factory catalogue

Table 3.1: All point and vector allocation families. Each result is a shared pointer to the type named after `make_`.

Call arguments after the template	Created value
`make_point2<REAL>()`	The 2D origin.
`make_point2<REAL>(x, y)`	A 2D position with the supplied coordinates.
`make_point3<REAL>()`	The 3D origin.
`make_point3<REAL>(x, y, z)`	A 3D position with the supplied coordinates.
`make_vector2<REAL>()`	The 2D zero vector.
`make_vector2<REAL>(x, y)`	The supplied 2D components.
`make_vector2<REAL>(point)`	Coordinates copied from a `point2`.
`make_vector2<REAL>(circle)`	Center coordinates copied from a `circle2`.
`make_vector2<REAL>(ray)`	Origin coordinates copied from a `ray2`.
`make_vector3<REAL>()`	The 3D zero vector.

Call arguments after the template	Created value
`make_vector3<REAL>(x, y, z)`	The supplied 3D components.
`make_vector3<REAL>(point)`	Coordinates copied from a `point3`.
`make_vector3<REAL>(sphere)`	Center coordinates copied from a `sphere3`.
`make_vector3<REAL>(plane)`	Parameter-origin coordinates copied from a `plane3`.
`make_vector3<REAL>(ray)`	Origin coordinates copied from a `ray3`.

The entity-to-vector factories perform the same explicit component copy as the constructors in Chapter 2. To allocate a copy of a ray direction, obtain `ray.direction()` and use its components, or use `std::make_shared<vector3<double»(ray.direction())`.

3.2 Ray, circle, and sphere creation

The origin-and-direction helpers are `make_ray2` and `make_ray3`. The two-point helpers are `make_ray2_from_points` and `make_ray3_from_points`. The first takes `(origin, direction, tolerance)`; the second takes `(origin, through_point, tolerance)`. Tolerance is optional and defaults to `vector2<REAL>::default_tolerance()` or its 3D counterpart. The point-based helper uses `through_point - origin` directly, with no normalization. The through point is therefore at native parameter $s = 1$.

```
const point2<double> a2{1.0, 1.0};
const point2<double> b2{3.0, 2.0};
auto ray_a = make_ray2<double>(a2, b2 - a2);
auto ray_b = make_ray2_from_points<double>(a2, b2);
auto circle = make_circle2<double>(point2<double>{0.0, 0.0}, 2.0);

const point3<double> a3{1.0, 1.0, 0.0};
const point3<double> b3{3.0, 2.0, 1.0};
auto ray_c = make_ray3<double>(a3, b3 - a3);
auto ray_d = make_ray3_from_points<double>(a3, b3);
auto sphere = make_sphere3<double>(point3<double>{0.0, 0.0, 0.0}, 2.0);
```

A direction or point separation within the ray tolerance throws `std::invalid_argument`. `make_circle2(center, radius)` and `make_sphere3(center, radius)` require finite, positive radii and throw `std::invalid_argument` otherwise. These are the complete circle and sphere factory families; they do not infer a circle or sphere from several boundary points.

3.3 Five ways to allocate a plane

The three `make_plane3` overloads mirror the three constructors:

1. `(normal, signed_distance, tolerance)` uses Hessian normal form.
2. `(origin, normal, tolerance)` chooses a stable in-plane frame.
3. `(origin, normal, u_hint, tolerance)` sets the preferred positive-u direction by in-plane projection.

Two named helpers derive an oriented frame from construction geometry:

- `make_plane3_from_u_direction` takes `origin`, `u_direction`, `plane_point`, and optional `tolerance`.
- `make_plane3_from_points` takes `origin`, `u_point`, `plane_point`, and optional `tolerance`; it uses `u_point - origin` as its positive-u direction.

The third point determines the positive-v side by rejection from that axis:

$$\widehat{u} = \frac{U}{\|U\|}, \quad V = (P - O) - ((P - O) \cdot \widehat{u})\widehat{u}, \quad \widehat{v} = \frac{V}{\|V\|}, \quad \widehat{n} = \widehat{u} \times \widehat{v}.$$

All five helpers accept the optional vector tolerance. A short normal, u-direction, or rejected v-component causes a `std::domain_error` exception. A non-finite Hessian distance causes `std::invalid_argument`.

```
const point3<double> origin{1.0, 2.0, 3.0};
const point3<double> u_point{3.0, 2.0, 3.0};
const point3<double> plane_point{2.0, 4.0, 3.0};
auto hessian = make_plane3<double>(vector3<double>{0.0, 0.0, 2.0}, 3.0);
auto from_normal = make_plane3<double>(origin, vector3<double>::unit_z());
auto with_hint = make_plane3<double>(
    origin, vector3<double>::unit_z(), vector3<double>::unit_x());
auto from_direction = make_plane3_from_u_direction<double>(
    origin, vector3<double>{2.0, 0.0, 0.0}, plane_point);
auto from_points = make_plane3_from_points<double>(origin, u_point, plane_point);
```

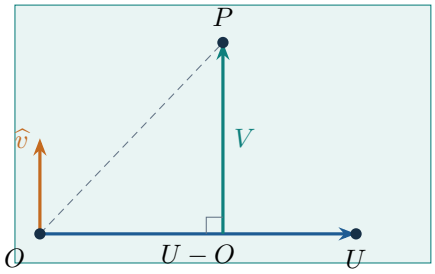


Figure 3.1: Three-point plane construction viewed in the plane itself. The third point chooses the positive-v side; its along-u component is discarded when the frame is built.

Moving the third point across the u-axis reverses both $\widehat{v}$ and $\widehat{n}$. Three collinear points do not determine a plane through this interface. A supplied normal or u-direction need not have unit length.

3.4 Ordinary spline constructors and factories

The following catalogue applies independently to `nurbs_spline2` and `nurbs_spline3`. Write `S` for either type, `P` for its same-world point type, `A` for the corresponding array-backed type, `p` for the degree, `n` for the control-point count, and `tol` for the positive definition tolerance. The factories `make_nurbs_spline2` and `make_nurbs_spline3` return `std::shared_ptr<S>`.

The omitted spline tolerance is `64 * std::numeric_limits<REAL>::epsilon()`. The standard constructor places n control points uniformly from start to end with unit weights and open-clamped knots over $[0, \text{distance}(\text{start}, \text{end})]$. It creates a straight curve, with a positive domain extent required by validation. The explicit-tolerance fifth argument is a scalar; the closure overload requires an actual `bool`, avoiding accidental interpretation of a tolerance as a closure flag.

Table 3.2: Complete ordinary-spline construction families. Optional brackets describe permitted omitted arguments, not C++ syntax.

Argument sequence	Availability and behavior
`start, end, n, p`	Constructor and factory; standard open definition.
`start, end, n, p, tol`	Constructor and factory; explicit tolerance.
`start, end, n, p, closed [, tol]`	Constructor and factory; explicit Boolean closure request.
`points, weights, knots, p [, tol]`	Constructor and factory with `std::vector` collections.
`points, weights, knots, p, closed [, tol]`	Constructor and factory with vector collections and a closure request.
`points, weights, knots, p [, tol]`	Factory also accepts point/scalar `std::valarray` collections.
`points, weights, knots, p, closed [, tol]`	Factory also accepts valarrays and a closure request.
`array_spline`	Explicit constructor and factory; copy from corresponding `A`.

```
const point2<double> start{0.0, 0.0};
const point2<double> end{4.0, 0.0};
nurbs_spline2<double> line(start, end, 5, 2);
auto shared_line = make_nurbs_spline2<double>(start, end, 5, 2);
auto explicit_tolerance = make_nurbs_spline2<double>(start, end, 5, 2, 1e-10);
auto explicit_open = make_nurbs_spline2<double>(start, end, 5, 2, false, 1e-10);
```

Complete definitions supply every control point, positive weight, and knot. For n control points and degree p, the knot count is $n + p + 1$, with $0 < p < n$, nondecreasing finite knots, and a nonempty active domain. The standard and complete-definition families have matching validation contracts; the full mathematical requirements are in Chapter 4.

```
std::vector<point3<double>> points{
    {0.0, 0.0, 0.0}, {1.0, 2.0, 0.5}, {3.0, 0.0, 1.0}};
std::vector<double> weights{1.0, 1.0, 1.0};
std::vector<double> knots{0.0, 0.0, 0.0, 2.0, 2.0, 2.0};
nurbs_spline3<double> value_curve(points, weights, knots, 2);
auto shared_curve = make_nurbs_spline3<double>(points, weights, knots, 2);

std::valarray<point3<double>> array_points{points.data(), points.size()};
std::valarray<double> array_weights{weights.data(), weights.size()};
std::valarray<double> array_knots{knots.data(), knots.size()};
auto copied_arrays = make_nurbs_spline3<double>(
    array_points, array_weights, array_knots, 2, false);
```

The ordinary constructors take vectors. The ordinary factories additionally accept const, mutable, lvalue, or rvalue valarrays of same-world points plus scalar valarrays; they copy them in index order into ordinary vector storage. Brace-list calls remain available through the vector overloads.

3.4.1 Closure describes a seam

A `closed = true` request requires coincident evaluated endpoints of the active domain within the spline tolerance. It does not append a control point or synthesize a loop from a straight start/end construction. Use a complete definition or closed interpolation samples to create a useful closed shape.

```
auto closed_polygon = make_nurbs_spline2<double>(
    std::vector<point2<double>>{
        {0.0, 0.0}, {2.0, 0.0}, {1.0, 1.5}, {0.0, 0.0}},
    std::vector<double>{1.0, 1.0, 1.0, 1.0},
    std::vector<double>{0.0, 0.0, 1.0, 2.0, 3.0, 3.0},
    1, true);
```

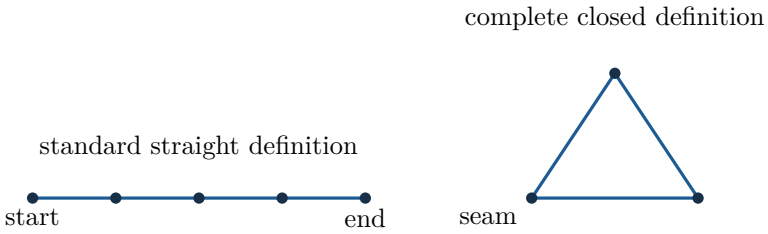


Figure 3.2: The standard factory places control points along a segment. A closed flag validates a seam; the supplied definition determines the shape.

3.5 Array-backed spline construction

`nurbs_arr_spline2` and `nurbs_arr_spline3` are separate classes, not derived ordinary splines. Their public `std::valarray<REAL> parameters` contains point-major control coordinates, then all weights, then all knots. The degree, tolerance, and closure state are stored separately. This layout is useful when an optimizer must copy or replace the entire scalar state. The conversion operations retain the Cartesian world.

For dimension d, n controls, and degree p, the flattened length is $(d+2)n+p+1$. In 2D it is

$$[x_0, y_0, \ldots, x_{n-1}, y_{n-1}, w_0, \ldots, w_{n-1}, k_0, \ldots, k_{n+p}],$$

and in 3D each point contributes X/Y/Z coordinates.

The matching factory names are `make_nurbs_arr_spline2` and `make_nurbs_arr_spline3`. The component collection overloads take `std::vectors`; a flattened overload takes one scalar `std::valarray`. The point-valarray overloads listed for ordinary factories are not separate array-backed factory overloads. The constructors and factories reject malformed array shapes before indexing the data.

```
std::valarray<double> parameters{
    0.0, 0.0,  1.0, 2.0,  3.0, 0.0,  // three 2D controls
    1.0, 1.0, 1.0,                    // weights
    0.0, 0.0, 0.0, 2.0, 2.0, 2.0};  // quadratic knots
nurbs_arr_spline2<double> array_curve(parameters, 2);
auto shared_array = make_nurbs_arr_spline2<double>(parameters, 2, false);
```

Table 3.3: Complete array-backed construction families, for both dimensions. Direct constructors and shared-pointer factories support the same families.

Argument sequence	Created definition
`start, end, n, p`	Standard open straight curve.
`start, end, n, p, tol`	Standard curve with explicit tolerance.
`start, end, n, p, closed [, tol]`	Standard curve with a Boolean closure request.
`parameters, p [, tol]`	Open definition from flattened scalar valarray.
`parameters, p, closed [, tol]`	Flattened definition with a Boolean closure request.
`points, weights, knots, p [, tol]`	Open definition from component vectors.
`points, weights, knots, p, closed [, tol]`	Component-vector definition with a Boolean closure request.
`ordinary_spline`	Independent copy from corresponding ordinary spline.

3.6 Conversions and definition updates

Conversions between the ordinary and array-backed representations are explicit, copying control points, weights, knots, degree, tolerance, closure, and the native parameter domain. They allocate independent storage:

```
auto ordinary = make_nurbs_spline2<double>(*shared_array);
auto flattened = make_nurbs_arr_spline2<double>(*ordinary);
nurbs_spline2<double> ordinary_value(*flattened);
nurbs_arr_spline2<double> array_value(ordinary_value);
```

The same calls are available for the 3D pair. These conversions never add, remove, or reinterpret a coordinate dimension. Converting an array-backed definition validates its current public storage; a malformed state throws rather than yielding a partially constructed spline.

Ordinary spline definition storage is private. Checked setters preserve the degree, validate a complete proposed state, and refresh cached endpoints only after success. A failed update leaves the spline unchanged. The array-backed alternatives expose scalar storage and derive endpoint state when needed; direct edits must be followed by `validate()` or a checked geometry operation. Chapter 4 gives the complete editing and evaluation APIs for both representations.

Chapter 4

NURBS curves and interpolation

4.1 One curve definition, two separate worlds

The ordinary curve types are `nurbs_spline2<REAL>` and `nurbs_spline3<REAL>`. Their coordinate storage is private, their definition is checked at construction and after each setter, and their active-domain endpoints are cached. The corresponding `nurbs_arr_spline2<REAL>` and `nurbs_arr_spline3<REAL>` types provide a different storage contract: a public, flattened `std::valarray<REAL>` that an optimizer can edit directly. All four types describe rational B-spline curves and provide position, derivative, tangent, curvature, length, and interpolation operations.

The suffix is part of the geometry contract. A 2D curve evaluates to `point2<REAL>` and differentiates to `vector2<REAL>`; a 3D curve uses the corresponding 3D types. Constructing an array-backed curve from an ordinary curve, or converting it back, preserves that suffix. Embedding a 2D curve into 3D is an explicit `project(plane, spline)` operation, described in Chapter 2.

For a curve with C control points, degree p, weights w_i, and knots K_j, the complete definition satisfies

$$C > p \geq 1, \qquad \#w = C, \qquad \#K = C + p + 1, \qquad w_i > 0, \qquad K_j \leq K_{j+1}. \tag{4.1}$$

Coordinates, weights, knots, and the strictly positive definition tolerance must be finite. The knot domain used by the curve is

$$\boxed{s \in [K_p, K_C] = [\texttt{s_min()}, \texttt{s_max()}]} \tag{4.2}$$

and must have positive length. It is the *active* domain; the first and last values in the stored knot vector need not be its bounds.

Native parameters

The parameter is named s, but a general NURBS definition does not make s equal to distance traveled. Keep the native knot units when passing s to evaluation, intersections, distances, or derivative functions. Interpolation preserves the supplied station values, including a nonzero first station.

4.2 Construction from a complete definition

The following fragment establishes a running example in the native 2D world. It uses five control points, cubic basis functions, nonunit weights, and a native domain of $[0, 5]$. Code fragments in this chapter use the umbrella header `<nurbspath/nurbspath.hpp>`; standard-library facilities need their usual headers. The namespace alias `np` below is used throughout the chapter.

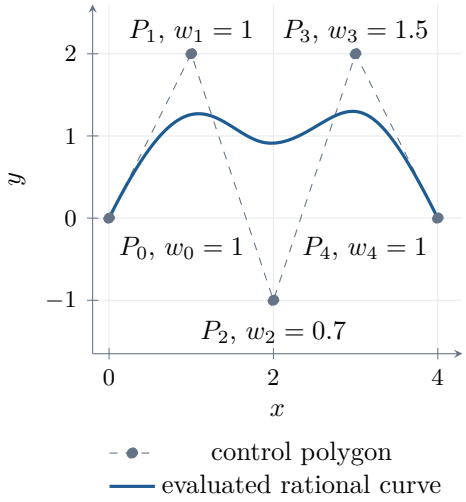


Figure 4.1: Control points steer the curve; they are generally not samples on it. The blue path is evaluated by the library. The clamped endpoint blocks in this example make the evaluated endpoints coincide with P_0 and P_4.

Listing 4.1: A complete ordinary 2D curve definition.

```
namespace np = nurbspath;
using real = double;
using point = np::point2<real>;

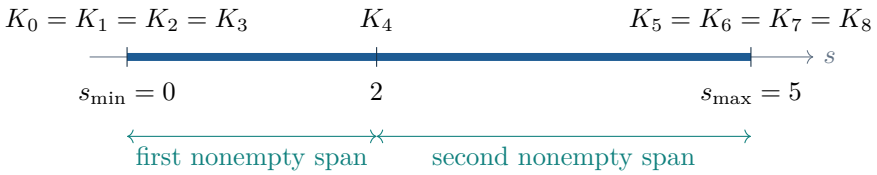
const std::vector<point> controls{
    {0, 0}, {1, 2}, {2, -1}, {3, 2}, {4, 0}};
const std::vector<real> weights{1, 1, 0.7, 1.5, 1};
const std::vector<real> knots{0, 0, 0, 0, 2, 5, 5, 5, 5};

np::nurbs_spline2<real> curve(
    controls, weights, knots, 3, false, 1e-10);
const point start = curve.get_start();
const point end = curve.get_end();
const point middle = curve.point_at(2.5);
```

The complete-definition constructors accept three `std::vectors` followed by the degree. Omitting the closure argument constructs an open curve; an optional tolerance follows the degree in that overload. Supplying `closed` before the tolerance enables the seam requirement. In 3D, replace `point2` and `nurbs_spline2` with their 3D counterparts. There is no default empty spline object: construction establishes a usable definition.

An open curve is allowed to have coincident endpoints. A closed curve requires that coincidence, but does not by itself guarantee a smooth seam, matching tangent vectors, matching curvature, or a periodic knot representation. For the ordinary definition check, endpoint separation must not exceed $16\,\tau\,\max(1, |\text{control coordinates}|)$, where τ is the definition tolerance. Closed interpolation also checks its first and last input samples against the supplied tolerance before solving.

The getter families `get_control_points()`, `get_weights()`, and `get_knots()` return constant references for ordinary splines. `get_control_point(i)`, `get_weight(i)`, and `get_knot(i)` select individual values; an invalid index throws `std::out_of_range`. `degree()`, `tolerance()`, and `is_closed()` report metadata. `get_start()` and `get_end()` return constant references to evaluated, cached endpoints, not editable definition fields.



Cubic running example: $C = 5$, $p = 3$, nine stored knots.
The two spans have different native parameter lengths.

Figure 4.2: Stored knots and the active domain. Endpoint multiplicities describe clamping; they are not extra finite intervals to sample.

4.2.1 Starting with a standard straight curve

When only endpoints and a number of controls are known, the standard constructor lays the control points uniformly along the segment using `lerp`, assigns unit weights, and builds standard open-clamped knots. Its native domain is $[0, \text{distance}(\text{start}, \text{end})]$. The degree must be positive and smaller than the requested control count. The endpoints must define a finite, positive domain length.

Listing 4.2: Standard construction and knot replacement.

```
np::nurbs_spline3<real> straight(
    np::point3<real>{0, 0, 0},
    np::point3<real>{4, 0, 3}, 6, 3);
// Native domain is initially [0, 5].
straight.set_standard_knots(10.0, 20.0);
// Explicit-tolerance overload, without a closure argument:
np::nurbs_spline2<real> straight2(
    point{0, 0}, point{4, 0}, 5, 2, 1e-10);
straight2.set_standard_knots(8.0);  // Domain [0, 8].
```

`set_standard_knots(s_end)` uses $[0, s_{\text{end}}]$. `set_standard_knots(s_start,s_end)` uses the stated bounds. Both repeat each endpoint $p + 1$ times and distribute any simple interior knots uniformly. If $C = p + 1$, the two endpoint blocks form the entire knot vector. The generated spans must remain distinct and have finite positive lengths with finite reciprocals in `REAL`. Invalid domains or numerically unusable spans are rejected before the curve changes.

Replacing standard knots can change the distribution of points along an existing curve. If the intent is only to rescale its parameter, apply one affine transformation to *every* old knot and pass the complete result to `set_knots`; this preserves the geometric locus while changing its native s values and derivative scale. A closure flag on the straight constructor still requires the ordinary endpoint check; it does not invent a loop between two different endpoints.

4.3 Weights and exact conic geometry

For basis functions $N_{i,p}$, a rational curve is

$$\mathbf{C}(s) = \frac{\sum_{i=0}^{C-1} N_{i,p}(s) w_i \mathbf{P}_i}{\sum_{i=0}^{C-1} N_{i,p}(s) w_i}. \tag{4.3}$$

The library evaluates only the nonzero basis functions in the selected knot span. With positive weights, a control point with a larger relative weight has more influence where its basis function is nonzero. Uniformly multiplying the weights leaves the mathematical quotient unchanged, but a

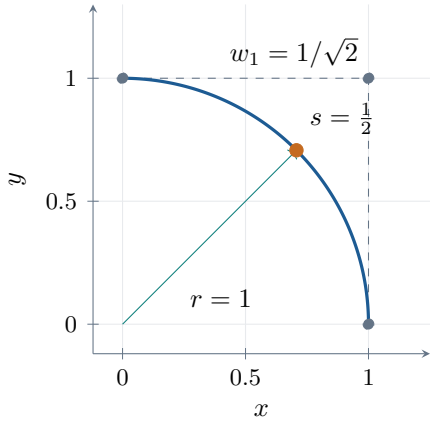


Figure 4.3: A quadratic rational curve lies on the unit circle, even though its control polygon follows the two sides of a square. Its curvature is one in inverse world units. The plotted blue points are library evaluations.

very small common scale can trigger the implementation's near-zero homogeneous-weight check. In particular, positivity alone is not a guarantee that every arbitrarily scaled floating-point definition can be evaluated.

A rational quadratic represents a quarter of the unit circle exactly in real arithmetic. It uses three controls, with the corner control carrying weight $1/\sqrt{2}$. The native parameter below is $s \in [0, 1]$; it is neither the circle angle u nor the traveled arc length.

Listing 4.3: An exact rational quadratic quarter circle.

```
np::nurbs_spline2<real> quarter(
    std::vector<point>{{1, 0}, {1, 1}, {0, 1}},
    std::vector<real>{1, std::sqrt(0.5), 1},
    std::vector<real>{0, 0, 0, 1, 1, 1}, 2);
const point diagonal = quarter.evaluate(0.5);
const real quarter_curvature = quarter.curvature(0.5);
const auto rational_third = quarter.third_derivative(0.5);
```

Rounded output at the midpoint is

Listing 4.4: Rounded numerical preview; scientific precision is four decimals.

```
position: 7.0711e-01  7.0711e-01
curvature: 1.0000e+00
```

Although the homogeneous numerator and denominator have degree two within a span, their quotient can have a nonzero third derivative. Calling `third_derivative` on this quadratic is therefore meaningful; setting it to zero solely because $p = 2$ would be incorrect.

4.4 Evaluation and analytic differential geometry

`evaluate(s)` and `point_at(s)` are equivalent position calls. `derivatives_at(s)` computes position, first derivative, and second derivative together. The result is `spline_derivatives2<REAL>` or `spline_derivatives3<REAL>`, with members `point`, `first`, and `second`. Separate calls are available as `first_derivative(s)`, `second_derivative(s)`, and `third_derivative(s)`.

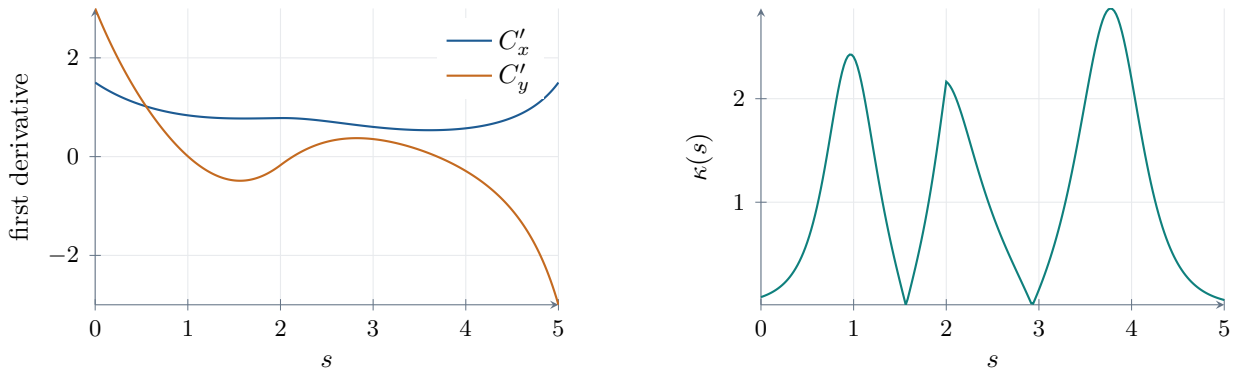


Figure 4.4: Analytic first-derivative components and curvature for the running cubic curve. Speed and turning vary through the native domain. The plotted values come from `first_derivative` and `curvature`.

Listing 4.5: Position, derivatives, tangent, and curvature.

```
const real query_s = 2.5;
const auto jet = curve.derivatives_at(query_s);
const auto velocity = jet.first;
const auto acceleration = jet.second;
const auto third = curve.third_derivative(query_s);
const auto unit_tangent = curve.tangent(query_s);
const real curvature = curve.curvature(query_s);
// A consumer chooses how long a derivative arrow should appear:
const point arrow_tip = jet.point + 0.25 * velocity;
```

Let $\mathbf{A}^{(k)} = \sum_i N_{i,p}^{(k)} w_i \mathbf{P}_i$ and $W^{(k)} = \sum_i N_{i,p}^{(k)} w_i$. Analytic differentiation is performed in homogeneous form and then converted with the quotient recurrence

$$\mathbf{C}^{(0)} = \mathbf{A}^{(0)} / W^{(0)}, \tag{4.4}$$

$$\mathbf{C}^{(r)} = \frac{1}{W^{(0)}} \left(\mathbf{A}^{(r)} - \sum_{j=1}^{r} \binom{r}{j} W^{(j)} \mathbf{C}^{(r-j)} \right), \qquad r = 1, 2, 3. \tag{4.5}$$

Homogeneous derivatives above the degree are zero, while the quotient recurrence continues through the requested order. These are analytic basis derivatives, rather than finite differences between neighboring points.

The derivatives are with respect to the native s. If s has units of length, the first derivative is dimensionless, the second has inverse-length units, and the third has inverse-square-length units. If s is dimensionless, all three retain world-length units. More generally, $\mathbf{C}^{(r)}$ has units of world length divided by parameter units to power r. Scaling a derivative arrow for a diagram does not change these API meanings.

4.4.1 Tangent and curvature contracts

`tangent(s,tolerance)` normalizes the first derivative and throws `std::domain_error` when its length is too small. Its default threshold is the matching vector type's `default_tolerance()`. This is a derivative length threshold; it is distinct from the spline definition tolerance and from the world-distance tolerances used by geometric queries.

`curvature(s,tangent_tolerance)` returns an unsigned magnitude:

$$\kappa_2 = \frac{|\mathbf{C}' \times \mathbf{C}''|}{\|\mathbf{C}'\|^3}, \qquad \kappa_3 = \frac{\|\mathbf{C}' \times \mathbf{C}''\|}{\|\mathbf{C}'\|^3}. \tag{4.6}$$

The 2D cross product in this formula is a scalar. Curvature has inverse world-length units and is independent of regular reparameterization while the required arithmetic remains representable. It does not encode the sign of left or right turning. The threshold must be finite and nonnegative; an invalid threshold throws `std::invalid_argument`. A nonfinite or insufficient first-derivative length, a near-zero homogeneous weight, or an unrepresentable curvature throws `std::domain_error`.

4.4.2 Domain edges and repeated knots

A finite s just outside the active domain by no more than the definition tolerance is clamped to the nearest bound. A nonfinite s, or a larger excursion, throws `std::out_of_range`. This tolerance is a boundary allowance, not an instruction to extrapolate the spline.

At an internal knot, a derivative that is discontinuous uses the right-hand nonempty span. At `s_min()`, evaluation chooses the first nonempty span on the right, and at `s_max()` it chooses the last nonempty span on the left. Repeated boundary knots can leave empty spans; those spans are skipped. This convention is especially relevant when plotting second or third derivatives near knots with reduced continuity. A single returned derivative at such a knot must not be mistaken for a two-sided smooth derivative.

4.5 Global interpolation through sample positions

Use the static `interpolate(samples,stations,...)` factory when positions are measured points that the result should pass through. The positions are *samples*, not control points. Interpolation solves a global linear system for a new set of control points and assigns unit weights. It produces a valid non-rational subset of the NURBS representation, with averaged clamped knots.

Listing 4.6: Interpolation preserves the supplied station domain.

```
const std::vector<point> samples{
    {0, 0}, {1, 1.2}, {2, 0.2}, {3, 1.5}, {4, 1}};
const std::vector<real> stations{2, 3.2, 4.7, 6.5, 8};
auto fit = np::nurbs_spline2<real>::interpolate(
    samples, stations, 3, false, 1e-10);

for (std::size_t i = 0; i < samples.size(); ++i) {
    const point fitted_sample = fit.evaluate(stations[i]);
    const real residual = np::distance(fitted_sample, samples[i]);
    // Inspect residual in the same 2D world as the input.
    static_cast<void>(residual);
}
// fit.s_min() == 2; fit.s_max() == 8.
```

The input vectors must have equal size and contain at least two samples. All samples and stations must be finite; station values must be strictly increasing. The requested degree must be positive, and the actual degree is

$$p = \min(\text{requested degree}, \text{sample count} - 1). \tag{4.7}$$

The default requested degree is three and the default interpolation tolerance is 10^{-10}. Invalid samples, stations, closure, or tolerance throw `std::invalid_argument`; a singular interpolation system throws `std::domain_error`. Floating-point residuals should be assessed with a suitable tolerance rather than exact coordinate equality.

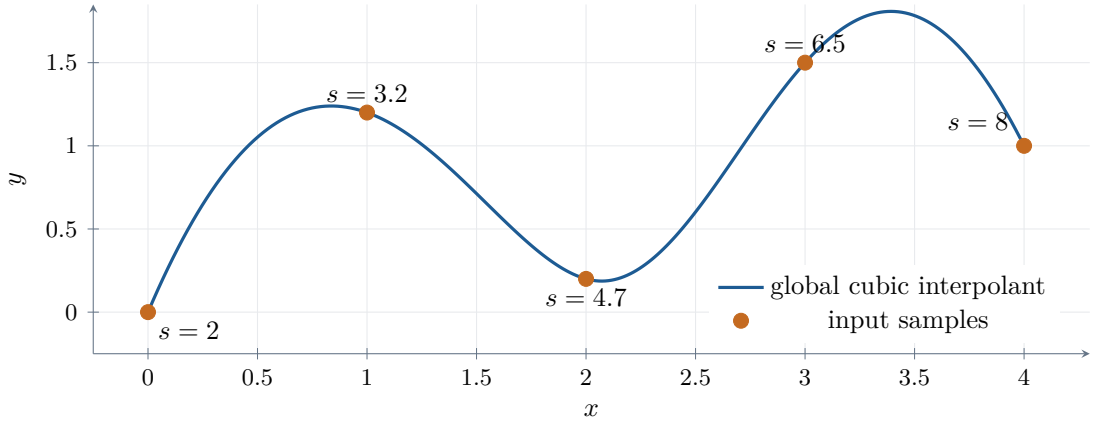


Figure 4.5: A curve through measured positions. Each annotation is the original native station, not a newly normalized coordinate. The library supplies the evaluated curve and sample positions used in this TikZ picture.

Let the sample stations be $t_0, \ldots, t_n$. The clamped endpoint blocks contain $p+1$ copies of t_0 and t_n. The simple interior knots are averages,

$$K_{j+p} = \frac{t_j + t_{j+1} + \cdots + t_{j+p-1}}{p}, \qquad j = 1, \ldots, n-p. \tag{4.8}$$

For the example, the sole interior knot is $(3.2 + 4.7 + 6.5)/3 = 4.8$. Solving $\sum_i N_{i,p}(t_j)\mathbf{P}_i = \mathbf{Q}_j$ independently for each coordinate makes all samples influence the global result. Adding or moving one sample can therefore change more than the adjacent visible arc.

Stations remain stations

The caller supplies the arc-length stations; interpolation does not compute them from the sample positions. Their values are retained, without silently normalizing the domain. Even if the stations describe measured arc length, the interpolated curve between stations does not promise unit speed or an exact continuous arc-length parameterization.

4.5.1 Closed input and replacing an existing curve

The overload with `closed=true` requires coincident first and final sample positions at *different*, strictly increasing station values. Repeat the closing position, not its station. This constructs a closed interpolant using the same averaged clamped-knot approach. It does not impose periodic derivatives at the seam.

Listing 4.7: Closed interpolation and adopting a new sample set.

```
const std::vector<point> loop_samples{
    {0, 0}, {1, 0}, {1, 1}, {0, 1}, {0, 0}};
const std::vector<real> loop_stations{0, 1, 2, 3, 4};
auto loop = np::nurbs_spline2<real>::interpolate(
    loop_samples, loop_stations, 3, true, 1e-10);

auto refitted = curve;
refitted.adopt_to_points(samples, stations, 2, false, 1e-10);
// Unlike definition setters, adoption creates a new degree-two fit.
```

`adopt_to_points` has the same open and closed overload families as `interpolate`. It first constructs the interpolant and then replaces the current object. A failed solve leaves the original object available.

Its requested degree, tolerance, and closure describe the replacement; they are not constrained to the old curve's metadata. Both array-backed curve types also supply these interpolation and adoption operations in their own world.

4.6 Two different notions of length

`get_polygon_length()` sums distances between consecutive stored control points:

$$L_{\text{polygon}} = \sum_{i=1}^{C-1} \|\mathbf{P}_i - \mathbf{P}_{i-1}\|. \tag{4.9}$$

This is the length of the *control polygon*, not an integration of the curve. It depends on neither weights nor knots. Even for a closed curve, it does not add an extra last-to-first edge. For ordinary splines this getter requires no evaluation; for array-backed splines it first validates the current public definition.

`approximate_arc_length(segment_count)` instead evaluates a polyline at uniformly spaced native parameters, including both active-domain endpoints:

$$s_j = s_{\min} + \frac{j}{M}(s_{\max} - s_{\min}), \qquad L_M = \sum_{j=1}^{M} \|\mathbf{C}(s_j) - \mathbf{C}(s_{j-1})\|. \tag{4.10}$$

The default is $M = 512$; zero segments throw `std::invalid_argument`. The result is a sampling estimate in world units. It is not an adaptive quadrature result and provides no requested-error certificate. Tight turns or short features may need many more segments, especially when the native parameter allocates little interval length to them.

Listing 4.8: Compare a control polygon with sampled curve lengths.

```
const real polygon_length = curve.get_polygon_length();
const real coarse_length = curve.approximate_arc_length(16);
const real finer_length = curve.approximate_arc_length(512);
const real circle_length = quarter.approximate_arc_length(512);
// circle_length approaches pi/2 as the segments resolve the arc.
```

When comparing successive estimates, retain the sampling count along with the value. Agreement at two finite counts is useful evidence for that curve, but cannot establish that all continuous features were resolved. The same qualification applies to the sample-based geometric queries in Chapter 5.

4.7 Checked editing and flattened scalar parameters

Ordinary spline definitions are edited through setters. The single-value setters validate a complete detached candidate: `set_control_point` edits one position, `set_weight` one weight, and `set_knot` one knot. The vector replacements `set_control_points`, `set_weights`, and `set_knots` retain the other definition fields. `set_tolerance` and `set_closed` also recheck the complete candidate, including any seam requirement. None of these operations changes the degree.

`set_definition(control_points,weights,knots,closed,tolerance)` replaces every mutable definition field together while retaining the current degree. It is the appropriate call for changing correlated counts, rescaling an entire knot vector, or editing both sides of a closed seam. Successful edits refresh the cached endpoints; failed edits preserve the original curve. References, iterators, and pointers previously obtained from ordinary definition getters become invalid after a successful setter.

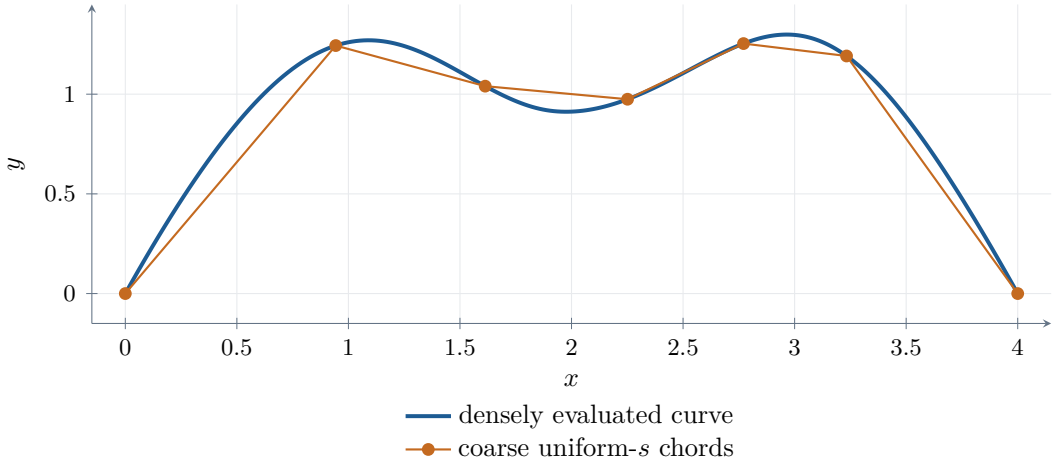
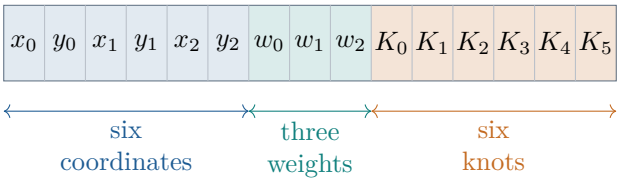


Figure 4.6: Arc-length sampling joins evaluated points with straight chords. Uniform native-parameter steps generally produce unequal world-space chord lengths. The coarse orange polyline illustrates the approximation, not an adaptive accuracy guarantee.

2D quadratic example: $C = 3$, $p = 2$, $N = 15$.



Scalar indices: coordinates $[0, 6)$, weights $[6, 9)$, knots $[9, 15)$.

Figure 4.7: Flattened numeric ordering for an ordinary spline's indexed parameters and an array-backed spline's public `parameters` storage. Typed metadata is stored separately.

Listing 4.9: Atomic bulk edits keep correlated changes together.

```
auto edited = curve;
auto edited_controls = edited.get_control_points();  // Detached copy.
auto edited_knots = edited.get_knots();              // Detached copy.
edited_controls.front() = {-1, 0};
edited_controls.back() = {5, 0};
for (real& knot : edited_knots) {
    knot = 10 + 2 * knot;
}
edited.set_definition(
    edited_controls, edited.get_weights(), edited_knots,
    edited.is_closed(), edited.tolerance());
// New endpoints and domain [10, 20] are immediately available.
```

4.7.1 An indexable view for fitting code

`number_of_parameters()`, `get_parameter(index)`, and `parameter_name(index)` expose the same flattened scalar ordering in all four spline types. Control coordinates come first in point-major order, followed by all weights, followed by all knots. In dimension d, the total is $(d + 2)C + p + 1$ scalars. Degree, tolerance, and closure are deliberately outside this scalar sequence. Names are `P[0].x`, `P[0].y`, optionally `P[0].z`, then later controls, `W[0]`, and finally `K[0]`, all with zero-based indices.

`set_parameter(index,value)` returns `true` only if the full edited candidate is accepted. It returns `false` for an invalid index or a rejected definition, retaining the original state. Allocation failures can still throw. This return-value convention differs from the named component setters, whose invalid candidate or index is reported by an exception.

`set_parameters(const REAL*)` copies exactly `number_of_parameters()` values in this ordering before validating. The pointer is not retained. It must be nonnull and address a sufficiently long readable buffer; the library cannot discover a raw pointer's extent. For a complete numeric change, prepare a detached vector and call once:

Listing 4.10: Indexed parameter names and one validated bulk copy.

```
std::vector<real> candidate(curve.number_of_parameters());
for (std::size_t i = 0; i < candidate.size(); ++i) {
    candidate[i] = curve.get_parameter(i);
    const std::string name = curve.parameter_name(i);
    static_cast<void>(name);
}
candidate[3] += 0.1;  // P[1].y in the 2D point-major layout.
auto scalar_edited = curve;
scalar_edited.set_parameters(candidate.data());
const bool accepted = scalar_edited.set_parameter(0, -0.2);
static_cast<void>(accepted);
```

There is no endpoint setter: an endpoint is derived from the definition. Change the appropriate control-point definition, then read the refreshed endpoint. Do not assume the first control point is always the active start: that familiar identity depends on clamped endpoint knots.

4.8 Array-backed curves for direct numeric work

The array-backed classes are independent types, not subclasses of the ordinary splines. They offer the same core geometry, checked setters, and interpolation calls, plus a public `parameters` array and layout/range queries. Their aliases `real_t` and `parameter_index_range` name the scalar type and the allocation-free range of flattened scalar indices.

Construction can use endpoints with a control count and degree, component vectors with a degree, a flattened `std::valarray<REAL>` with a degree, or an explicit copy from the corresponding ordinary spline. The optional closure and tolerance forms mirror these choices. The flattened constructor derives C from the scalar count N:

$$\begin{aligned} \text{2D:} \quad & N = 4C + p + 1, \\ \text{3D:} \quad & N = 5C + p + 1. \end{aligned} \tag{4.11}$$

The inferred count must be integral and exceed the degree. A 2D sequence cannot be interpreted as a 3D sequence simply by choosing another type.

Listing 4.11: A flattened quadratic and same-world conversions.

```
const std::valarray<real> flat_quarter{
    1, 0,  1, 1,  0, 1,       // Three XY control points.
    1, std::sqrt(0.5), 1,       // Three weights.
    0, 0, 0, 1, 1, 1};        // Six knots.
np::nurbs_arr_spline2<real> array_quarter(flat_quarter, 2);

np::nurbs_arr_spline2<real> array_curve(curve);
np::nurbs_spline2<real> vector_curve(array_curve);
```

```
auto vector_copy = array_curve.to_nurbs_spline();
const point array_point = array_curve.evaluate(2.5);
```

These conversions copy the complete definition and preserve control points, weights, knots, degree, tolerance, closure, and native domain. They validate the current source array and can fail if it was edited into an invalid state. The same operations exist between `nurbs_arr_spline3` and `nurbs_spline3`. For shared ownership, the matching `make_nurbs_spline2`, `make_nurbs_spline3`, `make_nurbs_arr_spline2`, and `make_nurbs_arr_spline3` allocation helpers are covered in Chapter 3. Ordinary allocation helpers also accept component valarrays; that convenience overload does not turn an ordinary curve into the flattened array-backed type.

4.8.1 Direct editing, validation, and range meanings

Direct array edits intentionally bypass checked setters. They can temporarily leave invalid weights, bad knot order, a broken closed seam, or a malformed array size. `validate()` checks the current layout, values, native domain, and required seam. Geometric operations perform their own current-state validation before using the array; they reject malformed storage before indexing through its inferred layout.

No endpoint or other geometry cache is retained. `get_start()`, `get_end()`, `s_min()`, and `s_max()` derive their answers from the current validated storage. Point getters reconstruct point values from their coordinates; they do not alias adjacent scalars as a point object. `get_control_points()`, `get_weights()`, and `get_knots()` return detached vectors, unlike the ordinary spline getters' constant references. Layout and component-access getters check shape; call `validate()` when the whole definition, including numerical values and closure, must be checked. `number_of_parameters()` reports the public array size even if its current shape is malformed.

`control_point_count()`, `weight_count()`, and `knot_count()` report component counts. The three range getters return *flattened scalar indices*, despite the word `count` in their names:

Range getter	2D indices	3D indices
`get_control_point_count_range()`	$[0, 2C)$	$[0, 3C)$
`get_weight_count_range()`	$[2C, 3C)$	$[3C, 4C)$
`get_knot_count_range()`	$[3C, N)$	$[4C, N)$

Listing 4.12: Use scalar ranges without assuming their offsets.

```
auto optimized = array_curve;
for (const auto i : optimized.get_weight_count_range()) {
    optimized.parameters[i] *= 1.25;
}
optimized.validate();

auto proposed = optimized.parameters;  // Detached valarray copy.
proposed[3] += 0.2;                    // Modify P[1].y.
optimized.set_parameters(proposed);    // Check before committing.
```

A range captures bounds when requested. Obtain a new range after replacing or resizing `parameters`; do not reuse stale bounds. To preserve atomic updates, prefer the array-specific `set_parameters(std::valarray<REAL>)` or the common checked setter families. The raw-pointer overload copies the current array's scalar count and supports a source pointer into that current storage by detaching first. `set_parameter` additionally returns `false` for a malformed current shape, while allocation failure can still throw.

4.8.2 Choosing the storage contract

Use an ordinary spline when callers should see a continuously valid object with private storage, stable metadata access, and cached endpoint queries. Use an array-backed spline when numeric fitting code needs an openly replaceable scalar array and can take responsibility for validating its edits. Both evaluate in their own Cartesian world and use analytic derivatives. Neither conversion nor direct array arithmetic is a coordinate transform.

The persistence, projection, drawing, distance, and intersection interfaces are documented with their accepted types in later chapters and the API reference. When an operation accepts an ordinary spline specifically, call `to_nurbs_spline()` explicitly; do not assume inheritance or an implicit conversion. The ordinary 3D spline supplies `tag_write` and its static `tag_read`; array-backed splines can use an explicit ordinary conversion for that 3D tagged representation.

Chapter 5

Intersections, distances, and numerical tools

An intersection query asks where a one-dimensional entity meets a circle, sphere, or plane. A distance query asks for a closest point as well as its parameter. These two questions share numerical machinery, but their results have different meanings: an empty intersection set does not say how far apart the entities are, and a distance rounded to zero does not create an intersection record.

The examples in this chapter use `namespace np = nurbspath;` and `using REAL = double;`, with the umbrella header included. All native 2D queries stay entirely in the 2D world. An explicit `project(plane, entity)` constructs a 3D entity when an application deliberately wants to cross that boundary; it is not an implementation step of a 2D intersection or distance.

5.1 Choose a query and its controls

World	Named intersection function	Result type
2D	`intersect_ray_circle(ray, circle, settings)`	`intersection_result2<REAL>`
2D	`intersect_spline_circle(spline, circle, settings)`	`intersection_result2<REAL>`
3D	`intersect_ray_sphere(ray, sphere, settings)`	`intersection_result3<REAL>`
3D	`intersect_ray_plane(ray, plane, settings)`	`intersection_result3<REAL>`
3D	`intersect_spline_sphere(spline, sphere, settings)`	`intersection_result3<REAL>`
3D	`intersect_spline_plane(spline, plane, settings)`	`intersection_result3<REAL>`

Table 5.1: The six supported intersection pairs. Each also has an `intersect(first, second, settings)` convenience overload with the same argument order and behavior.

The first argument is always the ray or ordinary NURBS spline. These overloads do not provide arbitrary entity-pair intersections: there is no ray–ray, spline–spline, sphere–sphere, circle–circle, or reversed-argument overload. For an array-backed spline, first construct its corresponding ordinary spline explicitly; that conversion validates the current public array storage.

5.1.1 Two tolerances, two different units

`numerical_settings<REAL>` carries all controls for numerical intersections and point-to-spline distances. The defaults are:

```
np::numerical_settings<REAL> settings;
// residual_tolerance = 1.0000e-08
// parameter_tolerance = 1.0000e-10
// max_iterations = 100
// sample_count = 256

settings.residual_tolerance = 1.0000e-07;
settings.parameter_tolerance = 1.0000e-09;
settings.sample_count = 512;
settings.validate();
```

`residual_tolerance` is measured in the units of the supplied function's result. For the library's surface intersections, the residual is a signed world distance; for `find_roots` it has whatever units the callable returns. `parameter_tolerance` measures a parameter interval. For spline queries it uses the original knot-domain units, not a normalized fraction of the domain. Changing a spline's native domain therefore changes the interpretation of a fixed parameter tolerance.

`sample_count` is the number of initial uniform intervals, so a sampled search evaluates `sample_count + 1` initial positions. The iteration limit applies to each refinement, rather than being a budget shared by all contacts or local minima. Increase sampling density when a curve or residual oscillates rapidly, has tightly spaced contacts, or has narrow distance minima.

`validate()` throws `std::invalid_argument` unless both tolerances are finite and strictly positive, `max_iterations` is positive, and `sample_count` is at least two. The numerical query functions call this validation themselves. A larger iteration budget sharpens a basin that was found; it cannot recover a basin that the initial sampling missed.

What a sampled answer guarantees

Sampling and local refinement provide useful coverage controlled by `sample_count`. They do not establish an unconditional continuous global guarantee for every NURBS curve or user-supplied function. Check stability under increased sampling when the application requires confidence that all contacts or the smallest distance have been found.

5.2 Read an intersection result

`intersection_kind` has three values. `none` means that the query found no contact in its searched domain; `discrete` means that isolated contacts are listed; `coincident` denotes a continuous overlap that a finite list of points cannot represent. A coincident result deliberately has an empty `points` vector. Thus `points.empty()` alone cannot distinguish no contact from continuous overlap. Use `has_intersection()` to accept either kind of contact, and `is_coincident()` to branch on continuous overlap.

The vector `points` is sorted by increasing `s`. Numerically close root parameters are merged; the root solver's merge threshold is the greater of four parameter tolerances and a roundoff allowance proportional to the search interval. This is a parameter-based rule, not a promise to merge every pair of spatially close points. A self-intersecting spline can visit the same world position at two distinct native parameters, and those visits can remain two distinct contacts.

Field	`intersection2<REAL>`	`intersection3<REAL>`
`point`	Contact position in 2D	Contact position in 3D
`s`	Native ray or spline parameter	Native ray or spline parameter
`u`	Circle angle in $[0, 2\pi)$	Sphere longitude or plane u coordinate
`v`	Not present	Sphere latitude or plane v coordinate
`residual`	Absolute circle-distance residual	Absolute sphere- or plane-distance residual

Table 5.2: A contact preserves both the world position and the participating entities' parameters. Sphere latitude lies in $[-\pi/2, \pi/2]$; plane parameters are lengths along its orthonormal local frame.

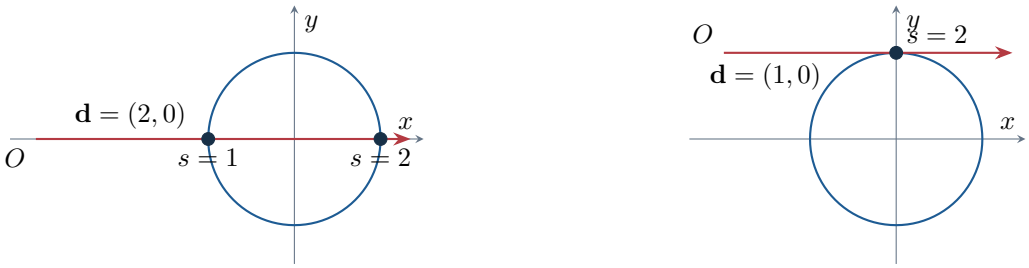


Figure 5.1: Crossing and tangent ray–circle contacts. The black points are computed by the library; TikZ constructs the unit circles and rays. The tangent contact has no residual sign change.

5.3 Rays: forward domains and direction scales

5.3.1 A ray through a circle or sphere

The ray equation is $R(s) = O + s\mathbf{d}$, with $s \geq 0$. A direction is not normalized by the entity. For the left-hand example in Figure 5.1, each increase of one in s moves two world units. This gives the two unit-circle contacts at $s = 1$ and $s = 2$.

```
const np::ray2<REAL> ray{{-3, 0}, {2, 0}};
const np::circle2<REAL> circle{{0, 0}, 1};
const auto hits = np::intersect_ray_circle(ray, circle);
// Equivalent: np::intersect(ray, circle)

std::cout << std::scientific << std::setprecision(4);
if (hits.kind == np::intersection_kind::discrete) {
    for (const auto& hit : hits.points) {
        std::cout << hit.s << ' ' << hit.point.x << ' '
                  << hit.point.y << '\n';
    }
}
```

The rounded output is:

```
1.0000e+00 -1.0000e+00 0.0000e+00
2.0000e+00  1.0000e+00 0.0000e+00
```

The 3D counterpart changes the target from a circle to a sphere:

```
const np::ray3<REAL> ray3{{-3, 0, 0}, {2, 0, 0}};
const np::sphere3<REAL> sphere{{0, 0, 0}, 1};
const auto sphere_hits = np::intersect_ray_sphere(ray3, sphere);
// Equivalent: np::intersect(ray3, sphere)
```

Both radial queries refine the signed distance

$$f(t) = \|O + t\hat{\mathbf{d}} - C\| - r$$

using world-distance travel t, then return the native parameter $s = t/\|\mathbf{d}\|$. The finite forward search extends to $\|O - C\| + r + \varepsilon_r$. Scaling the direction by a positive factor therefore preserves world contacts while dividing their native s values by that factor. A negative factor reverses the forward half-line and can change the set of contacts. A ray starting inside a circle or sphere normally has one forward exit; a ray pointing away from the target may have none. Contact at the ray origin is included.

5.3.2 A ray and an infinite plane

`intersect_ray_plane` uses the plane's unit normal and the signed world-distance residual $f(s) = \hat{\mathbf{n}} \cdot (R(s) - P_0)$. The native rate of change is $\mathbf{d} \cdot \hat{\mathbf{n}}$. If that rate is exactly zero, an origin within residual tolerance gives `coincident`; otherwise the result is `none`. This parallel test uses a zero normal component rather than comparing the direction's magnitude to a tolerance.

For a nonzero rate, an origin already within tolerance is an isolated contact at $s = 0$. Otherwise a negative estimated contact parameter means that the plane is behind the ray. A forward contact is bracketed and refined through the shared numerical root search. Refinement tightens the parameter tolerance where needed for a large normal-direction rate.

```
const np::plane3<REAL> plane{{0, 0, 0}, {0, 0, 1}};
const np::ray3<REAL> toward{{0.5, 0.25, 2}, {0, 0, -2}};
const auto plane_hit = np::intersect_ray_plane(toward, plane);
// Equivalent: np::intersect(toward, plane)
// One contact: s = 1, world point (0.5, 0.25, 0).

const np::ray3<REAL> parallel{{0, 0, 2}, {1, 0, 0}};
const np::ray3<REAL> contained{{0, 0, 0}, {1, 0, 0}};
const auto no_hit = np::intersect(parallel, plane);
const auto overlap = np::intersect(contained, plane);
// no_hit.kind == intersection_kind::none
// overlap.is_coincident() && overlap.points.empty()
```

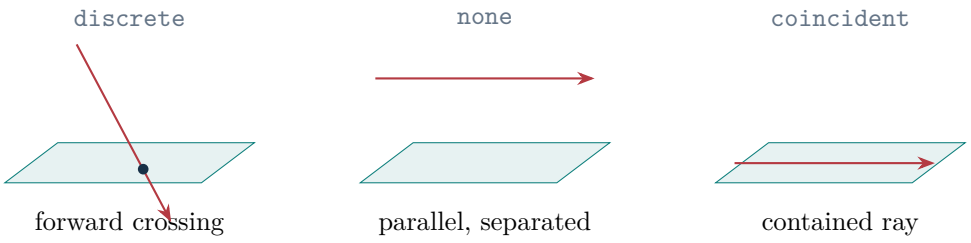


Figure 5.2: The three ray–plane classifications. The drawn patch represents an infinite plane; its finite outline is only a diagram aid.

5.4 Splines: crossings, tangencies, and continuous contact

The three spline queries evaluate the full active interval $[s_{\min}, s_{\max}]$. Their residuals follow the target surface:

- `intersect_spline_circle` uses a 2D radial distance.
- `intersect_spline_sphere` uses a 3D radial distance.
- `intersect_spline_plane` uses the plane's signed distance.

Their `intersect` aliases preserve exactly these domains and meanings.

A compact degree-one example makes the native domain visible:

```
const np::nurbs_spline2<REAL> segment2{
    std::vector<np::point2<REAL>>{{-2, 0}, {2, 0}},
    std::vector<REAL>{1, 1},
    std::vector<REAL>{10, 10, 14, 14}, 1};
const np::circle2<REAL> unit_circle{{0, 0}, 1};
const auto circle_hits = np::intersect_spline_circle(
    segment2, unit_circle, settings);
// Native s values are approximately 11 and 13, not 0.25 and 0.75.

const np::nurbs_spline3<REAL> segment3{
    std::vector<np::point3<REAL>>{{-2, 0, 0}, {2, 0, 0}},
    std::vector<REAL>{1, 1},
    std::vector<REAL>{10, 10, 14, 14}, 1};
const np::sphere3<REAL> unit_sphere{{0, 0, 0}, 1};
const np::plane3<REAL> yz_plane{{0, 0, 0}, {1, 0, 0}};
const auto sphere_contacts = np::intersect_spline_sphere(
    segment3, unit_sphere, settings);
const auto plane_contact = np::intersect_spline_plane(
    segment3, yz_plane, settings);
// Sphere: s approximately 11 and 13. Plane: s approximately 12.
```

Endpoints are included. A tangent contact may have residual zero with the same sign on both sides, so the search does more than look for sign changes. It also refines sampled local minima of the absolute residual. Increasing `sample_count` remains necessary when tangencies lie in narrow or crowded features.

Before returning isolated roots, each spline query checks whether the surface residual stays within tolerance at a uniform set of $\max(\texttt{sample_count}, 16) + 1$ points. If so, it returns `coincident`. This classification is numerical and sampled: it is not an algebraic proof that the entire rational curve lies on the surface. A spline contained in a plane or tracing a circle on a sphere is a natural use case.

For a spline explicitly marked closed, a root at the last active parameter is removed when roots are already present within parameter tolerance of both domain ends. The first-end contact remains as the seam representative. Closure does not mean that arbitrary equal world positions elsewhere on the curve are merged, and it does not itself imply periodic derivatives.

5.5 Distances return geometry, not just a number

The detailed queries in `distance.hpp` keep the closest world point and its target parameter. The corresponding `distance` overload calls the detailed function and returns only its `distance` field.

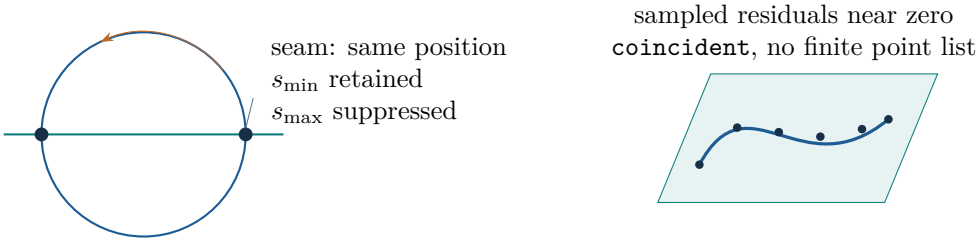


Figure 5.3: Closed-seam deduplication and sampled continuous-contact detection. The left-hand surface is shown by its trace through the curve's drawing plane. The right-hand dots illustrate residual checks rather than isolated intersection records.

Argument order is always query point first, target entity second.

World	Detailed query	Result and target parameters
2D	`distance_to_circle(point, circle, tolerance)`	`point_circle_distance2<REAL>`: `u`
2D	`distance_to_spline(point, spline, settings)`	`point_spline_distance2<REAL>`: `s`
3D	`distance_to_plane(point, plane, tolerance)`	`point_surface_distance3<REAL>`: `u`, `v`
3D	`distance_to_sphere(point, sphere, tolerance)`	`point_surface_distance3<REAL>`: `u`, `v`
3D	`distance_to_spline(point, spline, settings)`	`point_spline_distance3<REAL>`: `s`

Table 5.3: Every result also has `distance` and `closest_point` fields. The scalar alternative for each row is `distance(point, target, tolerance_or_settings)`.

All reported distances are nonnegative. When the measured separation is at most the contact tolerance, the returned scalar is snapped to zero; the closest point and its parameters remain those calculated on the target. A zero result therefore permits a small geometric gap. For a spline, the contact tolerance is `settings.residual_tolerance`; for a circle, sphere, or plane, the separate tolerance argument defaults to `1.0000e-08`.

5.5.1 Circle and sphere: radial closest points

For a center C, radius r, and query $Q \neq C$,

$$P = C + r\frac{Q - C}{\|Q - C\|}, \qquad d_{\text{measured}} = \big|\|Q - C\| - r\big|.$$

The expression applies in 2D to a circle and in 3D to a sphere. It finds the closest point on the boundary, including when Q is inside. At the center, every boundary point is equally near; the library deterministically chooses the positive world-x point. Its circle angle is $u = 0$, and its spherical parameters are $(u, v) = (0, 0)$.

```
const np::circle2<REAL> circle{{0, 0}, 1};
const auto radial2 = np::distance_to_circle(
    np::point2<REAL>{3, 4}, circle);
// Rounded closest_point = (6.0000e-01, 8.0000e-01).
// Rounded distance = 4.0000e+00.
const REAL scalar2 = np::distance(np::point2<REAL>{3, 4}, circle);

const np::sphere3<REAL> sphere{{0, 0, 0}, 1};
const auto radial3 = np::distance_to_sphere(
    np::point3<REAL>{0, 0, 3}, sphere);
// closest_point = (0, 0, 1), distance = 2, v = pi/2.
const REAL scalar3 = np::distance(np::point3<REAL>{0, 0, 3}, sphere);
```

The circle-distance tolerance must be finite and strictly positive. The plane- and sphere-distance overloads reject nonpositive tolerances; applications should also supply finite values, consistently with the settings-based queries.

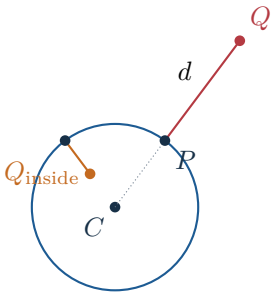
5.5.2 Plane: orthogonal closest points

Let $\hat{\mathbf{n}}$ be the unit normal and P_0 its parameter origin. Then

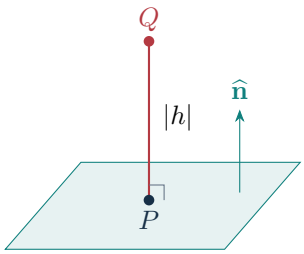
$$h = \hat{\mathbf{n}} \cdot (Q - P_0), \qquad P = Q - h\hat{\mathbf{n}}, \qquad d_{\text{measured}} = |h|.$$

The detailed query returns plane parameters of P, so `plane.point_at(result.u, result.v)` reconstructs the closest position. `plane.signed_distance_to(point)` remains available when the sign, rather than the nonnegative distance query, is required.

```
const np::plane3<REAL> plane{
    np::point3<REAL>{0, 0, 0}, np::vector3<REAL>{0, 0, 1},
    np::vector3<REAL>{1, 0, 0}};
const np::point3<REAL> query{2, 3, 4};
const auto nearest = np::distance_to_plane(query, plane);
// closest_point = (2, 3, 0), distance = 4, (u,v) = (2,3).
const REAL separation = np::distance(query, plane);
```



radial distance to the boundary circle or sphere cross-section



orthogonal distance to an infinite plane (u, v) describe the foot point P

Figure 5.4: Analytic closest points for radial surfaces and planes. These distances need no sampling. Interior radial queries still measure to the surface, rather than returning zero throughout the filled region.

5.6 Closest points on NURBS splines

`distance_to_spline` minimizes the squared Euclidean separation $D(s) = \|Q - C(s)\|^2$ over the active knot domain. Both 2D and 3D forms evaluate the initial samples, retain the best sample, then refine each sampled local basin using golden-section minimization. The first and last sample are also eligible basins, with refinement over the adjacent boundary interval. This matters when the true nearest point lies just inside an endpoint.

The same rational curve used in Chapter 4 provides a more interesting distance example than a straight line:

```
const np::nurbs_spline2<REAL> curve{
    std::vector<np::point2<REAL>>{
        {0, 0}, {1, 2}, {2, -1}, {3, 2}, {4, 0}},
    std::vector<REAL>{1, 1, 0.7, 1.5, 1},
    std::vector<REAL>{0, 0, 0, 0, 2, 5, 5, 5, 5}, 3};
const np::point2<REAL> query{2, 2.5};
const auto closest = np::distance_to_spline(query, curve, settings);
const REAL only_distance = np::distance(query, curve, settings);

// closest.s stays in the original [0,5] knot domain.
// curve.evaluate(closest.s) reproduces closest.closest_point.
```

Figure 5.5 uses points evaluated by the library itself. The target parameter remains the native s, even if its units were inherited from interpolation stations. Neither the reported closest point nor the distance query renormalizes the knot vector.

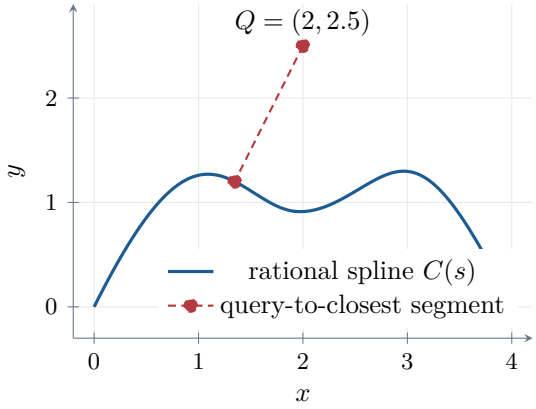


Figure 5.5: A sampled and refined 2D closest point. The curve samples and the two endpoints of the red distance segment come from the library reference-data generator in `manual/pictures/`. The 3D distance overload follows the same search strategy in its independent world.

The 3D call has the same shape:

```
const np::nurbs_spline3<REAL> segment{
    std::vector<np::point3<REAL>>{{0, 0, 0}, {1, 0, 0}},
    std::vector<REAL>{1, 1},
    std::vector<REAL>{0, 0, 1, 1}, 1};
const auto near_start = np::distance_to_spline(
    np::point3<REAL>{0.001, 0, 0}, segment, settings);
// closest point is just inside the segment, not its start endpoint.
// Reported distance is zero within residual_tolerance.
```

Endpoint retention prevents a refinement from discarding a true boundary minimum, while boundary-interval refinement can improve a coarse endpoint sample to an interior minimum. Multiple sampled minima are compared; no unique closest-point guarantee is made for equal minima. In particular, a closed seam or a symmetric curve can have more than one equally good parameter, while the result type intentionally returns one.

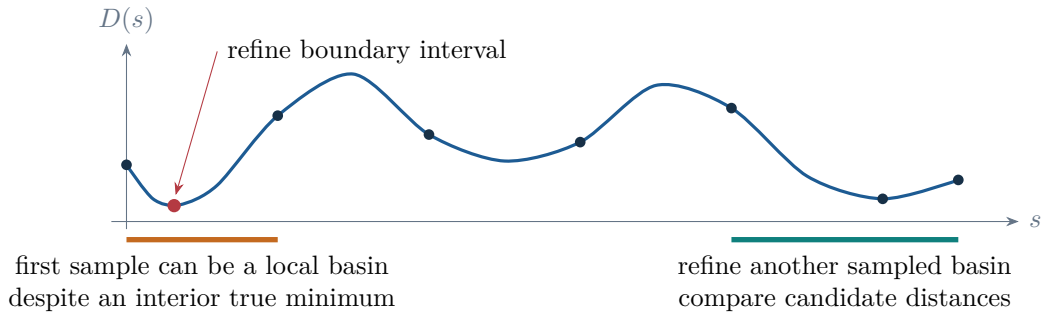


Figure 5.6: Why closest-point searches refine more than the single best sample. This schematic squared-distance graph illustrates a minimum near the beginning of the domain and another basin near its end. It is not an additional library test curve.

5.7 Reusable scalar algorithms

The utilities in `utility.hpp` are public and independent of a particular geometry class. Their callable parameters accept a scalar and return a scalar. The application is responsible for choosing meaningful units, finite working ranges, and functions compatible with the search assumptions.

5.7.1 Find all sampled roots, including even-multiplicity contacts

`find_roots<REAL>(function, lower, upper, settings)` requires a strictly increasing interval. It samples that interval, refines sign changes with bisection, and minimizes absolute residual near sampled sign-preserving local minima. Contiguous within-tolerance sample bands are represented by a single best candidate, avoiding a separate result for every sample on a flat contact. The final roots are sorted and tolerance-deduplicated.

```
const auto function = [](REAL x) {
    return (x + 0.8) * np::square(x - 0.3) * (x - 1.2);
};
const auto roots = np::find_roots<REAL>(
    function, -1.5, 1.6, settings);
for (REAL root : roots) {
    std::cout << std::scientific << std::setprecision(4)
              << root << '\n';
}
```

The three expected roots, shown as rounded reference values, are -8.0000×10^{-1}, 3.0000×10^{-1}, and 1.2000×10^{0}. The middle root is double and does not change the sign of the residual. `find_roots` rejects invalid settings or a nonpositive interval length with `std::invalid_argument`; a nonfinite initial sample value raises `std::domain_error`. A discontinuous callable or a nonfinite value encountered during refinement falls outside a well-behaved root-search problem; do not treat a sign change across a discontinuity as a root proof.

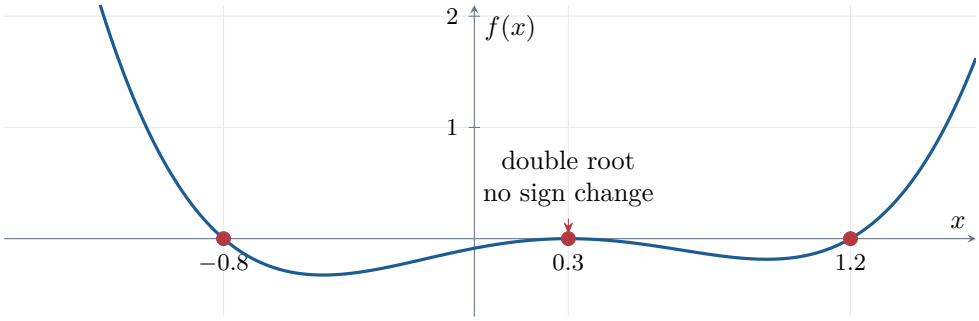


Figure 5.7: The scalar residual $f(x) = (x + 0.8)(x - 0.3)^2(x - 1.2)$. Its sampled graph is generated by C++ using the library's scalar helper; TikZ marks the known reference roots. Sign changes alone would miss the contact at $x = 0.3$.

5.7.2 Refine one sign-changing bracket

`bisect_root<REAL>(function, lower, upper, settings)` accepts endpoints in either order. An endpoint whose absolute residual is already within tolerance is returned immediately. Otherwise the endpoints must have opposite residual signs, or `std::invalid_argument` is thrown. Each iteration keeps the half-interval that still brackets the sign change. The function returns when the residual is small enough, the interval is narrow enough, or the iteration budget has been used.

```
const REAL root_two = np::bisect_root<REAL>(
    [](REAL x) { return np::square(x) - 2; }, 1, 2, settings);
// Rounded: 1.4142e+00
```

The return type is a parameter alone: it does not carry an iteration count or a convergence flag. In particular, reaching the parameter tolerance or the iteration limit does not independently establish a small residual. Evaluate the returned residual if the application requires that additional check. Bisection is appropriate for a continuous sign-changing bracket; use the sampled root finder to seek an unbracketed tangent contact.

5.7.3 Minimize one bounded basin

`golden_section_minimize<REAL>(function, lower, upper, settings)` also accepts reversed endpoints. Two interior probes progressively contract a closed interval, reusing one function value after each comparison. It returns the center of the final interval, stopping at the parameter tolerance or iteration limit. A unimodal objective over the chosen interval is the useful working assumption. The function does not search every local minimum of an arbitrary multi-basin objective, and it does not separately select endpoint values; callers needing boundary minima should compare the endpoints too.

```
const REAL minimum_x = np::golden_section_minimize<REAL>(
    [](REAL x) { return np::square(x - 0.75) + 2; },
    -1, 2, settings);
// Rounded minimizer: 7.5000e-01; objective value: 2.0000e+00.
```

Both refiners call `settings.validate()`. Golden-section search uses `parameter_tolerance` and `max_iterations` to stop; changing `residual_tolerance` does not supply a separate objective-value stopping rule. It is nevertheless validated as part of the settings object.

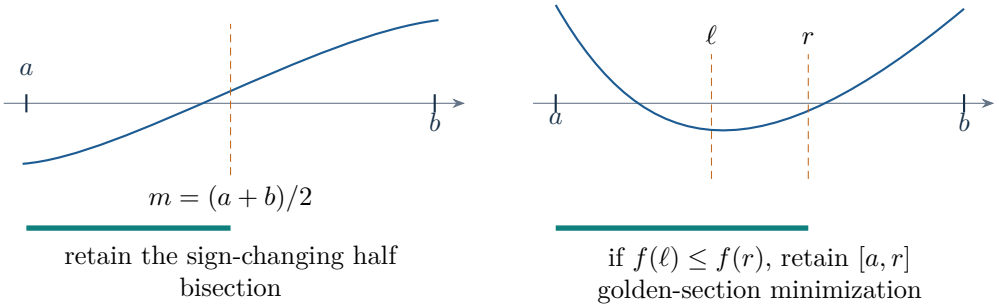


Figure 5.8: A root refiner and a minimum refiner answer different questions. The green segments show one possible retained interval. The diagrams are schematic; neither refiner reports a continuous global-search certificate.

5.8 Linear systems and small helpers

5.8.1 Solve a square system

`solve_linear_system(matrix, right_hand_side, pivot_tolerance)` solves $A\mathbf{x} = \mathbf{b}$ by elimination with scaled partial pivoting. Each row's largest absolute entry supplies a scale, so the pivot comparison is relative to that row's magnitude. The matrix and right-hand-side arguments are passed by value and copied for elimination; the caller's containers are preserved.

```
const std::vector<std::vector<REAL>> matrix{{2, 1}, {1, 3}};
const std::vector<REAL> right_hand_side{5, 10};
const auto solution = np::solve_linear_system(matrix, right_hand_side);
// Rounded solution: {1.0000e+00, 3.0000e+00}.
```

The default pivot rejection threshold is 64 `std::numeric_limits<REAL>::epsilon()`. The function requires a nonempty square matrix and a matching right-hand side, otherwise it throws `std::invalid_argument`. A zero row or a pivot too small relative to its row scale raises `std::domain_error`, identifying a singular or rejected ill-conditioned system. Use finite coefficients and a sensible finite nonnegative pivot threshold; this is a small dense solver, not a rank-revealing factorization or a condition-number estimator. Global interpolation uses this reusable solver rather than duplicating it inside spline classes.

5.8.2 Square, absolute comparison, and definition sizing

`square(value)` returns `value * value`; it is `constexpr` and `noexcept`, with ordinary floating-point range and rounding behavior. It does not trap an overflow. `approximately_equal(first, second, tolerance)` returns whether the absolute difference is at most the supplied tolerance. It performs no relative scaling and does not validate the tolerance. Select a nonnegative tolerance in the compared quantities' units. For example, a fixed absolute tolerance suitable for millimeter coordinates need not suit the same data expressed in kilometers.

`nurbs_knot_count(degree, control_point_count)` returns $C + p + 1$. It is shared by both worlds and both spline storage forms. The degree must be positive and smaller than the control-point count; invalid values raise `std::invalid_argument`. An unrepresentable result raises `std::overflow_error`. This helper sizes the complete knot vector; it does not generate knot values or decide whether a candidate vector is clamped.

```
constexpr std::size_t knots_needed = np::nurbs_knot_count(3, 5);
static_assert(knots_needed == 9);
const REAL area_factor = np::square(REAL(2.5));
const bool close = np::approximately_equal(
    REAL(1), REAL(1) + REAL(5.0000e-09), REAL(1.0000e-08));
// area_factor rounds to 6.2500e+00; close is true.
```

For two points in the same world, `distance_squared(first, second)` and `distance(first, second)` are also available from their point headers. They return ordinary squared Euclidean distance and Euclidean distance, respectively, without contact snapping or a search. Unlike the target-entity distance overloads, they need neither a tolerance nor numerical settings.

A practical verification pattern

Preserve the parameters and world positions from detailed results when they matter to downstream work. Re-evaluate the entity at those parameters, inspect the residual, and repeat sampled queries at higher sampling density when coverage is uncertain. Round only presentation output; keep the underlying floating-point values for subsequent calculations and persistence.

Chapter 6

Rendering, persistence, and diagnostics

6.1 A flat native 2D canvas

`svg_view2<REAL>` maps the independent 2D world directly onto a rectangular SVG image. Supply its world centre, minimum visible world width and height, and optional image dimensions. The mapping uses one uniform scale; a viewport with a different aspect ratio is letterboxed rather than stretching circles into ellipses. Positive world y points up while SVG image y points down.

```
namespace np = nurbspath;
using REAL = double;
const np::svg_view2<REAL> view({0.0, 0.0}, 6.0, 4.0, 900, 600);
np::svg_graphics_options2<REAL> style;
style.line_width = 1.5;
style.spline_segment_count = 160;
np::svg_canvas2<REAL> canvas(view, style);
canvas.add(np::point2<REAL>{-1.0, 1.0});
canvas.add(np::ray2<REAL>({-2.0, -1.0}, {1.0, 0.25}));
canvas.add(np::circle2<REAL>({0.5, 0.0}, 1.0));
const std::string svg = canvas.svg();
// canvas.write(output) streams the same self-contained document.
```

The canvas accepts native 2D points, rays, circles, and ordinary splines. `add(spline)` draws its sampled polyline, control polygon, controls, and evaluated endpoint dots. All flat-canvas strokes are solid, including construction geometry. Unit red/green XY references remain visible at the 2D origin. Vectors cannot be drawn alone because they do not supply a position.

6.2 World-space cameras in 3D

`svg_view3<REAL>::orthographic` takes a world-space eyepoint and look-at point, orthographic world width/height, and image dimensions. `svg_view3<REAL>::perspective` instead takes the vertical field angle in radians and a positive near distance. A camera must have a nonzero viewing direction. Image dimensions and orthographic extents must be positive; a perspective field angle must be strictly between zero and π. Camera getters expose the selected settings without changing the camera.

```
const np::point3<REAL> eye{5.0, -7.0, 4.5};
const np::point3<REAL> target{0.0, 0.0, 0.5};
```

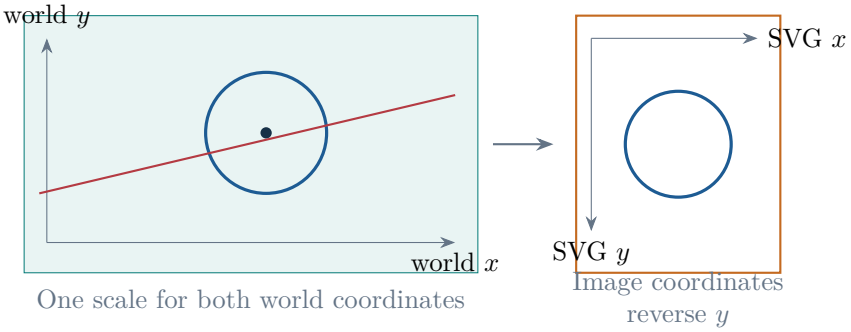


Figure 6.1: A flat drawing needs no plane embedding. Uniform scale preserves shape; the image coordinate direction is a display convention.

```
const auto ortho = np::svg_view3<REAL>::orthographic(
    eye, target, 6.0, 4.0, 900, 600);
const auto perspective = np::svg_view3<REAL>::perspective(
    eye, target, std::numbers::pi_v<REAL> / 4.0,
    900, 600, 0.05);
np::svg_document3<REAL> scene(perspective);
scene.add(np::sphere3<REAL>({0.0, 0.0, 0.0}, 1.0));
scene.add(np::plane3<REAL>({0.0, 0.0, -1.0}, {0.0, 0.0, 1.0}));
scene.add(np::ray3<REAL>({-2.0, 0.0, 0.0}, {2.0, 0.0, 0.0}));
const std::string scene_svg = scene.svg();
```

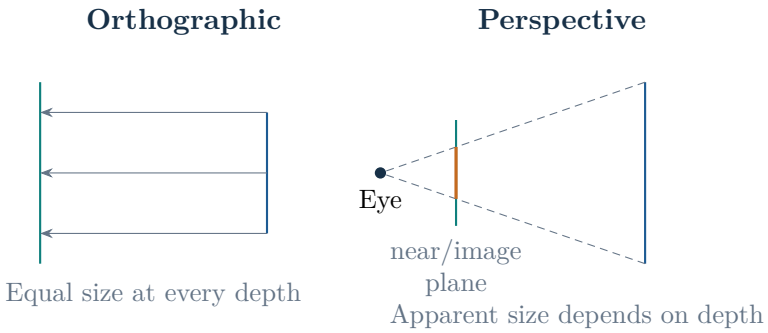


Figure 6.2: Camera projection is a display operation on the same 3D geometry. The perspective diagram is a 2D cross-section of the viewing cone.

Perspective world segments are clipped at the near plane before division by depth. A segment crossing the plane retains its visible part; an entirely hidden segment disappears. Orthographic output uses a uniform screen scale. The sphere silhouette is sampled from geometry appropriate to the camera.

6.3 Diagnostic glyphs and convenience rendering

The default 3D scene is deliberately diagnostic rather than photorealistic. The configurable `line_width` is in SVG viewBox units. The graphics options also control spline/sphere segment counts, the plane square size, and other documented construction lengths. `validate()` rejects invalid settings and derived stroke sizes that cannot remain finite.

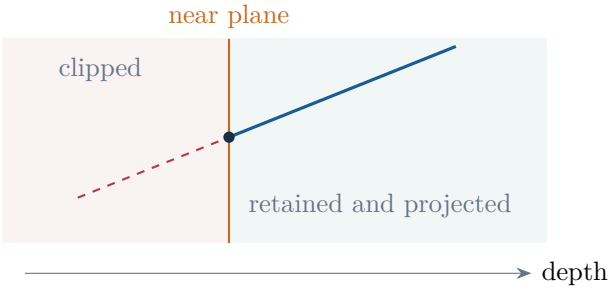


Figure 6.3: Near-plane clipping prevents an invisible endpoint from producing an unbounded perspective line. The clipped endpoint is an intermediate rendering point, not a new library entity.

Entity	Default diagnostic conventions
Point	Black location dot.
Ray	Pale-red dashed infinite supporting line, solid positive unit-direction marker, and origin dot. Native direction magnitude does not change its glyph.
Sphere	Pale-blue silhouette, visible/hidden equator strokes, and centre dot.
Plane	Pale-green square (default side one quarter unit), unit normal, half-unit positive u/v markers, and parameter-origin dot.
Spline	Piecewise-linear curve, dashed control polygon, control dots, and evaluated endpoint dots.
World axes	Persistent coloured XYZ unit axes and labels at half line width.

A native 2D entity can be explicitly embedded for rendering with `scene.add(plane, entity2)`. Its primary strokes are dash-dotted, with dotted auxiliary/hidden/control strokes. The 2D circle bridge carries centre and radius into a complete sphere. The separate flat canvas performs no such embedding and always uses solid strokes.

`to_svg(view2, entity2, options2)` constructs a temporary flat canvas. `to_svg(view3, entity3, options3)` constructs a 3D document. `to_svg(view3, plane, entity2, options3)` performs explicit embedding. All return self-contained XML strings; `svg()` and `write(output)` expose the corresponding accumulated document. These functions do not render free vectors or transform an arbitrary coordinate frame. A projected 2D circle whose finite world radius overflows the screen radius has its outline omitted; a finite centre still receives its dot.

6.4 Coordinate streams, CSV/TSV, and binary records

`point2`, `point3`, `vector2`, and `vector3` support `operator«` and `operator»`. Their fields are whitespace-separated coordinates in $x, y[, z]$ order. Reads commit only when a complete coordinate record has been obtained. Vector/point semantics remain explicit even though the scalar layout is similar.

```
np::point3<REAL> position{1.0, 2.0, 3.0};
std::ostringstream csv;
position.csv_write(csv, np::text_format::csv, 4);
// Display: 1.0000e+00,2.0000e+00,3.0000e+00
std::istringstream csv_input(csv.str());
np::point3<REAL> restored;
restored.csv_read(csv_input, np::text_format::csv);
std::stringstream binary(std::ios::in | std::ios::out | std::ios::binary);
position.write(binary);
```

```
np::point3<REAL> binary_copy;
binary_copy.read(binary);
```

`csv_write(output, format, decimal_places)` uses scientific notation; the default precision is six decimal places. Select `text_format::csv`, `tsv`, or `txt`. Delimited reads expect the chosen separators in the coordinate record; this is not a general quoted-field CSV parser. These helpers do not append a newline. Invalid format/negative output precision are rejected. Stream errors use the normal stream state and exception mask.

Binary `write`/`read` store two or three native `REAL` values, without a header or type discriminator. The layout is appropriate for matching readers/writers, not portable interchange across byte orders, scalar types, or floating-point representations. A failed complete read leaves the original entity unchanged.

6.5 Tagged records and independent presentation rows

Tagged persistence supports four concrete 3D geometry types and three presentation types. The scientific numbers in this displayed preview have four decimal places; the canonical writers deliberately use full round-trip `REAL` precision, independent of the destination stream formatting.

```
42 point3 v1 1.0000e+00 2.0000e+00 3.0000e+00
42 vector3 v1 0.0000e+00 1.0000e+00 0.0000e+00
42 sphere3 v1 0.0000e+00 0.0000e+00 0.0000e+00 1.0000e+00
42 linecolor v1 2.0000e-01 4.0000e-01 8.0000e-01 7.5000e-01
42 linefont v1 1.5000e+00 4.0000e+00 2.0000e+00
42 text v1 1.2000e+01 "Path \"A\"\nnext pass"
```

`spline3` is the token for `nurbs_spline3`; its payload contains degree, closure flag, tolerance, control and knot counts, interleaved control x, y, z, weight groups, then all knots. Array-backed splines convert to an ordinary same-world spline before tagged output. There is no tagged 2D geometry format or implicit 2D-to-3D persistence conversion.

Presentation value	Validation and meaning
`line_color<REAL>`	`red`, `green`, `blue`, `alpha` are finite in $[0, 1]$. Alpha is opacity; default is opaque black.
`line_font<REAL>`	Finite positive width; `on`/`off` are both zero for solid lines, or both finite positive for dashes. Default is `{1,0,0}`.
`text_label<REAL>`	Finite positive size, default 12, and a string. Width, dash lengths, and text size use consumer-selected display units.

```
std::stringstream rows;
position.tag_write(42, rows);
np::line_color<REAL>{0.2, 0.4, 0.8, 0.75}.tag_write(42, rows);
np::line_font<REAL>{1.5, 4.0, 2.0}.tag_write(42, rows);
np::text_label<REAL>{12.0, "Path \"A\"\nnext pass"}.tag_write(42, rows);
while (auto record = np::tag_read_record<REAL>(rows)) {
    std::visit([tag = record->tag](const auto& owner) {
        // *owner has its concrete geometry or presentation type.
        (void)tag;
        (void)owner;
    }, record->value);
}
```

Tags are unsigned base-ten `std::size_t` application values, not unique identifiers. Repeated tags, nonadjacent rows, and empty files are permitted. The consumer chooses association, omitted-style defaults, and label placement. Presentation rows do not alter SVG drawing state.

`T::tag_read(input)` returns `std::optional<tagged_read_result<T>>`, with `tag` and a non-null `std::shared_ptr<T>` named `entity`. A typed reader requires its matching non-advisory type. `tag_read<REAL>` returns a `tagged_entity3` whose `entity3_ptr` variant contains only point, vector, sphere, and ordinary spline pointers. `tag_read_record<REAL>` returns a `tagged_record` with the seven-type `value` variant.

Quoted labels escape backslash, double quote, newline, carriage return, and tab. Each record remains one physical row. Other ASCII control bytes are rejected; bytes at least 128 are preserved without UTF-8 validation. Writers validate public fields before emitting invalid data.

6.6 Advisory application and positioned messages

`application_info` writes an untagged row with two nonempty, unquoted tokens. ASCII whitespace/control bytes are not allowed in the tokens. `diagnostic_message3<REAL>` selects `warning`, `information`, or `error` using `diagnostic_level`, and supplies a finite 3D position, positive finite size, and escaped text. The default is informational, at the origin, size 12, with empty text.

```
np::application_info{"path_exporter", "1.0"}.write(output);
np::diagnostic_message3<REAL>{
    np::diagnostic_level::warning, {1.0, 2.0, 3.0},
    12.0, "Small clearance"}.tag_write(42, output);
```

```
application v1 path_exporter 1.0
42 warning v1 1.0000e+00 2.0000e+00 3.0000e+00 1.2000e+01 "Small clearance"
42 information v1 2.0000e+00 1.0000e+00 0.0000e+00 1.0000e+01 "Ready"
43 error v1 3.0000e+00 0.0000e+00 1.0000e+00 1.4000e+01 "No contact"
```

These are output-only annotations: readers may ignore them and the built-in readers skip them. The application prefix alone identifies its advisory row; a diagnostic must have a valid unsigned tag and a recognised diagnostic type. Their versions and payloads are not decoded when skipped. Neither reader variant gains an advisory alternative. An error annotation does not itself terminate export or set the destination stream's failure state.

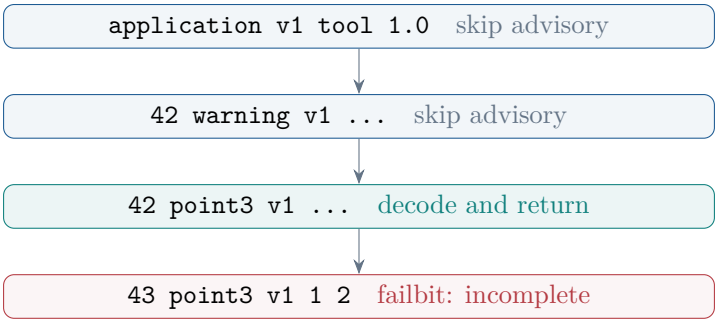


Figure 6.4: Reader calls skip advisory rows, then consume at most one required geometry or presentation row. Malformed required data still fails strictly.

Malformed required rows set `failbit` after consuming their complete physical row. Unknown types, unsupported required versions, missing/extra fields, invalid tags, and invalid definitions fail. After clearing a recoverable state, reading resumes from the next row; a reader never searches

past malformed required data. EOF and failure both return `nullopt`, so inspect the stream state. CRLF and a final valid row without a newline are accepted. Buffering and entity allocation can throw; applications needing resource bounds must enforce their own line-size and complexity limits.

6.7 Configuration and the hand-authored extension point

`config.hpp` exposes semantic version strings/numbers and Git metadata, including commit availability and dirty-state flags. Direct source includes use the source fallback. A normal CMake build generates an updated header; this standalone manual does not change that workflow. A repository without a first commit can report an unborn state. Frequently used macros are:

- `NURBSPATH_VERSION_STRING`, `NURBSPATH_VERSION_NUMBER`;
- `NURBSPATH_GIT_COMMIT`, `NURBSPATH_GIT_COMMIT_AVAILABLE`;
- `NURBSPATH_GIT_DIRTY`, `NURBSPATH_GIT_DESCRIBE`;
- `NURBSPATH_CONFIG_GENERATED`.

The umbrella also includes `manual.hpp`, whose `nurbspath::manual` namespace is an extension point for future hand-authored functions. The current snapshot defines no algorithms there. It is distinct from this document's top-level `manual/` directory.

Appendix A

Public API catalogue

This catalogue is generated from the public headers, using the installed Doxygen XML extractor. It records individual callable overloads, constructors, public member data, result records, and namespace aliases/enums. Private members and `nurbspath::detail` implementation helpers are excluded.

The containing section supplies each class's template context, normally `REAL`. A `static` member is called on the class; other members are called on an instance. Template requirements are part of the header contract. The compact declarations below retain the documented argument lists and defaults; diagnostic attributes such as `[[nodiscard]]` can be seen in the source headers. See the illustrated chapters for units, construction choices, and numerical limitations. All names belong to namespace `nurbspath`.

A catalogue rather than generated implementation

The entries show public signatures and short source comments. They do not include function bodies. The checked-in LaTeX snapshot makes this appendix available for a normal local TeX build even if Doxygen is unavailable later. `python3 tools/generate_api.py` refreshes it without changing CMake.

A.1 nurbspath::application_info

Optional output identification of the application producing a stream.

```
std::string name;
```

Nonempty application-name token.

```
std::string version;
```

Nonempty application-version token.

```
std::ostream & write(std::ostream &output) const;
```

Write one optional application v1 <name> <version> output row.
Returns: The destination stream, with its formatting preserved.

A.2 nurbspath::circle2

Circle in the independent two-dimensional Cartesian world.

```
circle2(const point2< REAL > &center_value, REAL radius_value);
```

Construct a circle.

```
const point2< REAL > & center() const noexcept;
```

Get the center.
Returns: Constant center reference.

```
REAL radius() const noexcept;
```
Get the radius.
Returns: Radius in 2D world units.

```
const point2< REAL > & get_center() const noexcept;
```
Get the center.
Returns: Constant reference to the center in the 2D world.

```
REAL get_radius() const noexcept;
```
Get the radius.
Returns: Radius in 2D world units.

```
void set_center(const point2< REAL > &center_value) noexcept;
```
Replace the center while preserving the radius.

```
template <typename VECTOR>
void set_center(const VECTOR &center_value) noexcept;
```
Replace the center using vector components while preserving radius.

```
void set_radius(REAL radius_value);
```
Replace the radius while preserving the center.

```
void set_center_and_radius(const point2< REAL > &center_value, REAL radius_value);
```
Replace the center and radius together.

```
template <typename VECTOR>
void set_center_and_radius(const VECTOR &center_value, REAL radius_value);
```
Replace the center from vector components and replace the radius.

```
circle2 & operator=(const vector2< REAL > &center_value) noexcept;
```
Replace the center using vector components while preserving the radius.
Returns: Reference to this circle.

```
point2< REAL > point_at(REAL u) const noexcept;
```
Evaluate the circle parameterization.
Returns: Point on the circle in the 2D world.

```
vector2< REAL > normal_at(const point2< REAL > &point) const;
```
Compute the outward radial unit normal at a point.
Returns: Outward unit direction after radial projection.

```
REAL parameter_of(const point2< REAL > &point, REAL tolerance=REAL(0)) const;
```
Recover the angular parameter by radial projection.
Returns: Counterclockwise angle in [0,2*pi).

A.3 nurbspath::diagnostic_message3

Optional output diagnostic anchored at a position in the 3D world.

```
diagnostic_level level = diagnostic_level::information;
```
Output severity.

```
point3< REAL > position {};
```
Finite world-space position of the annotation.

```
REAL size = REAL(12);
```
Positive text size in consumer display units.

```
std::string value;
```
Text, optionally containing escaped line breaks.

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```
Write an optional positioned warning, information, or error row.
Returns: The destination stream, with its formatting preserved.

A.4 nurbspath::intersection2

2D contact with parameters of both participating entities.

```
point2< REAL > point;
```
Contact point in the independent 2D world.

```
REAL s = REAL(0);
```
Ray or spline parameter.

```
REAL u = REAL(0);
```
Circle angle in [0,2*pi).

```
REAL residual = REAL(0);
```
Absolute 2D circle residual after refinement.

A.5 nurbspath::intersection3

World-space contact with parameters of both participating entities.

```
point3< REAL > point;
```
Contact point in world coordinates.

```
REAL s = REAL(0);
```
Ray or spline parameter.

```
REAL u = REAL(0);
```
Sphere or plane u parameter.

```
REAL v = REAL(0);
```
Sphere or plane v parameter.

```
REAL residual = REAL(0);
```
Absolute surface residual after refinement.

A.6 nurbspath::intersection_result2

Complete result of one two-dimensional intersection query.

```
intersection_kind kind = intersection_kind::none;
```
Set classification.

```
std::vector< intersection2< REAL > > points;
```
Isolated contacts sorted by s.

```
bool has_intersection() const noexcept;
```
Test for isolated or coincident contact.
Returns: True unless kind is none.

```
bool is_coincident() const noexcept;
```
Test for continuous overlap.
Returns: True when kind is coincident.

A.7 nurbspath::intersection_result3

Complete result of one intersection query.

```
intersection_kind kind = intersection_kind::none;
```
Set classification.

```
std::vector< intersection3< REAL > > points;
```
Isolated contacts sorted by s.

```
bool has_intersection() const noexcept;
```
Test for any isolated or coincident contact.
Returns: True unless kind is none.

```
bool is_coincident() const noexcept;
```

Test for continuous overlap.
Returns: True when kind is coincident.

A.8 nurbspath::line_color

Optional normalized RGBA line color associated with an application tag.

```
REAL red = REAL(0);
```

Normalized red component in [0, 1].

```
REAL green = REAL(0);
```

Normalized green component in [0, 1].

```
REAL blue = REAL(0);
```

Normalized blue component in [0, 1].

```
REAL alpha = REAL(1);
```

Normalized opacity in [0, 1].

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```

Write one version-1 linecolor row without changing stream format.
Returns: The destination stream.

```
static std::optional< tagged_read_result< line_color > > tag_read(std::istream &input);
```

Read one linecolor v1 row and allocate a validated color. Advisory application and diagnostic rows are skipped without decoding.
Returns: Preserved tag and non-null color owner, or nullopt on EOF/failure. Malformed rows set failbit and consume only their own physical row.

A.9 nurbspath::line_font

Optional line width and repeating dash lengths for tagged line elements.

```
REAL width = REAL(1);
```

Positive stroke width in consumer display units.

```
REAL on = REAL(0);
```

Visible dash length; zero with off for a solid line.

```
REAL off = REAL(0);
```

Gap length; zero with on for a solid line.

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```

Write one version-1 linefont row without changing stream format.
Returns: The destination stream.

```
static std::optional< tagged_read_result< line_font > > tag_read(std::istream &input);
```

Read one linefont v1 row and allocate a validated line font. Advisory application and diagnostic rows are skipped without decoding.
Returns: Preserved tag and non-null font owner, or nullopt on EOF/failure. Malformed rows set failbit and consume only their own physical row.

A.10 nurbspath::numerical_settings

Controls iterative and sampled numerical algorithms.

```
REAL residual_tolerance = REAL(1e-8);
```

Accepted absolute function residual.

```
REAL parameter_tolerance = REAL(1e-10);
```

Accepted parameter interval width.

```
std::size_t max_iterations = 100;
```

Maximum refinement iterations.

```
std::size_t sample_count = 256;
```
Number of initial uniform intervals.

```
void validate() const;
```
Validate all numerical controls.

A.11 nurbspath::nurbs_arr_spline2

Self-contained 2D NURBS curve backed by one public scalar valarray.

```
using real_t = REAL;
```
Floating-point scalar stored by this spline.

```
using parameter_index_range = std::ranges::iota_view< std::size_t, std::size_t >;
```
Allocation-free range of flattened indices into parameters.

```
std::valarray< REAL > parameters;
```
Flattened control coordinates, weights, and knots.

```
nurbs_arr_spline2(const point2< REAL > &start, const point2< REAL > &end, std::size_t
    control_point_count, std::size_t degree, REAL tolerance);
```
Construct a standard open 2D spline with explicit tolerance.

```
template <std::same_as< bool > CLOSED = bool>
nurbs_arr_spline2(const point2< REAL > &start, const point2< REAL > &end, std::size_t
    control_point_count, std::size_t degree, CLOSED closed=false, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```
Construct a standard open or closed spline between two points.

```
nurbs_arr_spline2(std::valarray< REAL > values, std::size_t degree, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```
Construct an open array-backed spline from flattened parameters.

```
template <std::same_as< bool > CLOSED>
nurbs_arr_spline2(std::valarray< REAL > values, std::size_t degree, CLOSED closed, REAL tolerance=
    REAL(64) *std::numeric_limits< REAL >::epsilon());
```
Construct an open or closed spline from flattened parameters.

```
nurbs_arr_spline2(std::vector< point2< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, REAL tolerance=REAL(64) *std::numeric_limits< REAL
    >::epsilon());
```
Construct an open array-backed spline from component vectors.

```
template <std::same_as< bool > CLOSED>
nurbs_arr_spline2(std::vector< point2< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, CLOSED closed, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```
Construct an open or closed spline from component vectors.

```
nurbs_arr_spline2(const nurbs_spline2< REAL > &spline);
```
Copy an ordinary 2D NURBS spline into flattened storage.

```
std::size_t degree() const noexcept;
```
Get the polynomial degree.
Returns: Positive spline degree.

```
REAL tolerance() const noexcept;
```
Get the definition and parameter-boundary tolerance.
Returns: Current tolerance.

```
bool is_closed() const noexcept;
```
Report whether the spline requires a closed seam.
Returns: True when active-domain endpoints must coincide.

```
std::size_t number_of_parameters() const noexcept;
```
Get the number of flattened scalar parameters.
Returns: Current public array size, including malformed layouts.

```
std::size_t control_point_count() const;
```
Derive the control-point count from the current array layout.
Returns: Number of represented control points.

```
std::size_t knot_count() const;
```
Derive the knot count from the current array layout.
Returns: Number of represented knots.

```
std::size_t weight_count() const;
```
Derive the weight count from the current array layout.
Returns: Number of represented weights, equal to the control-point count.

```
parameter_index_range get_control_point_count_range() const;
```
Get the flattened indices occupied by control-point coordinates.
Returns: Iterable scalar-index range [0, 2 * control_point_count()).

```
parameter_index_range get_weight_count_range() const;
```
Get the flattened indices occupied by rational weights.
Returns: Iterable scalar-index range [2 * C, 3 * C), where C is the control-point count.

```
parameter_index_range get_knot_count_range() const;
```
Get the flattened indices occupied by knots.
Returns: Iterable scalar-index range [3 * C, parameters.size()), where C is the control-point count.

```
std::vector< point2< REAL > > get_control_points() const;
```
Get all control points as detached point values.
Returns: Control points reconstructed in index order.

```
std::vector< REAL > get_weights() const;
```
Get all weights as a detached vector.
Returns: Weights copied in index order.

```
std::vector< REAL > get_knots() const;
```
Get all knots as a detached vector.
Returns: Knots copied in index order.

```
point2< REAL > get_control_point(std::size_t index) const;
```
Get one control point by index.
Returns: Point reconstructed by value from two adjacent scalars.

```
REAL get_weight(std::size_t index) const;
```
Get one rational weight by index.
Returns: Selected weight value.

```
REAL get_knot(std::size_t index) const;
```
Get one knot by index.
Returns: Selected knot value.

```
REAL get_parameter(std::size_t index) const;
```
Get one scalar by its flattened index.
Returns: Selected scalar value.

```
std::string parameter_name(std::size_t index) const;
```
Get the descriptive name of one flattened scalar.
Returns: A P[i].x, P[i].y, W[i], or K[i] name.

```
void validate() const;
```
Validate the current public array and all typed metadata.

```
void set_parameters(std::valarray< REAL > values);
```
Atomically validate and replace the complete public scalar array.

```
void set_parameters(const REAL *values);
```
Atomically copy and replace the complete public scalar array.

```
bool set_parameter(std::size_t index, REAL value);
```
Atomically validate and replace one flattened scalar.
Returns: True when the candidate was valid and committed; false for an out-of-range index, malformed current shape, or a rejected value.

```
void set_control_point(std::size_t index, const point2< REAL > &point);
```
Atomically replace one control point.

```
void set_weight(std::size_t index, REAL value);
```
Atomically replace one rational weight.

```
void set_knot(std::size_t index, REAL value);
```
Atomically replace one knot.

```
void set_control_points(std::vector< point2< REAL > > control_points);
```
Atomically replace all control points while retaining other data.

```
void set_weights(std::vector< REAL > values);
```
Atomically replace all weights while retaining other data.

```
void set_knots(std::vector< REAL > values);
```
Atomically replace all knots while retaining other data.

```
void set_standard_knots(REAL s_end);
```
Set standard open-clamped knots on [0, s_end].

```
void set_standard_knots(REAL s_start, REAL s_end);
```
Set standard open-clamped knots on [s_start, s_end].

```
void set_tolerance(REAL value);
```
Atomically replace the validation tolerance.

```
void set_closed(bool value);
```
Atomically change the closed-seam requirement.

```
void set_definition(std::vector< point2< REAL > > control_points, std::vector< REAL > weights, std
    ::vector< REAL > knots, bool closed, REAL tolerance);
```
Atomically replace all numeric components and mutable metadata.

```
REAL s_min() const;
```
Get the lower active parameter bound from the current knot data.
Returns: Minimum valid native s.

```
REAL s_max() const;
```
Get the upper active parameter bound from the current knot data.
Returns: Maximum valid native s.

```
point2< REAL > get_start() const;
```
Evaluate the current point at the active-domain start.
Returns: Start point derived on demand from public storage.

```
point2< REAL > get_end() const;
```
Evaluate the current point at the active-domain end.
Returns: End point derived on demand from public storage.

```
spline_derivatives2< REAL > derivatives_at(REAL s) const;
```
Evaluate position and the first two analytic derivatives.
Returns: Position, first derivative, and second derivative in 2D.

```
point2< REAL > evaluate(REAL s) const;
```
Evaluate the curve position.
Returns: Curve point in the independent 2D world.

```
point2< REAL > point_at(REAL s) const;
```
Evaluate the curve position.
Returns: Curve point in the independent 2D world.

```
vector2< REAL > first_derivative(REAL s) const;
```
Evaluate the first analytic derivative.
Returns: First derivative with respect to native s.

```
vector2< REAL > second_derivative(REAL s) const;
```
Evaluate the second analytic derivative.
Returns: Second derivative with respect to native s.

```
vector2< REAL > third_derivative(REAL s) const;
```

Evaluate the third analytic rational derivative.
Returns: Third derivative with respect to native s.

```
vector2< REAL > tangent(REAL s, REAL tolerance=vector2< REAL >::default_tolerance()) const;
```

Evaluate a unit tangent.
Returns: Normalized first derivative.

```
REAL curvature(REAL s, REAL tangent_tolerance=vector2< REAL >::default_tolerance()) const;
```

Evaluate unsigned geometric curvature in the 2D world.
Returns: Nonnegative curvature magnitude in inverse 2D world units.

```
REAL get_polygon_length() const;
```

Get the total length of the current 2D control polygon.
Returns: Control-polygon length in 2D world units.

```
REAL approximate_arc_length(std::size_t segment_count=512) const;
```

Estimate arc length using a uniform-native-s polyline.
Returns: Approximate length in 2D world units.

```
void adopt_to_points(const std::vector< point2< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e-10));
```

Replace this curve with an open interpolant through samples.

```
void adopt_to_points(const std::vector< point2< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree, bool closed, REAL tolerance=REAL(1e-10));
```

Replace this curve with an open or closed interpolant.

```
nurbs_spline2< REAL > to_nurbs_spline() const;
```

Convert to the ordinary vector-backed 2D NURBS type.
Returns: Valid ordinary spline with identical geometry and metadata.

```
static nurbs_arr_spline2 interpolate(const std::vector< point2< REAL > > &samples, const std::
    vector< REAL > &arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e
    -10));
```

Build an open unit-weight spline interpolating measured samples.
Returns: Array-backed interpolating curve.

```
static nurbs_arr_spline2 interpolate(const std::vector< point2< REAL > > &samples, const std::
    vector< REAL > &arc_length_parameters, std::size_t requested_degree, bool closed, REAL
    tolerance=REAL(1e-10));
```

Build an open or closed unit-weight interpolating spline.
Returns: Array-backed interpolating curve.

A.12 nurbspath::nurbs_arr_spline3

Self-contained 3D NURBS curve backed by one public scalar valarray.

```
using real_t = REAL;
```

Scalar type stored by this spline.

```
using parameter_index_range = std::ranges::iota_view< std::size_t, std::size_t >;
```

Allocation-free range of flattened indices into parameters.

```
std::valarray< REAL > parameters;
```

Flattened control-coordinate, weight, and knot storage.

```
nurbs_arr_spline3(const point3< REAL > &start, const point3< REAL > &end, std::size_t
    control_point_count, std::size_t degree, REAL tolerance);
```

Construct a standard open 3D spline with explicit tolerance.

```
template <std::same_as< bool > CLOSED = bool>
nurbs_arr_spline3(const point3< REAL > &start, const point3< REAL > &end, std::size_t
    control_point_count, std::size_t degree, CLOSED closed=false, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Construct a standard open or closed spline between two 3D points.

```
nurbs_arr_spline3(std::valarray< REAL > parameter_values, std::size_t degree, REAL tolerance=REAL
    (64) *std::numeric_limits< REAL >::epsilon());
```

Construct an open array spline from flattened scalar parameters.

```
template <std::same_as< bool > CLOSED>
nurbs_arr_spline3(std::valarray< REAL > parameter_values, std::size_t degree, CLOSED closed, REAL
    tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Construct an open or closed array spline from flattened scalars.

```
nurbs_arr_spline3(std::vector< point3< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, REAL tolerance=REAL(64) *std::numeric_limits< REAL
    >::epsilon());
```

Construct an open array spline from ordinary vector collections.

```
template <std::same_as< bool > CLOSED>
nurbs_arr_spline3(std::vector< point3< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, CLOSED closed, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Construct an open or closed array spline from vector collections.

```
nurbs_arr_spline3(const nurbs_spline3< REAL > &spline);
```

Copy an ordinary 3D NURBS into independent array storage.

```
std::size_t number_of_parameters() const noexcept;
```

Get the current flattened scalar count.
Returns: Number of values in parameters, including an invalid draft.

```
std::size_t control_point_count() const;
```

Derive the current control-point count from shape and degree.
Returns: Number of control points and weights.

```
std::size_t weight_count() const;
```

Derive the current weight count.
Returns: Number of rational weights, equal to control_point_count().

```
std::size_t knot_count() const;
```

Derive the current knot count.
Returns: Complete knot-vector length.

```
parameter_index_range get_control_point_count_range() const;
```

Get the flattened control-coordinate index range.
Returns: Lazy ascending range of valid control-coordinate scalar indices.

```
parameter_index_range get_weight_count_range() const;
```

Get the flattened weight index range.
Returns: Lazy ascending range of valid flattened weight indices.

```
parameter_index_range get_knot_count_range() const;
```

Get the flattened knot index range.
Returns: Lazy ascending range of valid flattened knot indices.

```
std::size_t degree() const noexcept;
```

Get the polynomial degree.
Returns: Immutable spline degree.

```
REAL tolerance() const noexcept;
```

Get the definition tolerance.
Returns: Positive tolerance.

```
bool is_closed() const noexcept;
```

Report closure state.
Returns: True when closure is required.

```
std::vector< point3< REAL > > get_control_points() const;
```

Copy all control points out of scalar storage.
Returns: Point vector in control-point order.

```
std::vector< REAL > get_weights() const;
```

Copy all rational weights out of scalar storage.
Returns: Weight vector in control-point order.

```
std::vector< REAL > get_knots() const;
```

Copy the complete knot vector out of scalar storage.
Returns: Knot vector in stored order.

```
REAL get_parameter(std::size_t index) const;
```

Get one flattened scalar parameter.
Returns: Stored parameter value.

```
std::string parameter_name(std::size_t index) const;
```

Get the descriptive name of one flattened scalar.
Returns: P[i].x, P[i].y, P[i].z, W[i], or K[i].

```
bool set_parameter(std::size_t index, REAL value);
```

Atomically validate and replace one scalar parameter.
Returns: True when the candidate is valid and committed; false for an invalid shape, out-of-range index, or rejected candidate definition.

```
point3< REAL > get_control_point(std::size_t index) const;
```

Materialize one control point from three stored scalars.
Returns: Point value copied from X/Y/Z storage.

```
REAL get_weight(std::size_t index) const;
```

Get one rational weight.
Returns: Stored weight value.

```
REAL get_knot(std::size_t index) const;
```

Get one knot.
Returns: Stored knot value.

```
void validate() const;
```

Validate the current public array and all spline metadata.

```
void set_parameters(std::valarray< REAL > parameter_values);
```

Atomically replace the complete flattened parameter array.

```
void set_parameters(const REAL *parameter_values);
```

Atomically copy and replace the complete flattened parameter array.

```
void set_control_point(std::size_t index, const point3< REAL > &point);
```

Atomically replace one control point.

```
void set_weight(std::size_t index, REAL value);
```

Atomically replace one rational weight.

```
void set_knot(std::size_t index, REAL value);
```

Atomically replace one knot.

```
void set_control_points(std::vector< point3< REAL > > control_points);
```

Atomically replace all control points while retaining other data.

```
void set_weights(std::vector< REAL > weight_values);
```

Atomically replace all weights while retaining other data.

```
void set_knots(std::vector< REAL > knot_values);
```

Atomically replace the complete knot vector.

```
void set_definition(std::vector< point3< REAL > > control_points, std::vector< REAL >
    weight_values, std::vector< REAL > knot_values, bool closed, REAL tolerance);
```

Atomically replace all vector definition fields and metadata.

```
void set_tolerance(REAL tolerance);
```

Atomically change the validation tolerance.

```
void set_closed(bool closed);
```

Atomically change the closed-seam requirement.

```
void set_standard_knots(REAL s_end);
```

Set standard open-clamped knots on [0, s_end].

```
void set_standard_knots(REAL s_start, REAL s_end);
```

Set standard open-clamped knots on [s_start, s_end].

```
point3< REAL > get_start() const;
```

Compute the current point at the start of the active domain.
Returns: Evaluated start point by value; no endpoint is cached.

```
point3< REAL > get_end() const;
```

Compute the current point at the end of the active domain.
Returns: Evaluated end point by value; no endpoint is cached.

```
REAL s_min() const;
```

Get the current lower active parameter bound.
Returns: Knot at index degree().

```
REAL s_max() const;
```

Get the current upper active parameter bound.
Returns: Knot at index control_point_count().

```
spline_derivatives3< REAL > derivatives_at(REAL s) const;
```

Evaluate position and the first two analytic derivatives.
Returns: Position and derivatives with respect to native s.

```
point3< REAL > evaluate(REAL s) const;
```

Evaluate a world-space curve point.
Returns: Evaluated point.

```
point3< REAL > point_at(REAL s) const;
```

Evaluate a world-space curve point.
Returns: Evaluated point.

```
vector3< REAL > first_derivative(REAL s) const;
```

Evaluate the first analytic derivative.
Returns: First derivative with respect to native s.

```
vector3< REAL > second_derivative(REAL s) const;
```

Evaluate the second analytic derivative.
Returns: Second derivative with respect to native s.

```
vector3< REAL > third_derivative(REAL s) const;
```

Evaluate the third analytic rational derivative.
Returns: Third derivative with respect to native s.

```
vector3< REAL > tangent(REAL s, REAL tangent_tolerance=vector3< REAL >::default_tolerance()) const;
```

Evaluate a unit tangent.
Returns: Normalized first derivative.

```
REAL curvature(REAL s, REAL tangent_tolerance=vector3< REAL >::default_tolerance()) const;
```

Evaluate unsigned geometric curvature in world space.
Returns: Nonnegative curvature magnitude in inverse world units.

```
REAL get_polygon_length() const;
```

Get the total length of the current world-space control polygon.
Returns: Control-polygon length in world units.

```
REAL approximate_arc_length(std::size_t segment_count=512) const;
```

Estimate arc length using a uniform-native-parameter polyline.
Returns: Approximate world-space arc length.

```
void adopt_to_points(const std::vector< point3< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e-10));
```

Replace this spline with an open interpolant through samples.

```
void adopt_to_points(const std::vector< point3< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree, bool closed, REAL tolerance=REAL(1e-10));
```

Replace this spline with an open or closed interpolant.

```
nurbs_spline3< REAL > to_nurbs_spline() const;
```

Create an independent ordinary 3D NURBS spline.
Returns: Validated ordinary spline with identical geometry and metadata.

```
static nurbs_arr_spline3 interpolate(const std::vector< point3< REAL > > &samples, const std::
    vector< REAL > &arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e
    -10));
```

Build an open unit-weight spline interpolating measured samples.
Returns: Self-contained interpolating array spline.

```
static nurbs_arr_spline3 interpolate(const std::vector< point3< REAL > > &samples, const std::
    vector< REAL > &arc_length_parameters, std::size_t requested_degree, bool closed, REAL
    tolerance=REAL(1e-10));
```

Build an open or closed unit-weight interpolating array spline.
Returns: Self-contained interpolating array spline.

A.13 nurbspath::nurbs_spline2

Rational B-spline curve in the independent 2D world.

```
nurbs_spline2(const point2< REAL > &start, const point2< REAL > &end, std::size_t
    control_point_count, std::size_t degree, REAL tolerance);
```

Construct a standard open 2D spline with explicit tolerance.

```
template <std::same_as< bool > CLOSED = bool>
nurbs_spline2(const point2< REAL > &start, const point2< REAL > &end, std::size_t
    control_point_count, std::size_t degree, CLOSED closed=false, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Construct a standard open or closed 2D spline between two points.

```
nurbs_spline2(std::vector< point2< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, REAL tolerance=REAL(64) *std::numeric_limits< REAL
    >::epsilon());
```

Construct a 2D NURBS curve from its complete definition.

```
nurbs_spline2(std::vector< point2< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, bool closed, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Construct an open or closed 2D NURBS curve.

```
nurbs_spline2(const nurbs_arr_spline2< REAL > &spline);
```

Copy an array-backed 2D NURBS into ordinary vector storage.

```
const std::vector< point2< REAL > > & get_control_points() const noexcept;
```

Get all 2D control points.
Returns: Constant control-point vector reference.

```
const std::vector< REAL > & get_weights() const noexcept;
```

Get all rational weights.
Returns: Constant weight vector reference.

```
const std::vector< REAL > & get_knots() const noexcept;
```

Get the complete knot vector.
Returns: Constant knot vector reference.

```
std::size_t degree() const noexcept;
```

Get the polynomial degree.
Returns: Spline degree.

```
REAL tolerance() const noexcept;
```

Get the spline definition tolerance.
Returns: Positive tolerance.

```
bool is_closed() const noexcept;
```

Report whether the spline has a closed seam.
Returns: True for closed curves.

```
std::size_t number_of_parameters() const noexcept;
```

Get the number of editable scalar parameters.
Returns: Current number of indexed scalar parameters.

```
REAL get_parameter(std::size_t index) const;
```

Get one editable scalar parameter by its flattened index.
Returns: Current parameter value.

```
std::string parameter_name(std::size_t index) const;
```

Get the descriptive name of one scalar parameter in the current definition.
Returns: Parameter name in the documented flattened order.

```
void set_parameters(const REAL *parameter_values);
```

Atomically replace all scalar parameters from a raw sequence.

```
bool set_parameter(std::size_t index, REAL value);
```

Validate and atomically replace one scalar parameter.
Returns: True when the complete candidate is valid and committed; false for an out-of-range index or rejected spline definition.

```
const point2< REAL > & get_control_point(std::size_t index) const;
```

Get one control point by index.
Returns: Constant reference to the selected control point.

```
REAL get_weight(std::size_t index) const;
```

Get one rational weight by index.
Returns: Selected rational weight.

```
REAL get_knot(std::size_t index) const;
```

Get one knot by index.
Returns: Selected knot value.

```
void set_control_point(std::size_t index, const point2< REAL > &point);
```

Replace one control point through a checked atomic update.

```
void set_weight(std::size_t index, REAL weight);
```

Replace one rational weight through a checked atomic update.

```
void set_knot(std::size_t index, REAL knot);
```

Replace one knot through a checked atomic update.

```
void set_control_points(std::vector< point2< REAL > > control_points);
```

Replace the complete control-point vector while retaining other fields.

```
void set_weights(std::vector< REAL > weights);
```

Replace the complete weight vector while retaining other fields.

```
void set_knots(std::vector< REAL > knots);
```

Replace the complete knot vector while retaining other fields.

```
void set_standard_knots(REAL s_end);
```

Set standard open-clamped knots on the native domain [0, s_end].

```
void set_standard_knots(REAL s_start, REAL s_end);
```

Set standard open-clamped knots on [s_start, s_end].

```
void set_tolerance(REAL tolerance);
```

Change the validation and parameter-boundary tolerance atomically.

```
void set_closed(bool closed);
```

Change the closed-seam requirement atomically.

```
void set_definition(std::vector< point2< REAL > > control_points, std::vector< REAL > weights, std
    ::vector< REAL > knots, bool closed, REAL tolerance);
```

Atomically replace every definition field except the degree.

```
const point2< REAL > & get_start() const noexcept;
```

Get the cached point at the start of the active domain.
Returns: Constant reference to the point at s_min() without evaluation.

```
const point2< REAL > & get_end() const noexcept;
```

Get the cached point at the end of the active domain.
Returns: Constant reference to the point at s_max() without evaluation.

```
REAL s_min() const noexcept;
```

Get the lower active parameter bound.
Returns: Minimum valid s.

```
REAL s_max() const noexcept;
```

Get the upper active parameter bound.
Returns: Maximum valid s.

```
spline_derivatives2< REAL > derivatives_at(REAL s) const;
```

Evaluate position and the first two analytic derivatives.
Returns: Position, first derivative, and second derivative in 2D.

```
point2< REAL > evaluate(REAL s) const;
```

Evaluate the curve position.
Returns: Curve point in the 2D world.

```
point2< REAL > point_at(REAL s) const;
```

Evaluate the curve position.
Returns: Curve point in the 2D world.

```
vector2< REAL > first_derivative(REAL s) const;
```

Evaluate the first derivative.
Returns: Derivative with respect to s in 2D.

```
vector2< REAL > second_derivative(REAL s) const;
```

Evaluate the second derivative.
Returns: Second derivative with respect to s in 2D.

```
vector2< REAL > third_derivative(REAL s) const;
```

Evaluate the third analytic rational derivative.
Returns: Third derivative with respect to s in 2D.

```
vector2< REAL > tangent(REAL s, REAL tolerance=vector2< REAL >::default_tolerance()) const;
```

Evaluate a unit tangent.
Returns: Normalized first derivative.

```
REAL curvature(REAL s, REAL tangent_tolerance=vector2< REAL >::default_tolerance()) const;
```

Evaluate the unsigned curvature magnitude.
Returns: Nonnegative curvature magnitude at s.

```
REAL get_polygon_length() const noexcept;
```

Get the total length of the 2D control polygon.
Returns: Control-polygon length in 2D world units.

```
REAL approximate_arc_length(std::size_t segment_count=512) const;
```

Estimate 2D arc length using a uniform-s polyline.
Returns: Approximate length in 2D world units.

```
void adopt_to_points(const std::vector< point2< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e-10));
```

Replace this curve with an interpolant through measured samples.

```
void adopt_to_points(const std::vector< point2< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree, bool closed, REAL tolerance=REAL(1e-10));
```

Replace this curve with an open or closed interpolant.

```
static nurbs_spline2 interpolate(const std::vector< point2< REAL > > &samples, const std::vector<
    REAL > &arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e-10));
```

Build a unit-weight B-spline interpolating measured 2D samples.
Returns: Interpolating 2D NURBS curve with unit weights.

```
static nurbs_spline2 interpolate(const std::vector< point2< REAL > > &samples, const std::vector<
    REAL > &arc_length_parameters, std::size_t requested_degree, bool closed, REAL tolerance=REAL
    (1e-10));
```

Build an open or closed unit-weight interpolating 2D spline.
Returns: Interpolating 2D NURBS curve with unit weights.

A.14 nurbspath::nurbs_spline3

Rational B-spline curve embedded in the shared 3D world system.

```
nurbs_spline3(const point3< REAL > &start, const point3< REAL > &end, std::size_t
    control_point_count, std::size_t degree, REAL tolerance);
```

Construct a standard open 3D spline with explicit tolerance.

```
template <std::same_as< bool > CLOSED = bool>
nurbs_spline3(const point3< REAL > &start, const point3< REAL > &end, std::size_t
    control_point_count, std::size_t degree, CLOSED closed=false, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Construct a standard open or closed spline between two 3D points.

```
nurbs_spline3(std::vector< point3< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, REAL tolerance=REAL(64) *std::numeric_limits< REAL
    >::epsilon());
```

Construct a NURBS curve from its complete definition.

```
nurbs_spline3(std::vector< point3< REAL > > control_points, std::vector< REAL > weights, std::
    vector< REAL > knots, std::size_t degree, bool closed, REAL tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Construct an open or closed 3D NURBS curve.

```
nurbs_spline3(const nurbs_arr_spline3< REAL > &spline);
```

Copy an array-backed 3D NURBS into ordinary vector storage.

```
const std::vector< point3< REAL > > & get_control_points() const noexcept;
```

Get all control points.
Returns: Constant control-point vector reference.

```
const std::vector< REAL > & get_weights() const noexcept;
```

Get all rational weights.
Returns: Constant weight vector reference.

```
const std::vector< REAL > & get_knots() const noexcept;
```

Get the full knot vector.
Returns: Constant knot vector reference.

```
std::size_t degree() const noexcept;
```

Get the polynomial degree.
Returns: Spline degree.

```
REAL tolerance() const noexcept;
```

Get the spline definition tolerance.
Returns: Positive tolerance.

```
bool is_closed() const noexcept;
```

Report whether the spline has a closed seam.
Returns: True for closed curves.

```
std::size_t number_of_parameters() const noexcept;
```

Get the number of editable scalar parameters.
Returns: Current number of indexed scalar parameters.

```
REAL get_parameter(std::size_t index) const;
```

Get one editable scalar parameter by its flattened index.
Returns: Current parameter value.

```
std::string parameter_name(std::size_t index) const;
```

Get the descriptive name of one scalar parameter in the current definition.
Returns: Parameter name in the documented flattened order.

```
void set_parameters(const REAL *parameter_values);
```

Atomically replace all scalar parameters from a raw sequence.

```
bool set_parameter(std::size_t index, REAL value);
```

Validate and atomically replace one scalar parameter.
Returns: True when the complete candidate is valid and committed; false for an out-of-range index or rejected spline definition.

```
const point3< REAL > & get_control_point(std::size_t index) const;
```

Get one control point by index.
Returns: Constant reference to the selected control point.

```
REAL get_weight(std::size_t index) const;
```

Get one rational weight by index.
Returns: Selected rational weight.

```
REAL get_knot(std::size_t index) const;
```

Get one knot by index.
Returns: Selected knot value.

```
void set_control_point(std::size_t index, const point3< REAL > &point);
```

Replace one control point through a checked atomic update.

```
void set_weight(std::size_t index, REAL weight);
```

Replace one rational weight through a checked atomic update.

```
void set_knot(std::size_t index, REAL knot);
```

Replace one knot through a checked atomic update.

```
void set_control_points(std::vector< point3< REAL > > control_points);
```

Replace the complete control-point vector while retaining other fields.

```
void set_weights(std::vector< REAL > weights);
```

Replace the complete weight vector while retaining other fields.

```
void set_knots(std::vector< REAL > knots);
```

Replace the complete knot vector while retaining other fields.

```
void set_standard_knots(REAL s_end);
```

Set standard open-clamped knots on the native domain [0, s_end].

```
void set_standard_knots(REAL s_start, REAL s_end);
```

Set standard open-clamped knots on [s_start, s_end].

```
void set_tolerance(REAL tolerance);
```

Change the validation and parameter-boundary tolerance atomically.

```
void set_closed(bool closed);
```

Change the closed-seam requirement atomically.

```
void set_definition(std::vector< point3< REAL > > control_points, std::vector< REAL > weights, std
    ::vector< REAL > knots, bool closed, REAL tolerance);
```

Atomically replace every definition field except the degree.

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```

Write one version-1 tagged spline3 text record.
Returns: Reference to output after attempting to write one newline-terminated row.

```
const point3< REAL > & get_start() const noexcept;
```

Get the cached point at the start of the active domain.
Returns: Constant reference to the point at s_min() without evaluation.

```
const point3< REAL > & get_end() const noexcept;
```

Get the cached point at the end of the active domain.
Returns: Constant reference to the point at s_max() without evaluation.

```
REAL s_min() const noexcept;
```

Get the lower active parameter bound.
Returns: Minimum valid s.

```
REAL s_max() const noexcept;
```

Get the upper active parameter bound.
Returns: Maximum valid s.

```
spline_derivatives3< REAL > derivatives_at(REAL s) const;
```

Evaluate position and the first two derivatives together.
Returns: Position, first derivative, and second derivative.

```
point3< REAL > evaluate(REAL s) const;
```

Evaluate the curve position.
Returns: World-space curve point.

```
point3< REAL > point_at(REAL s) const;
```

Evaluate the curve position.
Returns: World-space curve point.

```
vector3< REAL > first_derivative(REAL s) const;
```

Evaluate the first derivative.
Returns: Derivative with respect to s.

```
vector3< REAL > second_derivative(REAL s) const;
```

Evaluate the second derivative.
Returns: Second derivative with respect to s.

```
vector3< REAL > third_derivative(REAL s) const;
```

Evaluate the third analytic rational derivative.
Returns: Third derivative with respect to s.

```
vector3< REAL > tangent(REAL s, REAL tolerance=vector3< REAL >::default_tolerance()) const;
```

Evaluate a unit tangent.
Returns: Normalized first derivative.

```
REAL curvature(REAL s, REAL tangent_tolerance=vector3< REAL >::default_tolerance()) const;
```

Evaluate the unsigned curvature magnitude.
Returns: Nonnegative curvature magnitude at s.

```
REAL get_polygon_length() const noexcept;
```

Get the total length of the world-space control polygon.
Returns: Control-polygon length in world units.

```
REAL approximate_arc_length(std::size_t segment_count=512) const;
```

Estimate total arc length using a uniform-s polyline.
Returns: Approximate world-space arc length.

```
void adopt_to_points(const std::vector< point3< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e-10));
```

Replace this curve with an interpolant through measured samples.

```
void adopt_to_points(const std::vector< point3< REAL > > &samples, const std::vector< REAL > &
    arc_length_parameters, std::size_t requested_degree, bool closed, REAL tolerance=REAL(1e-10));
```

Replace this curve with an open or closed interpolant.

```
static std::optional< tagged_read_result< nurbs_spline3 > > tag_read(std::istream &input);
```

Read and allocate one version-1 tagged spline3 text record.
Returns: Tag and non-null shared spline after success, or std::nullopt at EOF, another read failure, or after a malformed row.

```
static nurbs_spline3 interpolate(const std::vector< point3< REAL > > &samples, const std::vector<
    REAL > &arc_length_parameters, std::size_t requested_degree=3, REAL tolerance=REAL(1e-10));
```

Build a unit-weight B-spline interpolating measured samples.
Returns: Interpolating NURBS curve with unit weights.

```
static nurbs_spline3 interpolate(const std::vector< point3< REAL > > &samples, const std::vector<
    REAL > &arc_length_parameters, std::size_t requested_degree, bool closed, REAL tolerance=REAL
    (1e-10));
```

Build an open or closed unit-weight interpolating 3D spline.
Returns: Interpolating 3D NURBS curve with unit weights.

A.15 nurbspath::plane3

Infinite plane with an orthonormal (u,v) coordinate frame.

```
plane3(const vector3< REAL > &normal, REAL signed_distance, REAL tolerance=vector3< REAL >::
    default_tolerance());
```

Construct a plane in Hessian normal form.

```
plane3(const point3< REAL > &origin, const vector3< REAL > &normal, REAL tolerance=vector3< REAL
    >::default_tolerance());
```

Construct a plane from a parameter origin and normal.

```
plane3(const point3< REAL > &origin, const vector3< REAL > &normal, const vector3< REAL > &u_hint,
     REAL tolerance=vector3< REAL >::default_tolerance());
```

Construct a plane with a preferred positive-u direction.

```
const point3< REAL > & origin() const noexcept;
```

Get the plane parameter origin.
Returns: Constant origin reference.

```
const vector3< REAL > & normal() const noexcept;
```

Get the unit normal.
Returns: Constant normal reference.

```
const vector3< REAL > & u_direction() const noexcept;
```

Get the positive unit-u direction.
Returns: Constant u direction reference.

```
const vector3< REAL > & v_direction() const noexcept;
```

Get the positive unit-v direction.
Returns: Constant v direction reference.

```
const point3< REAL > & get_origin() const noexcept;
```

Get the plane parameter origin.
Returns: Constant world-space origin reference.

```
const vector3< REAL > & get_normal() const noexcept;
```

Get the unit normal.
Returns: Constant normal reference.

```
const vector3< REAL > & get_u_direction() const noexcept;
```

Get the positive unit-u direction.
Returns: Constant u-direction reference.

```
const vector3< REAL > & get_v_direction() const noexcept;
```

Get the positive unit-v direction.
Returns: Constant v-direction reference.

```
void set_origin(const point3< REAL > &origin_value) noexcept;
```

Replace the parameter origin while preserving the frame directions.

```
template <typename VECTOR>
void set_origin(const VECTOR &origin_value) noexcept;
```

Replace the parameter origin using vector components.

```
void set_normal(const vector3< REAL > &normal_value, REAL tolerance=vector3< REAL >::
    default_tolerance());
```

Replace the normal and generate a stable orthonormal plane frame.

```
void set_u_direction(const vector3< REAL > &u_hint, REAL tolerance=vector3< REAL >::
    default_tolerance());
```

Replace positive u while preserving the origin and normal.

```
void set_definition(const vector3< REAL > &normal_value, REAL signed_distance, REAL tolerance=
    vector3< REAL >::default_tolerance());
```

Atomically replace the plane from Hessian normal form.

```
void set_definition(const point3< REAL > &origin_value, const vector3< REAL > &normal_value, REAL
    tolerance=vector3< REAL >::default_tolerance());
```

Atomically replace the origin, normal, and generated plane frame.

```
template <typename VECTOR>
void set_definition(const VECTOR &origin_value, const vector3< REAL > &normal_value, REAL
    tolerance=vector3< REAL >::default_tolerance());
```

Atomically replace the origin from vector components and the frame.

```
void set_definition(const point3< REAL > &origin_value, const vector3< REAL > &normal_value, const
     vector3< REAL > &u_hint, REAL tolerance=vector3< REAL >::default_tolerance());
```

Atomically replace the complete oriented plane definition.

```
template <typename VECTOR>
void set_definition(const VECTOR &origin_value, const vector3< REAL > &normal_value, const vector3
    < REAL > &u_hint, REAL tolerance=vector3< REAL >::default_tolerance());
```

Atomically replace an oriented plane using a vector origin.

```
REAL signed_distance_from_origin() const noexcept;
```

Get the equivalent Hessian signed distance.
Returns: d for the equation unit_normal dot x = d.

```
REAL get_signed_distance_from_origin() const noexcept;
```

Get the equivalent Hessian signed distance.
Returns: d for the equation unit_normal dot x = d.

```
point3< REAL > point_at(REAL u, REAL v) const noexcept;
```

Evaluate plane parameters.
Returns: World-space plane point.

```
REAL signed_distance_to(const point3< REAL > &point) const noexcept;
```

Compute oriented perpendicular distance to the plane.
Returns: Signed distance, positive in the normal direction.

```
point3< REAL > project(const point3< REAL > &point) const noexcept;
```

Orthogonally project a point onto the plane.
Returns: Closest point on the plane.

```
std::pair< REAL, REAL > parameters_of(const point3< REAL > &point) const noexcept;
```

Recover plane coordinates after orthogonal projection.
Returns: Pair (u,v) in the plane frame.

A.16 nurbspath::point2

Position in the independent two-dimensional Cartesian world.

```
using value_type = REAL;
```

Floating-point scalar type.

```
REAL x = REAL(0);
```

X coordinate in the 2D world.

```
REAL y = REAL(0);
```

Y coordinate in the 2D world.

```
bool operator==(const point2 &other) const noexcept=default;
```

Compare coordinates exactly.
Returns: True when both coordinates are exactly equal.

```
template <typename VECTOR>
point2 & operator=(const VECTOR &vector_value) noexcept;
```

Replace this point's coordinates with vector components.
Returns: Reference to this updated point.

```
std::ostream & csv_write(std::ostream &output, text_format format=text_format::csv, std::
    streamsize decimal_places=6) const;
```

Write the coordinates in CSV, TSV, or whitespace-delimited text.
Returns: Reference to output.

```
std::istream & csv_read(std::istream &input, text_format format=text_format::csv);
```

Read the coordinates from CSV, TSV, or whitespace-delimited text.
Returns: Reference to input. This point changes only after a complete record.

```
std::ostream & write(std::ostream &output) const;
```

Write two native REAL values in X/Y order as binary data.
Returns: Reference to output.

```
std::istream & read(std::istream &input);
```

Read two native REAL values in X/Y order from binary data.
Returns: Reference to input. This point changes only after a complete record.

```
REAL & operator[](std::size_t index);
```

Access a coordinate by index.
Returns: Mutable reference to the selected coordinate.

```
const REAL & operator[](std::size_t index) const;
```

Access a coordinate by index.
Returns: Constant reference to the selected coordinate.

```
REAL magnitude() const noexcept;
```

Compute distance from the Cartesian 2D origin.
Returns: Euclidean magnitude in the coordinate units.

```
REAL manhattan_distance() const noexcept;
```

Compute Manhattan distance from the Cartesian 2D origin.
Returns: Sum of the absolute coordinate values in the coordinate units.

```
point2 & operator+=(const vector2< REAL > &offset) noexcept;
```

Translate this point in place.
Returns: Reference to this point.

```
point2 & operator-=(const vector2< REAL > &offset) noexcept;
```

Translate this point by the negative of a vector.
Returns: Reference to this point.

```
bool approximately_equal(const point2 &other, REAL tolerance=vector2< REAL >::default_tolerance())
     const noexcept;
```

Compare points using Euclidean distance.
Returns: True when positions differ by at most tolerance.

```
static point2 origin() noexcept;
```

Construct the Cartesian 2D origin.
Returns: (0,0).

A.17 nurbspath::point3

Position in the shared Cartesian world coordinate system.

```
using value_type = REAL;
```

Floating-point scalar type.

```
REAL x = REAL(0);
```

X coordinate in the 3D world.

```
REAL y = REAL(0);
```

Y coordinate in the 3D world.

```
REAL z = REAL(0);
```

Z coordinate in the 3D world.

```
bool operator==(const point3 &other) const noexcept=default;
```

Compare coordinates exactly.
Returns: True when all coordinates are exactly equal.

```
template <typename VECTOR>
point3 & operator=(const VECTOR &vector) noexcept;
```

Replace this point's coordinates with vector components.
Returns: Reference to this updated point.

```
std::ostream & csv_write(std::ostream &output, text_format format=text_format::csv, std::
    streamsize decimal_places=6) const;
```

Write the coordinates in CSV, TSV, or whitespace-delimited text.
Returns: Reference to output.

```
std::istream & csv_read(std::istream &input, text_format format=text_format::csv);
```

Read the coordinates from CSV, TSV, or whitespace-delimited text.
Returns: Reference to input. This point changes only after a complete record.

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```

Write one version-1 tagged point3 text record.
Returns: Reference to output after attempting to write one newline-terminated row.

```
std::ostream & write(std::ostream &output) const;
```

Write three native REAL values in X/Y/Z order as binary data.
Returns: Reference to output.

```
std::istream & read(std::istream &input);
```

Read three native REAL values in X/Y/Z order from binary data.
Returns: Reference to input. This point changes only after a complete record.

```
REAL & operator[](std::size_t index);
```

Access a coordinate by index.
Returns: Mutable reference to the selected coordinate.

```
const REAL & operator[](std::size_t index) const;
```

Access a coordinate by index.
Returns: Constant reference to the selected coordinate.

```
REAL magnitude() const noexcept;
```

Compute distance from the Cartesian world origin.
Returns: Euclidean magnitude in the coordinate units.

```
REAL manhattan_distance() const noexcept;
```

Compute Manhattan distance from the Cartesian world origin.
Returns: Sum of the absolute coordinate values in the coordinate units.

```
point3 & operator+=(const vector3< REAL > &offset) noexcept;
```

Translate this point in place.
Returns: Reference to this point.

```
point3 & operator-=(const vector3< REAL > &offset) noexcept;
```

Translate this point by the negative of a vector.
Returns: Reference to this point.

```
bool approximately_equal(const point3 &other, REAL tolerance=vector3< REAL >::default_tolerance())
     const noexcept;
```

Compare points using Euclidean distance.
Returns: True when positions differ by at most tolerance.

```
static std::optional< tagged_read_result< point3 > > tag_read(std::istream &input);
```

Read and allocate one version-1 tagged point3 text record.
Returns: Tag and non-null shared point after success, or std::nullopt at EOF, another read failure, or after a malformed row.

```
static point3 origin() noexcept;
```

Construct the Cartesian world origin.
Returns: (0,0,0).

A.18 nurbspath::point_circle_distance2

Detailed 2D point-to-circle distance result.

```
REAL distance = REAL(0);
```

Nonnegative tolerance-adjusted 2D distance.

```
point2< REAL > closest_point;
```

Closest point on the 2D circle.

```
REAL u = REAL(0);
```

Circle angle at closest_point in [0,2*pi).

A.19 nurbspath::point_spline_distance2

Detailed 2D point-to-spline distance result.

```
REAL distance = REAL(0);
```
Nonnegative tolerance-adjusted 2D distance.

```
point2< REAL > closest_point;
```
Closest sampled-and-refined spline point.

```
REAL s = REAL(0);
```
Spline parameter at closest_point.

A.20 nurbspath::point_spline_distance3

Detailed point-to-spline distance result.

```
REAL distance = REAL(0);
```
Nonnegative tolerance-adjusted distance.

```
point3< REAL > closest_point;
```
Closest point found on the spline.

```
REAL s = REAL(0);
```
Spline parameter at closest_point.

A.21 nurbspath::point_surface_distance3

Detailed point-to-surface distance result.

```
REAL distance = REAL(0);
```
Nonnegative tolerance-adjusted distance.

```
point3< REAL > closest_point;
```
Closest point on the queried surface.

```
REAL u = REAL(0);
```
Surface u parameter at closest_point.

```
REAL v = REAL(0);
```
Surface v parameter at closest_point.

A.22 nurbspath::ray2

2D half-line parameterized as origin + s * direction for s >= 0.

```
ray2(const point2< REAL > &origin_value, const vector2< REAL > &direction_value, REAL tolerance=
    vector2< REAL >::default_tolerance());
```
Construct a forward 2D ray.

```
const point2< REAL > & origin() const noexcept;
```
Get the base point.
Returns: Constant origin reference.

```
const vector2< REAL > & direction() const noexcept;
```
Get the unnormalized direction.
Returns: Constant direction reference.

```
const point2< REAL > & get_origin() const noexcept;
```
Get the base point.
Returns: Constant reference to the origin in the 2D world.

```
const vector2< REAL > & get_direction() const noexcept;
```
Get the unnormalized direction.
Returns: Constant reference to the parameter direction.

```
void set_origin(const point2< REAL > &origin_value) noexcept;
```
Replace the origin while preserving the direction.

```
template <typename VECTOR>
void set_origin(const VECTOR &origin_value) noexcept;
```
Replace the origin using vector components while preserving direction.

```
void set_direction(const vector2< REAL > &direction_value, REAL tolerance=vector2< REAL >::
    default_tolerance());
```
Replace the unnormalized parameter direction.

```
void set_origin_and_direction(const point2< REAL > &origin_value, const vector2< REAL > &
    direction_value, REAL tolerance=vector2< REAL >::default_tolerance());
```
Replace the origin and direction together.

```
template <typename VECTOR>
void set_origin_and_direction(const VECTOR &origin_value, const vector2< REAL > &direction_value,
    REAL tolerance=vector2< REAL >::default_tolerance());
```
Replace the origin from vector components and replace the direction.

```
point2< REAL > point_at(REAL s) const noexcept;
```
Evaluate the ray parameterization.
Returns: origin + s * direction in the 2D world.

```
point2< REAL > evaluate(REAL s) const noexcept;
```
Evaluate the ray parameterization.
Returns: origin + s * direction in the 2D world.

```
vector2< REAL > tangent() const;
```
Get the unit positive parameter direction.
Returns: Normalized direction.

A.23 nurbspath::ray3

Half-line parameterized as origin + s * direction for s >= 0.

```
ray3(const point3< REAL > &origin_value, const vector3< REAL > &direction_value, REAL tolerance=
    vector3< REAL >::default_tolerance());
```
Construct a forward parametric ray.

```
const point3< REAL > & origin() const noexcept;
```
Get the base point.
Returns: Constant reference to the origin.

```
const vector3< REAL > & direction() const noexcept;
```
Get the unnormalized parameter direction.
Returns: Ray direction.

```
const point3< REAL > & get_origin() const noexcept;
```
Get the base point.
Returns: Constant reference to the origin.

```
const vector3< REAL > & get_direction() const noexcept;
```
Get the unnormalized parameter direction.
Returns: Constant reference to the ray direction.

```
void set_origin(const point3< REAL > &origin_value) noexcept;
```
Replace the base point while preserving the direction.

```
template <typename VECTOR>
void set_origin(const VECTOR &origin_value) noexcept;
```
Replace the base point using vector components.

```
void set_direction(const vector3< REAL > &direction_value, REAL tolerance=vector3< REAL >::
    default_tolerance());
```
Replace the unnormalized parameter direction.

```
void set_origin_and_direction(const point3< REAL > &origin_value, const vector3< REAL > &
    direction_value, REAL tolerance=vector3< REAL >::default_tolerance());
```

Atomically replace the base point and parameter direction.

```
template <typename VECTOR>
void set_origin_and_direction(const VECTOR &origin_value, const vector3< REAL > &direction_value,
    REAL tolerance=vector3< REAL >::default_tolerance());
```

Atomically replace the base point from vector components and the parameter direction.

```
point3< REAL > point_at(REAL s) const noexcept;
```

Evaluate the ray parameterization.
Returns: origin + s * direction.

```
point3< REAL > evaluate(REAL s) const noexcept;
```

Evaluate the ray parameterization.
Returns: origin + s * direction.

```
vector3< REAL > tangent() const;
```

Get the unit positive parameter direction.
Returns: Normalized ray direction.

A.24 nurbspath::sphere3

Sphere in world coordinates.

```
sphere3(const point3< REAL > &center_value, REAL radius_value);
```

Construct a sphere.

```
const point3< REAL > & center() const noexcept;
```

Get the world-space center.
Returns: Constant center reference.

```
REAL radius() const noexcept;
```

Get the radius.
Returns: Radius in world units.

```
const point3< REAL > & get_center() const noexcept;
```

Get the world-space center.
Returns: Constant center reference.

```
REAL get_radius() const noexcept;
```

Get the radius.
Returns: Radius in world units.

```
void set_center(const point3< REAL > &center_value) noexcept;
```

Replace the world-space center while preserving the radius.

```
template <typename VECTOR>
void set_center(const VECTOR &center_value) noexcept;
```

Replace the world-space center using vector components.

```
void set_radius(REAL radius_value);
```

Replace the radius while preserving the center.

```
void set_center_and_radius(const point3< REAL > &center_value, REAL radius_value);
```

Atomically replace the center and radius.

```
template <typename VECTOR>
void set_center_and_radius(const VECTOR &center_value, REAL radius_value);
```

Atomically replace the center from vector components and the radius.

```
sphere3 & operator=(const vector3< REAL > &center_vector) noexcept;
```

Replace the center with vector components while preserving radius.
Returns: Reference to this updated sphere.

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```

Write one version-1 tagged sphere3 text record.
Returns: Reference to output after attempting to write one newline-terminated row.

```
point3< REAL > point_at(REAL u, REAL v) const noexcept;
```
Evaluate spherical surface parameters.
Returns: World-space surface point.

```
vector3< REAL > normal_at(const point3< REAL > &point) const;
```
Compute the outward radial unit normal at a point.
Returns: Outward unit normal after radial projection.

```
std::pair< REAL, REAL > parameters_of(const point3< REAL > &point, REAL tolerance=REAL(0)) const;
```
Recover surface coordinates by radial projection.
Returns: Pair (u,v) containing longitude and latitude in radians.

```
static std::optional< tagged_read_result< sphere3 > > tag_read(std::istream &input);
```
Read and allocate one version-1 tagged sphere3 text record.
Returns: Tag and non-null shared sphere after success, or std::nullopt at EOF, another read failure, or after a malformed row.

A.25 nurbspath::spline_derivatives2

Position and first two derivatives at one 2D spline parameter.

```
point2< REAL > point;
```
Evaluated position in the 2D world.

```
vector2< REAL > first;
```
First derivative with respect to s.

```
vector2< REAL > second;
```
Second derivative with respect to s.

A.26 nurbspath::spline_derivatives3

Position and first two derivatives evaluated at one spline parameter.

```
point3< REAL > point;
```
Evaluated world-space position.

```
vector3< REAL > first;
```
First derivative with respect to s.

```
vector3< REAL > second;
```
Second derivative with respect to s.

A.27 nurbspath::svg_canvas2

Accumulate native 2D geometry and emit a flat SVG document.

```
svg_canvas2(const svg_view2< REAL > &view, const svg_graphics_options2< REAL > &options={});
```
Construct an empty flat canvas containing the 2D XY reference.

```
void add(const point2< REAL > &point);
```
Add a native 2D point as a black dot.

```
void add(const ray2< REAL > &ray);
```
Add a native 2D ray using solid pale-red lines.

```
void add(const circle2< REAL > &circle);
```
Add a native 2D circle and center using solid pale-blue lines.

```
void add(const nurbs_spline2< REAL > &spline);
```
Add a native 2D spline with solid curve and control lines.

```
std::string svg() const;
```
Serialize the flat canvas.
Returns: Self-contained SVG XML string.

```
void write(std::ostream &output) const;
```
Stream the complete flat SVG document.

A.28 nurbspath::svg_document3

Accumulate projected geometry and emit a self-contained SVG document.

```
svg_document3(const svg_view3< REAL > &view, const svg_graphics_options3< REAL > &options={});
```
Construct an empty SVG scene containing the world XYZ reference.

```
void add(const point3< REAL > &point);
```
Add a point as a black dot of diameter twice the line width.

```
void add(const ray3< REAL > &ray);
```
Add a dashed supporting line, positive unit direction, and origin.

```
void add(const sphere3< REAL > &sphere);
```
Add a sphere silhouette, visible/hidden equator, and center dot.

```
void add(const plane3< REAL > &plane);
```
Add a plane square, positive parameter axes, normal, and origin.

```
void add(const nurbs_spline3< REAL > &spline);
```
Add control geometry, a tessellated curve, and endpoint dots.

```
void add(const plane3< REAL > &plane, const point2< REAL > &point);
```
Project and add a 2D point as a black point marker.

```
void add(const plane3< REAL > &plane, const ray2< REAL > &ray);
```
Project and add a 2D ray with pale-red patterned strokes.

```
void add(const plane3< REAL > &plane, const circle2< REAL > &circle);
```
Project and add a 2D circle as a patterned center/radius sphere.

```
void add(const plane3< REAL > &plane, const nurbs_spline2< REAL > &spline);
```
Project and add a 2D spline using patterned black/violet strokes.

```
std::string svg() const;
```
Serialize the complete scene.
Returns: Self-contained SVG XML string.

```
void write(std::ostream &output) const;
```
Stream the complete SVG document.

A.29 nurbspath::svg_graphics_options2

Styling and tessellation controls for the flat 2D SVG canvas.

```
REAL line_width = REAL(1.5);
```

```
std::size_t spline_segment_count = 128;
```
Number of solid straight elements used to approximate a NURBS path.

```
void validate() const;
```
Validate all flat-canvas graphics controls.

A.30 nurbspath::svg_graphics_options3

Styling and tessellation controls for three-dimensional SVG diagnostics.

```
REAL line_width = REAL(1.5);
```

```
std::size_t spline_segment_count = 128;
```
Number of straight elements used to approximate a complete NURBS path.

```
std::size_t sphere_segment_count = 96;
```
Number of straight elements used for the sphere outline and equator.

```
REAL plane_square_side_length = REAL(0.25);
```

```
REAL plane_parameter_axis_length = REAL(0.5);
```

Length of the positive-u and positive-v plane direction markers.

```
void validate() const;
```

Validate all graphics controls.

A.31 nurbspath::svg_view2

Flat viewport definition for the two-dimensional SVG canvas.

```
svg_view2(const point2< REAL > &center, REAL viewport_width, REAL viewport_height, std::size_t
    image_width=800, std::size_t image_height=600);
```

Construct a flat 2D viewport.

```
const point2< REAL > & center() const noexcept;
```

Get the 2D world point at image center.
Returns: Center reference.

```
REAL viewport_width() const noexcept;
```

Get the minimum visible world width.
Returns: Positive width.

```
REAL viewport_height() const noexcept;
```

Get the minimum visible world height.
Returns: Positive height.

```
std::size_t image_width() const noexcept;
```

Get the SVG viewBox width.
Returns: Positive width.

```
std::size_t image_height() const noexcept;
```

Get the SVG viewBox height.
Returns: Positive height.

A.32 nurbspath::svg_view3

Camera and viewport definition for a three-dimensional SVG view.

```
static svg_view3 orthographic(const point3< REAL > &eyepoint, const point3< REAL > &lookatpoint,
    REAL viewport_width, REAL viewport_height, std::size_t image_width=800, std::size_t
    image_height=600);
```

Construct an orthographic camera.
Returns: Validated orthographic view definition.

```
static svg_view3 perspective(const point3< REAL > &eyepoint, const point3< REAL > &lookatpoint,
    REAL vertical_view_angle, std::size_t image_width=800, std::size_t image_height=600, REAL
    near_distance=REAL(1e-4));
```

Construct a perspective camera.
Returns: Validated perspective view definition.

```
const point3< REAL > & eyepoint() const noexcept;
```

Get camera location.
Returns: Constant eyepoint reference.

```
const point3< REAL > & lookatpoint() const noexcept;
```

Get center-of-view target.
Returns: Constant look-at reference.

```
svg_projection3 projection() const noexcept;
```

Get projection mode.
Returns: Orthographic or perspective mode.

```
REAL viewport_width() const noexcept;
```

Get orthographic world width.
Returns: Width, or zero for perspective.

```
REAL viewport_height() const noexcept;
```
Get orthographic world height.
Returns: Height, or zero for perspective.

```
REAL vertical_view_angle() const noexcept;
```
Get vertical perspective field angle.
Returns: Radians, or zero for orthographic.

```
REAL near_distance() const noexcept;
```
Get perspective near distance.
Returns: Distance, or zero for orthographic.

```
std::size_t image_width() const noexcept;
```
Get SVG viewBox width.
Returns: Positive width.

```
std::size_t image_height() const noexcept;
```
Get SVG viewBox height.
Returns: Positive height.

A.33 nurbspath::tagged_entity3

Application tag and dynamically selected concrete 3D entity.

```
std::size_t tag = 0;
```

```
entity3_ptr< REAL > entity;
```
Entity pointer; successful reads always select a non-null shared owner.

A.34 nurbspath::tagged_read_result

Result of reading one tagged row for a known concrete entity type.

```
std::size_t tag = 0;
```

```
std::shared_ptr< ENTITY > entity;
```
Shared owner; successful reads always provide a non-null pointer.

A.35 nurbspath::tagged_record

Application tag and one geometry or optional presentation record.

```
std::size_t tag = 0;
```
Application-defined tag associating related rows.

```
tagged_record_ptr< REAL > value;
```
Non-null owner after a successful read.

A.36 nurbspath::text_label

Optional sized text associated with an application tag.

```
REAL size = REAL(12);
```
Positive text size in consumer display units.

```
std::string value;
```
Label text, optionally including escaped line breaks.

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```
Write one version-1 text row with a quoted, escaped string.
Returns: The destination stream, with its formatting preserved.

```
static std::optional< tagged_read_result< text_label > > tag_read(std::istream &input);
```

Read one text v1 row and allocate a validated text label. Advisory application and diagnostic rows are skipped without decoding.
Returns: Preserved tag and non-null label owner, or nullopt on EOF/failure. Invalid quoting, escapes, fields, or values set failbit after that row.

A.37 nurbspath::vector2

Cartesian vector in the independent two-dimensional world.

```
using value_type = REAL;
```

Floating-point scalar type.

```
REAL x = REAL(0);
```

X component in the 2D world.

```
REAL y = REAL(0);
```

Y component in the 2D world.

```
constexpr vector2() noexcept=default;
```

Construct the zero vector.

```
constexpr vector2(REAL x_value, REAL y_value) noexcept;
```

Construct a vector from Cartesian components.

```
constexpr vector2(const point2< REAL > &point_value) noexcept;
```

Construct the displacement from the 2D origin to a point.

```
vector2(const circle2< REAL > &circle_value) noexcept;
```

Construct the displacement from the 2D origin to a circle center.

```
vector2(const ray2< REAL > &ray_value) noexcept;
```

Construct the displacement from the 2D origin to a ray origin.

```
bool operator==(const vector2 &other) const noexcept=default;
```

Compare components exactly.
Returns: True when both components are exactly equal.

```
std::ostream & csv_write(std::ostream &output, text_format format=text_format::csv, std::
    streamsize decimal_places=6) const;
```

Write the components in CSV, TSV, or whitespace-delimited text.
Returns: Reference to output.

```
std::istream & csv_read(std::istream &input, text_format format=text_format::csv);
```

Read the components from CSV, TSV, or whitespace-delimited text.
Returns: Reference to input. This vector changes only after a complete record.

```
std::ostream & write(std::ostream &output) const;
```

Write two native REAL values in X/Y order as binary data.
Returns: Reference to output.

```
std::istream & read(std::istream &input);
```

Read two native REAL values in X/Y order from binary data.
Returns: Reference to input. This vector changes only after a complete record.

```
REAL & operator[](std::size_t index);
```

Access a component by index.
Returns: Mutable reference to the selected component.

```
const REAL & operator[](std::size_t index) const;
```

Access a component by index.
Returns: Constant reference to the selected component.

```
vector2 operator+() const noexcept;
```

Return this vector unchanged.
Returns: A copy of this vector.

```
vector2 operator-() const noexcept;
```

Negate every component.
Returns: Negated vector.

```
vector2 & operator+=(const vector2 &other) noexcept;
```

Add another vector in place.
Returns: Reference to this vector.

```
vector2 & operator-=(const vector2 &other) noexcept;
```

Subtract another vector in place.
Returns: Reference to this vector.

```
vector2 & operator*=(REAL scalar) noexcept;
```

Multiply every component in place.
Returns: Reference to this vector.

```
vector2 & operator/=(REAL scalar);
```

Divide every component in place.
Returns: Reference to this vector.

```
REAL dot(const vector2 &other) const noexcept;
```

Compute the scalar dot product.
Returns: Dot product.

```
REAL cross(const vector2 &other) const noexcept;
```

Compute the signed scalar 2D cross product.
Returns: Z component of the corresponding embedded 3D cross product.

```
REAL length_squared() const noexcept;
```

Compute squared Euclidean length.
Returns: Squared length.

```
REAL length() const noexcept;
```

Compute Euclidean length.
Returns: Vector length.

```
bool is_near_zero(REAL tolerance=default_tolerance()) const noexcept;
```

Test whether vector length is within a tolerance of zero.
Returns: True when squared length is at most tolerance squared.

```
vector2 normalized(REAL tolerance=default_tolerance()) const;
```

Return a unit vector in the same direction.
Returns: Normalized vector.

```
void normalize();
```

Normalize this vector in place using the default vector tolerance.

```
vector2 perpendicular_left() const noexcept;
```

Rotate this vector counterclockwise by ninety degrees.
Returns: Left perpendicular vector (-y,x).

```
vector2 perpendicular_right() const noexcept;
```

Rotate this vector clockwise by ninety degrees.
Returns: Right perpendicular vector (y,-x).

```
vector2 projected_onto(const vector2 &axis, REAL tolerance=default_tolerance()) const;
```

Project this vector onto an axis.
Returns: Component parallel to axis.

```
vector2 rejected_from(const vector2 &axis, REAL tolerance=default_tolerance()) const;
```

Remove the component parallel to an axis.
Returns: Component perpendicular to axis.

```
vector2 reflected(const vector2 &normal, REAL tolerance=default_tolerance()) const;
```

Reflect this vector across a line with a supplied normal.
Returns: Reflected vector.

```
REAL angle_to(const vector2 &other, REAL tolerance=default_tolerance()) const;
```

Compute the unsigned angle to another vector.
Returns: Angle in radians in [0,pi].

```
REAL signed_angle_to(const vector2 &other, REAL tolerance=default_tolerance()) const;
```
Compute the signed counterclockwise angle to another vector.
Returns: Angle in radians in [-pi,pi].

```
vector2 component_product(const vector2 &other) const noexcept;
```
Multiply corresponding components.
Returns: Component-wise product.

```
REAL max_component() const noexcept;
```
Get the largest component.
Returns: Largest component value.

```
REAL min_component() const noexcept;
```
Get the smallest component.
Returns: Smallest component value.

```
bool approximately_equal(const vector2 &other, REAL tolerance=default_tolerance()) const noexcept;
```
Compare vectors using Euclidean distance.
Returns: True when vectors differ by at most tolerance.

```
static vector2 zero() noexcept;
```
Construct the zero vector.
Returns: (0,0).

```
static vector2 unit_x() noexcept;
```
Construct the positive X unit vector.
Returns: (1,0).

```
static vector2 unit_y() noexcept;
```
Construct the positive Y unit vector.
Returns: (0,1).

```
static REAL default_tolerance() noexcept;
```
Get the default absolute vector tolerance.
Returns: 64 times machine epsilon for REAL.

A.38 nurbspath::vector3

Cartesian three-dimensional vector.

```
using value_type = REAL;
```
Floating-point scalar type.

```
REAL x = REAL(0);
```
X component.

```
REAL y = REAL(0);
```
Y component.

```
REAL z = REAL(0);
```
Z component.

```
constexpr vector3() noexcept=default;
```
Construct the zero vector.

```
constexpr vector3(REAL x_value, REAL y_value, REAL z_value) noexcept;
```
Construct a vector from X, Y, and Z components.

```
constexpr vector3(const point3< REAL > &point_value) noexcept;
```
Construct a vector containing a point's coordinates.

```
vector3(const sphere3< REAL > &sphere_value) noexcept;
```
Construct a vector containing a sphere center's coordinates.

```
vector3(const plane3< REAL > &plane_value) noexcept;
```
Construct a vector containing a plane origin's coordinates.

```
vector3(const ray3< REAL > &ray_value) noexcept;
```
Construct a vector containing a ray origin's coordinates.

```
bool operator==(const vector3 &other) const noexcept=default;
```

Compare components exactly.
Returns: True when all components are exactly equal.

```
std::ostream & csv_write(std::ostream &output, text_format format=text_format::csv, std::
    streamsize decimal_places=6) const;
```

Write the components in CSV, TSV, or whitespace-delimited text.
Returns: Reference to output.

```
std::istream & csv_read(std::istream &input, text_format format=text_format::csv);
```

Read the components from CSV, TSV, or whitespace-delimited text.
Returns: Reference to input. This vector changes only after a complete record.

```
std::ostream & tag_write(std::size_t tag, std::ostream &output) const;
```

Write one version-1 tagged vector3 text record.
Returns: Reference to output after attempting to write one newline-terminated row.

```
std::ostream & write(std::ostream &output) const;
```

Write three native REAL values in X/Y/Z order as binary data.
Returns: Reference to output.

```
std::istream & read(std::istream &input);
```

Read three native REAL values in X/Y/Z order from binary data.
Returns: Reference to input. This vector changes only after a complete record.

```
REAL & operator[](std::size_t index);
```

Access a component by index.
Returns: Mutable reference to the selected component.

```
const REAL & operator[](std::size_t index) const;
```

Access a component by index.
Returns: Constant reference to the selected component.

```
vector3 operator+() const noexcept;
```

Return this vector unchanged.
Returns: A copy of this vector.

```
vector3 operator-() const noexcept;
```

Negate every component.
Returns: Negated vector.

```
vector3 & operator+=(const vector3 &other) noexcept;
```

Add another vector in place.
Returns: Reference to this vector.

```
vector3 & operator-=(const vector3 &other) noexcept;
```

Subtract another vector in place.
Returns: Reference to this vector.

```
vector3 & operator*=(REAL scalar) noexcept;
```

Multiply every component in place.
Returns: Reference to this vector.

```
vector3 & operator/=(REAL scalar);
```

Divide every component in place.
Returns: Reference to this vector.

```
REAL dot(const vector3 &other) const noexcept;
```

Compute the scalar dot product.
Returns: Dot product.

```
vector3 cross(const vector3 &other) const noexcept;
```

Compute the right-handed cross product.
Returns: Vector perpendicular to both operands.

```
REAL length_squared() const noexcept;
```

Compute squared Euclidean length.
Returns: Squared length.

```
REAL length() const noexcept;
```
Compute Euclidean length.
Returns: Vector length.

```
bool is_near_zero(REAL tolerance=default_tolerance()) const noexcept;
```
Test whether vector length is within a tolerance of zero.
Returns: True when squared length is at most tolerance squared.

```
vector3 normalized(REAL tolerance=default_tolerance()) const;
```
Return a unit vector in the same direction.
Returns: Normalized vector.

```
void normalize();
```
Normalize this vector in place using the default vector tolerance.

```
vector3 projected_onto(const vector3 &axis, REAL tolerance=default_tolerance()) const;
```
Project this vector onto an axis.
Returns: Component parallel to axis.

```
vector3 rejected_from(const vector3 &axis, REAL tolerance=default_tolerance()) const;
```
Remove the component parallel to an axis.
Returns: Component perpendicular to axis.

```
vector3 reflected(const vector3 &normal, REAL tolerance=default_tolerance()) const;
```
Reflect this vector across a plane normal.
Returns: Reflected vector.

```
REAL angle_to(const vector3 &other, REAL tolerance=default_tolerance()) const;
```
Compute the unsigned angle to another vector.
Returns: Angle in radians in the range [0, pi].

```
vector3 component_product(const vector3 &other) const noexcept;
```
Multiply corresponding components.
Returns: Component-wise product.

```
REAL max_component() const noexcept;
```
Get the largest component.
Returns: Largest component value.

```
REAL min_component() const noexcept;
```
Get the smallest component.
Returns: Smallest component value.

```
bool approximately_equal(const vector3 &other, REAL tolerance=default_tolerance()) const noexcept;
```
Compare vectors using Euclidean distance.
Returns: True when vectors differ by at most tolerance.

```
static std::optional< tagged_read_result< vector3 > > tag_read(std::istream &input);
```
Read and allocate one version-1 tagged vector3 text record.
Returns: Tag and non-null shared vector after success, or std::nullopt at EOF, another read failure, or after a malformed row.

```
static vector3 zero() noexcept;
```
Construct the zero vector.
Returns: (0,0,0).

```
static vector3 unit_x() noexcept;
```
Construct the positive X unit vector.
Returns: (1,0,0).

```
static vector3 unit_y() noexcept;
```
Construct the positive Y unit vector.
Returns: (0,1,0).

```
static vector3 unit_z() noexcept;
```
Construct the positive Z unit vector.
Returns: (0,0,1).

```
static REAL default_tolerance() noexcept;
```

Get the default absolute vector tolerance.
Returns: 64 times machine epsilon for REAL.

A.39 Namespace functions, aliases, and result enums

A.39.1 creators.hpp

```
template <std::floating_point REAL>
std::shared_ptr< vector2< REAL > > make_vector2();
```

Allocate a zero 2D vector with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector2< REAL > > make_vector2(REAL x, REAL y);
```

Allocate a 2D vector with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector2< REAL > > make_vector2(const point2< REAL > &point_value);
```

Allocate a 2D vector from a point with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector2< REAL > > make_vector2(const circle2< REAL > &circle_value);
```

Allocate a 2D vector from a circle center with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector2< REAL > > make_vector2(const ray2< REAL > &ray_value);
```

Allocate a 2D vector from a ray origin with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< point2< REAL > > make_point2();
```

Allocate the 2D origin with shared ownership.
Returns: Shared smart pointer owning the new point.

```
template <std::floating_point REAL>
std::shared_ptr< point2< REAL > > make_point2(REAL x, REAL y);
```

Allocate a 2D point with shared ownership.
Returns: Shared smart pointer owning the new point.

```
template <std::floating_point REAL>
std::shared_ptr< ray2< REAL > > make_ray2(const point2< REAL > &origin, const vector2< REAL > &
    direction, REAL tolerance=vector2< REAL >::default_tolerance());
```

Allocate a 2D ray with shared ownership.
Returns: Shared smart pointer owning the new ray.

```
template <std::floating_point REAL>
std::shared_ptr< ray2< REAL > > make_ray2_from_points(const point2< REAL > &origin, const point2<
    REAL > &through_point, REAL tolerance=vector2< REAL >::default_tolerance());
```

Allocate a 2D ray defined by two points with shared ownership.
Returns: Shared smart pointer owning the new ray. Its unnormalized direction is through_point - origin.

```
template <std::floating_point REAL>
std::shared_ptr< circle2< REAL > > make_circle2(const point2< REAL > &center, REAL radius);
```

Allocate a 2D circle with shared ownership.
Returns: Shared smart pointer owning the new circle.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline2< REAL > > make_nurbs_spline2(const point2< REAL > &start, const
    point2< REAL > &end, std::size_t control_point_count, std::size_t degree, std::type_identity_t
    < REAL > tolerance);
```

Allocate a standard open 2D NURBS with explicit tolerance.
Returns: Shared smart pointer owning the new 2D spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED = bool>
std::shared_ptr< nurbs_spline2< REAL > > make_nurbs_spline2(const point2< REAL > &start, const
    point2< REAL > &end, std::size_t control_point_count, std::size_t degree, CLOSED closed=false,
     std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate a standard open or closed 2D NURBS between two points.
Returns: Shared smart pointer owning the new 2D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline2< REAL > > make_nurbs_spline2(std::vector< point2< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate a complete 2D NURBS definition with shared ownership.
Returns: Shared smart pointer owning the new 2D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline2< REAL > > make_nurbs_spline2(std::vector< point2< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    bool closed, REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate an open or closed complete 2D NURBS definition.
Returns: Shared smart pointer owning the new 2D spline.

```
template <std::floating_point REAL, typename control_point_array>
std::shared_ptr< nurbs_spline2< REAL > > make_nurbs_spline2(control_point_array &&control_points,
    const std::valarray< REAL > &weights, const std::valarray< REAL > &knots, std::size_t degree,
    REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate a complete 2D NURBS definition from valarrays.
Returns: Shared smart pointer owning the new 2D spline.

```
template <std::floating_point REAL, typename control_point_array>
std::shared_ptr< nurbs_spline2< REAL > > make_nurbs_spline2(control_point_array &&control_points,
    const std::valarray< REAL > &weights, const std::valarray< REAL > &knots, std::size_t degree,
    bool closed, REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate an open or closed 2D NURBS definition from valarrays.
Returns: Shared smart pointer owning the new 2D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline2< REAL > > make_nurbs_spline2(const nurbs_arr_spline2< REAL > &
    spline);
```

Allocate an ordinary 2D NURBS by copying an array-backed spline.
Returns: Shared smart pointer owning an independent ordinary 2D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline2< REAL > > make_nurbs_arr_spline2(const point2< REAL > &start,
    const point2< REAL > &end, std::size_t control_point_count, std::size_t degree, std::
    type_identity_t< REAL > tolerance);
```

Allocate a standard open array-backed 2D NURBS with explicit tolerance.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED = bool>
std::shared_ptr< nurbs_arr_spline2< REAL > > make_nurbs_arr_spline2(const point2< REAL > &start,
    const point2< REAL > &end, std::size_t control_point_count, std::size_t degree, CLOSED closed=
    false, std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon()
    );
```

Allocate a standard open or closed array-backed 2D NURBS.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline2< REAL > > make_nurbs_arr_spline2(std::valarray< REAL >
    parameters, std::size_t degree, std::type_identity_t< REAL > tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Allocate an open array-backed 2D NURBS from flattened scalars.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED>
std::shared_ptr< nurbs_arr_spline2< REAL > > make_nurbs_arr_spline2(std::valarray< REAL >
    parameters, std::size_t degree, CLOSED closed, std::type_identity_t< REAL > tolerance=REAL(64)
     *std::numeric_limits< REAL >::epsilon());
```

Allocate an open or closed array-backed 2D NURBS from scalars.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline2< REAL > > make_nurbs_arr_spline2(std::vector< point2< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate an open array-backed 2D NURBS from vector collections.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED>
std::shared_ptr< nurbs_arr_spline2< REAL > > make_nurbs_arr_spline2(std::vector< point2< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    CLOSED closed, std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::
    epsilon());
```

Allocate an open or closed array-backed 2D NURBS from vectors.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline2< REAL > > make_nurbs_arr_spline2(const nurbs_spline2< REAL > &
    spline);
```

Allocate an array-backed copy of an ordinary 2D NURBS spline.
Returns: Shared smart pointer owning the independent array-backed spline.

```
template <std::floating_point REAL>
std::shared_ptr< vector3< REAL > > make_vector3();
```

Allocate a zero 3D vector with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector3< REAL > > make_vector3(REAL x, REAL y, REAL z);
```

Allocate a 3D vector with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector3< REAL > > make_vector3(const point3< REAL > &point_value);
```

Allocate a 3D vector from a point with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector3< REAL > > make_vector3(const sphere3< REAL > &sphere_value);
```

Allocate a 3D vector from a sphere center with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector3< REAL > > make_vector3(const plane3< REAL > &plane_value);
```

Allocate a 3D vector from a plane origin with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< vector3< REAL > > make_vector3(const ray3< REAL > &ray_value);
```

Allocate a 3D vector from a ray origin with shared ownership.
Returns: Shared smart pointer owning the new vector.

```
template <std::floating_point REAL>
std::shared_ptr< point3< REAL > > make_point3();
```

Allocate the 3D world origin with shared ownership.
Returns: Shared smart pointer owning the new point.

```
template <std::floating_point REAL>
std::shared_ptr< point3< REAL > > make_point3(REAL x, REAL y, REAL z);
```

Allocate a 3D point with shared ownership.
Returns: Shared smart pointer owning the new point.

```
template <std::floating_point REAL>
std::shared_ptr< ray3< REAL > > make_ray3(const point3< REAL > &origin, const vector3< REAL > &
    direction, REAL tolerance=vector3< REAL >::default_tolerance());
```

Allocate a 3D ray with shared ownership.
Returns: Shared smart pointer owning the new ray.

```
template <std::floating_point REAL>
std::shared_ptr< ray3< REAL > > make_ray3_from_points(const point3< REAL > &origin, const point3<
    REAL > &through_point, REAL tolerance=vector3< REAL >::default_tolerance());
```

Allocate a 3D ray defined by two world points with shared ownership.
Returns: Shared smart pointer owning the new ray. Its unnormalized direction is through_point - origin.

```
template <std::floating_point REAL>
std::shared_ptr< sphere3< REAL > > make_sphere3(const point3< REAL > &center, REAL radius);
```

Allocate a 3D sphere with shared ownership.
Returns: Shared smart pointer owning the new sphere.

```
template <std::floating_point REAL>
std::shared_ptr< plane3< REAL > > make_plane3(const vector3< REAL > &normal, REAL signed_distance,
     REAL tolerance=vector3< REAL >::default_tolerance());
```

Allocate a Hessian normal-form plane with shared ownership.
Returns: Shared smart pointer owning the new plane.

```
template <std::floating_point REAL>
std::shared_ptr< plane3< REAL > > make_plane3(const point3< REAL > &origin, const vector3< REAL >
    &normal, REAL tolerance=vector3< REAL >::default_tolerance());
```

Allocate a point-and-normal plane with shared ownership.
Returns: Shared smart pointer owning the new plane.

```
template <std::floating_point REAL>
std::shared_ptr< plane3< REAL > > make_plane3(const point3< REAL > &origin, const vector3< REAL >
    &normal, const vector3< REAL > &u_hint, REAL tolerance=vector3< REAL >::default_tolerance());
```

Allocate a plane with a preferred positive-u direction.
Returns: Shared smart pointer owning the new plane.

```
template <std::floating_point REAL>
std::shared_ptr< plane3< REAL > > make_plane3_from_u_direction(const point3< REAL > &origin, const
     vector3< REAL > &u_direction, const point3< REAL > &plane_point, REAL tolerance=vector3< REAL
     >::default_tolerance());
```

Allocate a plane from a parameter origin, positive-u direction, and another plane point with shared ownership.
Returns: Shared smart pointer owning a plane with a right-handed (u,v,normal) frame.

```
template <std::floating_point REAL>
std::shared_ptr< plane3< REAL > > make_plane3_from_points(const point3< REAL > &origin, const
    point3< REAL > &u_point, const point3< REAL > &plane_point, REAL tolerance=vector3< REAL >::
    default_tolerance());
```

Allocate a plane from three non-collinear world points with shared ownership.
Returns: Shared smart pointer owning a plane with a right-handed (u,v,normal) frame.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline3< REAL > > make_nurbs_spline3(const point3< REAL > &start, const
    point3< REAL > &end, std::size_t control_point_count, std::size_t degree, std::type_identity_t
    < REAL > tolerance);
```

Allocate a standard open 3D NURBS with explicit tolerance.
Returns: Shared smart pointer owning the new 3D spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED = bool>
std::shared_ptr< nurbs_spline3< REAL > > make_nurbs_spline3(const point3< REAL > &start, const
    point3< REAL > &end, std::size_t control_point_count, std::size_t degree, CLOSED closed=false,
     std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate a standard open or closed 3D NURBS between two points.
Returns: Shared smart pointer owning the new 3D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline3< REAL > > make_nurbs_spline3(std::vector< point3< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate a complete 3D NURBS definition with shared ownership.
Returns: Shared smart pointer owning the new 3D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline3< REAL > > make_nurbs_spline3(std::vector< point3< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    bool closed, REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate an open or closed complete 3D NURBS definition.
Returns: Shared smart pointer owning the new 3D spline.

```
template <std::floating_point REAL, typename control_point_array>
std::shared_ptr< nurbs_spline3< REAL > > make_nurbs_spline3(control_point_array &&control_points,
    const std::valarray< REAL > &weights, const std::valarray< REAL > &knots, std::size_t degree,
    REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate a complete 3D NURBS definition from valarrays.
Returns: Shared smart pointer owning the new 3D spline.

```
template <std::floating_point REAL, typename control_point_array>
std::shared_ptr< nurbs_spline3< REAL > > make_nurbs_spline3(control_point_array &&control_points,
    const std::valarray< REAL > &weights, const std::valarray< REAL > &knots, std::size_t degree,
    bool closed, REAL tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate an open or closed 3D NURBS definition from valarrays.
Returns: Shared smart pointer owning the new 3D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_spline3< REAL > > make_nurbs_spline3(const nurbs_arr_spline3< REAL > &
    spline);
```

Allocate an ordinary 3D NURBS by copying an array-backed spline.
Returns: Shared smart pointer owning an independent ordinary 3D spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline3< REAL > > make_nurbs_arr_spline3(const point3< REAL > &start,
    const point3< REAL > &end, std::size_t control_point_count, std::size_t degree, std::
    type_identity_t< REAL > tolerance);
```

Allocate a standard open array-backed 3D NURBS with explicit tolerance.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED = bool>
std::shared_ptr< nurbs_arr_spline3< REAL > > make_nurbs_arr_spline3(const point3< REAL > &start,
    const point3< REAL > &end, std::size_t control_point_count, std::size_t degree, CLOSED closed=
    false, std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon()
    );
```

Allocate a standard open or closed array-backed 3D NURBS.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline3< REAL > > make_nurbs_arr_spline3(std::valarray< REAL >
    parameters, std::size_t degree, std::type_identity_t< REAL > tolerance=REAL(64) *std::
    numeric_limits< REAL >::epsilon());
```

Allocate an open array-backed 3D NURBS from flattened scalars.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED>
std::shared_ptr< nurbs_arr_spline3< REAL > > make_nurbs_arr_spline3(std::valarray< REAL >
    parameters, std::size_t degree, CLOSED closed, std::type_identity_t< REAL > tolerance=REAL(64)
     *std::numeric_limits< REAL >::epsilon());
```

Allocate an open or closed array-backed 3D NURBS from scalars.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline3< REAL > > make_nurbs_arr_spline3(std::vector< point3< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon());
```

Allocate an open array-backed 3D NURBS from vector collections.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL, std::same_as< bool > CLOSED>
std::shared_ptr< nurbs_arr_spline3< REAL > > make_nurbs_arr_spline3(std::vector< point3< REAL > >
    control_points, std::vector< REAL > weights, std::vector< REAL > knots, std::size_t degree,
    CLOSED closed, std::type_identity_t< REAL > tolerance=REAL(64) *std::numeric_limits< REAL >::
    epsilon());
```

Allocate an open or closed array-backed 3D NURBS from vectors.
Returns: Shared smart pointer owning the new array-backed spline.

```
template <std::floating_point REAL>
std::shared_ptr< nurbs_arr_spline3< REAL > > make_nurbs_arr_spline3(const nurbs_spline3< REAL > &
    spline);
```

Allocate an array-backed copy of an ordinary 3D NURBS spline.
Returns: Shared smart pointer owning the independent array-backed spline.

A.39.2 distance.hpp

```
template <std::floating_point REAL>
point_circle_distance2< REAL > distance_to_circle(const point2< REAL > &point, const circle2< REAL
     > &circle, REAL tolerance=REAL(1e-8));
```

Compute radial point-to-circle distance information in 2D.
Returns: 2D distance, closest circle point, and circle angle.

```
template <std::floating_point REAL>
point_spline_distance2< REAL > distance_to_spline(const point2< REAL > &point, const nurbs_spline2
    < REAL > &spline, const numerical_settings< REAL > &settings={});
```

Numerically find the sampled global 2D point-to-spline minimum.
Returns: 2D distance, closest spline point found, and spline parameter.

```
template <std::floating_point REAL>
point_surface_distance3< REAL > distance_to_plane(const point3< REAL > &point, const plane3< REAL
    > &plane, REAL tolerance=REAL(1e-8));
```

Compute orthogonal point-to-plane distance information.
Returns: Distance, closest point, and plane parameters.

```
template <std::floating_point REAL>
point_surface_distance3< REAL > distance_to_sphere(const point3< REAL > &point, const sphere3<
    REAL > &sphere, REAL tolerance=REAL(1e-8));
```

Compute radial point-to-sphere distance information.
Returns: Distance, closest point, and spherical parameters.

```
template <std::floating_point REAL>
point_spline_distance3< REAL > distance_to_spline(const point3< REAL > &point, const nurbs_spline3
    < REAL > &spline, const numerical_settings< REAL > &settings={});
```

Numerically find the sampled global point-to-spline minimum.
Returns: Distance, closest point found, and spline parameter.

```
template <std::floating_point REAL>
REAL distance(const point3< REAL > &point, const plane3< REAL > &plane, REAL tolerance=REAL(1e-8));
```

Compute scalar point-to-plane distance.
Returns: Tolerance-adjusted nonnegative distance.

```
template <std::floating_point REAL>
REAL distance(const point3< REAL > &point, const sphere3< REAL > &sphere, REAL tolerance=REAL(1e
    -8));
```

Compute scalar point-to-sphere distance.
Returns: Tolerance-adjusted nonnegative distance.

```
template <std::floating_point REAL>
REAL distance(const point3< REAL > &point, const nurbs_spline3< REAL > &spline, const
    numerical_settings< REAL > &settings={});
```

Compute scalar numerical point-to-spline distance.
Returns: Tolerance-adjusted nonnegative distance.

```
template <std::floating_point REAL>
REAL distance(const point2< REAL > &point, const circle2< REAL > &circle, REAL tolerance=REAL(1e
    -8));
```

Compute scalar point-to-circle distance entirely in 2D.
Returns: Tolerance-adjusted nonnegative 2D distance.

```
template <std::floating_point REAL>
REAL distance(const point2< REAL > &point, const nurbs_spline2< REAL > &spline, const
    numerical_settings< REAL > &settings={});
```

Compute scalar numerical point-to-spline distance entirely in 2D.
Returns: Tolerance-adjusted nonnegative 2D distance.

A.39.3 graphics.hpp

```
enum class svg_projection3 { orthographic, perspective };
```

Camera projections supported by the three-dimensional SVG renderer.

```
template <std::floating_point REAL, typename ENTITY>
std::string to_svg(const svg_view2< REAL > &view, const ENTITY &entity, const
    svg_graphics_options2< REAL > &options={});
```

Render one native 2D entity to a standalone flat SVG document.
Returns: Complete self-contained flat SVG XML string.

```
template <std::floating_point REAL, typename ENTITY>
std::string to_svg(const svg_view3< REAL > &view, const ENTITY &entity, const
    svg_graphics_options3< REAL > &options={});
```

Render one supported entity to a standalone SVG document.
Returns: Complete self-contained SVG XML string.

```
template <std::floating_point REAL, typename ENTITY>
std::string to_svg(const svg_view3< REAL > &view, const plane3< REAL > &plane, const ENTITY &
    entity, const svg_graphics_options3< REAL > &options={});
```

Project one supported 2D entity through a plane and render it to SVG.
Returns: Complete self-contained SVG XML string.

A.39.4 intersections.hpp

```
enum class intersection_kind { none, discrete, coincident };
```

Classification of a geometric intersection set.

```
template <std::floating_point REAL>
intersection_result2< REAL > intersect_ray_circle(const ray2< REAL > &ray, const circle2< REAL > &
    circle, const numerical_settings< REAL > &settings={});
```

Numerically intersect a forward 2D ray with a 2D circle.
Returns: Classified 2D contacts sorted by increasing s.

```
template <std::floating_point REAL>
intersection_result2< REAL > intersect_spline_circle(const nurbs_spline2< REAL > &spline, const
    circle2< REAL > &circle, const numerical_settings< REAL > &settings={});
```

Numerically intersect a 2D NURBS spline with a 2D circle.
Returns: Classified 2D contacts sorted by increasing s.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect_ray_sphere(const ray3< REAL > &ray, const sphere3< REAL > &
    sphere, const numerical_settings< REAL > &settings={});
```

Numerically intersect a forward ray with a sphere.
Returns: Classified contacts sorted by increasing s.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect_ray_plane(const ray3< REAL > &ray, const plane3< REAL > &
    plane, const numerical_settings< REAL > &settings={});
```

Numerically intersect a forward ray with an infinite plane.
Returns: None, one isolated contact, or a coincident classification.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect_spline_sphere(const nurbs_spline3< REAL > &spline, const
    sphere3< REAL > &sphere, const numerical_settings< REAL > &settings={});
```

Numerically intersect a spline with a sphere over its active domain.
Returns: Classified sampled-and-refined contacts sorted by s.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect_spline_plane(const nurbs_spline3< REAL > &spline, const
    plane3< REAL > &plane, const numerical_settings< REAL > &settings={});
```

Numerically intersect a spline with an infinite plane.
Returns: Classified sampled-and-refined contacts sorted by s.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect(const ray3< REAL > &ray, const sphere3< REAL > &sphere,
    const numerical_settings< REAL > &settings={});
```

Convenience overload for ray-sphere intersection.
Returns: Ray-sphere intersection result.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect(const ray3< REAL > &ray, const plane3< REAL > &plane, const
     numerical_settings< REAL > &settings={});
```

Convenience overload for ray-plane intersection.
Returns: Ray-plane intersection result.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect(const nurbs_spline3< REAL > &spline, const sphere3< REAL >
    &sphere, const numerical_settings< REAL > &settings={});
```

Convenience overload for spline-sphere intersection.
Returns: Spline-sphere intersection result.

```
template <std::floating_point REAL>
intersection_result3< REAL > intersect(const nurbs_spline3< REAL > &spline, const plane3< REAL > &
    plane, const numerical_settings< REAL > &settings={});
```

Convenience overload for spline-plane intersection.
Returns: Spline-plane intersection result.

```
template <std::floating_point REAL>
intersection_result2< REAL > intersect(const ray2< REAL > &ray, const circle2< REAL > &circle,
    const numerical_settings< REAL > &settings={});
```

Convenience overload for 2D ray-circle intersection.
Returns: Two-dimensional ray-circle intersection result.

```
template <std::floating_point REAL>
intersection_result2< REAL > intersect(const nurbs_spline2< REAL > &spline, const circle2< REAL >
    &circle, const numerical_settings< REAL > &settings={});
```

Convenience overload for 2D spline-circle intersection.
Returns: Two-dimensional spline-circle intersection result.

A.39.5 point2.hpp

```
template <std::floating_point REAL>
std::ostream & operator<<(std::ostream &output, const point2< REAL > &value);
```

Write a 2D point as whitespace-separated X and Y coordinates.
Returns: Reference to output.

```
template <std::floating_point REAL>
std::istream & operator>>(std::istream &input, point2< REAL > &value);
```

Read whitespace-separated X and Y coordinates into a 2D point.
Returns: Reference to input.

```
template <std::floating_point REAL>
point2< REAL > operator+(point2< REAL > point, const vector2< REAL > &offset) noexcept;
```

Translate a point by a vector.
Returns: Translated point.

```
template <std::floating_point REAL>
point2< REAL > operator+(const vector2< REAL > &offset, point2< REAL > point) noexcept;
```

Translate a point with vector-first syntax.
Returns: Translated point.

```
template <std::floating_point REAL>
point2< REAL > operator-(point2< REAL > point, const vector2< REAL > &offset) noexcept;
```

Translate a point by the negative of a vector.
Returns: Translated point.

```
template <std::floating_point REAL>
vector2< REAL > operator-(const point2< REAL > &left, const point2< REAL > &right) noexcept;
```

Form the displacement from one point to another.
Returns: Vector from right to left.

```
template <std::floating_point REAL>
point2< REAL > lerp(const point2< REAL > &first, const point2< REAL > &second, REAL fraction)
    noexcept;
```

Linearly interpolate points.
Returns: Interpolated position.

```
template <std::floating_point REAL>
REAL distance_squared(const point2< REAL > &first, const point2< REAL > &second) noexcept;
```

Compute squared Euclidean point distance.
Returns: Squared distance.

```
template <std::floating_point REAL>
REAL distance(const point2< REAL > &first, const point2< REAL > &second) noexcept;
```

Compute Euclidean point distance.
Returns: Distance.

A.39.6 point3.hpp

```
template <std::floating_point REAL>
std::ostream & operator<<(std::ostream &output, const point3< REAL > &value);
```

Write a 3D point as whitespace-separated X, Y, and Z coordinates.
Returns: Reference to output.

```
template <std::floating_point REAL>
std::istream & operator>>(std::istream &input, point3< REAL > &value);
```

Read whitespace-separated X, Y, and Z coordinates into a 3D point.
Returns: Reference to input.

```
template <std::floating_point REAL>
point3< REAL > operator+(point3< REAL > point, const vector3< REAL > &offset) noexcept;
```

Translate a point by a vector.
Returns: Translated point.

```
template <std::floating_point REAL>
point3< REAL > operator+(const vector3< REAL > &offset, point3< REAL > point) noexcept;
```

Translate a point by a vector with vector-first syntax.
Returns: Translated point.

```
template <std::floating_point REAL>
point3< REAL > operator-(point3< REAL > point, const vector3< REAL > &offset) noexcept;
```

Translate a point by the negative of a vector.
Returns: Translated point.

```
template <std::floating_point REAL>
vector3< REAL > operator-(const point3< REAL > &left, const point3< REAL > &right) noexcept;
```

Form the displacement from one point to another.
Returns: Vector from right to left.

```
template <std::floating_point REAL>
point3< REAL > rotate(const vector3< REAL > &axis, REAL angle, const point3< REAL > &point);
```

Rotate a point about an axis through the Cartesian world origin.
Returns: Rotated point in the same Cartesian world coordinate system.

```
template <std::floating_point REAL>
point3< REAL > lerp(const point3< REAL > &first, const point3< REAL > &second, REAL fraction)
    noexcept;
```

Linearly interpolate two points.
Returns: Interpolated position.

```
template <std::floating_point REAL>
REAL distance_squared(const point3< REAL > &first, const point3< REAL > &second) noexcept;
```

Compute squared Euclidean distance between points.
Returns: Squared distance.

```
template <std::floating_point REAL>
REAL distance(const point3< REAL > &first, const point3< REAL > &second) noexcept;
```

Compute Euclidean distance between points.
Returns: Distance.

A.39.7 projection.hpp

```
template <std::floating_point REAL>
vector3< REAL > project(const plane3< REAL > &plane, const vector2< REAL > &vector) noexcept;
```

Embed a 2D vector in a plane's orthonormal parameter frame.
Returns: vector.x*u_direction + vector.y*v_direction in 3D.

```
template <std::floating_point REAL>
point3< REAL > project(const plane3< REAL > &plane, const point2< REAL > &point) noexcept;
```

Embed a 2D point in a plane's parameter frame.
Returns: 3D point plane.point_at(point.x, point.y).

```
template <std::floating_point REAL>
ray3< REAL > project(const plane3< REAL > &plane, const ray2< REAL > &ray);
```

Embed a 2D ray in a plane's parameter frame.
Returns: 3D ray with projected origin and direction and the same native s.

```
template <std::floating_point REAL>
sphere3< REAL > project(const plane3< REAL > &plane, const circle2< REAL > &circle);
```

Convert a 2D circle's center and radius to a 3D sphere.
Returns: Sphere with projected center and unchanged radius.

```
template <std::floating_point REAL>
nurbs_spline3< REAL > project(const plane3< REAL > &plane, const nurbs_spline2< REAL > &spline);
```

Embed a 2D NURBS spline by projecting its control points.
Returns: 3D NURBS spline with projected control points.

A.39.8 serialization.hpp

```
enum class text_format { csv, tsv, txt };
```

Delimited text format used for Cartesian coordinate input and output.

A.39.9 tagged_annotations.hpp

```
enum class diagnostic_level { warning, information, error };
```

Severity selecting an optional positioned diagnostic output token.

A.39.10 tagged_serialization.hpp

```
template <std::floating_point REAL>
using entity3_ptr = std::variant< std::shared_ptr< point3< REAL > >, std::shared_ptr< vector3<
    REAL > >, std::shared_ptr< sphere3< REAL > >, std::shared_ptr< nurbs_spline3< REAL > > >;
```

Type-safe pointer alternatives supported by tagged 3D input.

```
template <std::floating_point REAL>
using tagged_record_ptr = std::variant< std::shared_ptr< point3< REAL > >, std::shared_ptr<
    vector3< REAL > >, std::shared_ptr< sphere3< REAL > >, std::shared_ptr< nurbs_spline3< REAL >
    >, std::shared_ptr< line_color< REAL > >, std::shared_ptr< line_font< REAL > >, std::
    shared_ptr< text_label< REAL > > >;
```

Type-safe owners for geometry and optional presentation records.

```
template <std::floating_point REAL>
std::optional< tagged_entity3< REAL > > tag_read(std::istream &input);
```

Read one version-1 tagged row and allocate its concrete 3D entity.
Returns: Tag and entity variant selecting a non-null shared owner after success, or std::nullopt at EOF, another read failure, or after a malformed row. Unknown types, invalid definitions, and trailing fields set failbit.

```
template <std::floating_point REAL>
std::optional< tagged_record< REAL > > tag_read_record(std::istream &input);
```

Read one geometry, linecolor, linefont, or text version-1 row.
Returns: Preserved tag and variant holding a non-null owner, or nullopt on EOF/read failure. Invalid values, unknown types/versions, malformed quoting, or extra fields in a non-advisory row set failbit after consuming that row.

A.39.11 utility.hpp

```
std::size_t nurbs_knot_count(std::size_t degree, std::size_t control_point_count);
```

Calculate the knot-vector size required by a valid NURBS spline.
Returns: Required number of values in the complete knot vector.

```
template <std::floating_point REAL>
REAL square(REAL value) noexcept;
```

Square a scalar.
Returns: value * value.

```
template <std::floating_point REAL>
bool approximately_equal(REAL first, REAL second, REAL tolerance) noexcept;
```

Compare two scalar values with absolute tolerance.
Returns: True when absolute difference is at most tolerance.

```
template <std::floating_point REAL>
std::vector< REAL > solve_linear_system(std::vector< std::vector< REAL > > matrix, std::vector<
    REAL > right_hand_side, REAL pivot_tolerance=REAL(64) *std::numeric_limits< REAL >::epsilon())
    ;
```

Solve a square linear system using scaled partial pivoting.
Returns: Solution vector x for A*x=b.

```
template <std::floating_point REAL, typename FUNCTION>
REAL bisect_root(FUNCTION &&function, REAL lower, REAL upper, const numerical_settings< REAL > &
    settings={});
```

Refine a sign-changing root interval by bisection.
Returns: Refined root parameter.

```
template <std::floating_point REAL, typename FUNCTION>
REAL golden_section_minimize(FUNCTION &&function, REAL lower, REAL upper, const numerical_settings
    < REAL > &settings={});
```

Minimize a scalar function on a closed interval by golden-section search.
Returns: Parameter at the refined sampled minimum.

```
template <std::floating_point REAL, typename FUNCTION>
std::vector< REAL > find_roots(FUNCTION &&function, REAL lower, REAL upper, const
    numerical_settings< REAL > &settings={});
```

Discover and refine roots on a bounded sampled interval.
Returns: Sorted, tolerance-deduplicated root parameters.

A.39.12 vector2.hpp

```
template <std::floating_point REAL>
std::ostream & operator<<(std::ostream &output, const vector2< REAL > &value);
```

Write a 2D vector as whitespace-separated X and Y components.
Returns: Reference to output.

```
template <std::floating_point REAL>
std::istream & operator>>(std::istream &input, vector2< REAL > &value);
```

Read whitespace-separated X and Y components into a 2D vector.
Returns: Reference to input.

```
template <std::floating_point REAL>
vector2< REAL > operator+(vector2< REAL > left, const vector2< REAL > &right) noexcept;
```

Add two vectors.
Returns: Vector sum.

```
template <std::floating_point REAL>
vector2< REAL > operator-(vector2< REAL > left, const vector2< REAL > &right) noexcept;
```

Subtract two vectors.
Returns: Vector difference.

```
template <std::floating_point REAL>
vector2< REAL > operator*(vector2< REAL > value, REAL scalar) noexcept;
```

Multiply a vector by a scalar.
Returns: Scaled vector.

```
template <std::floating_point REAL>
vector2< REAL > operator*(REAL scalar, vector2< REAL > value) noexcept;
```

Multiply a scalar by a vector.
Returns: Scaled vector.

```
template <std::floating_point REAL>
vector2< REAL > operator/(vector2< REAL > value, REAL scalar);
```

Divide a vector by a scalar.
Returns: Divided vector.

```
template <std::floating_point REAL>
REAL dot(const vector2< REAL > &left, const vector2< REAL > &right) noexcept;
```

Compute the dot product.
Returns: Dot product.

```
template <std::floating_point REAL>
REAL cross(const vector2< REAL > &left, const vector2< REAL > &right) noexcept;
```

Compute the signed scalar cross product.
Returns: Signed scalar cross product.

```
template <std::floating_point REAL>
vector2< REAL > lerp(const vector2< REAL > &first, const vector2< REAL > &second, REAL fraction)
    noexcept;
```

Linearly interpolate vectors.
Returns: Interpolated vector.

A.39.13 vector3.hpp

```
template <std::floating_point REAL>
std::ostream & operator<<(std::ostream &output, const vector3< REAL > &value);
```

Write a 3D vector as whitespace-separated X, Y, and Z components.
Returns: Reference to output.

```
template <std::floating_point REAL>
std::istream & operator>>(std::istream &input, vector3< REAL > &value);
```

Read whitespace-separated X, Y, and Z components into a 3D vector.
Returns: Reference to input.

```
template <std::floating_point REAL>
vector3< REAL > operator+(vector3< REAL > left, const vector3< REAL > &right) noexcept;
```

Add two vectors.
Returns: Vector sum.

```
template <std::floating_point REAL>
vector3< REAL > operator-(vector3< REAL > left, const vector3< REAL > &right) noexcept;
```

Subtract two vectors.
Returns: Vector difference.

```
template <std::floating_point REAL>
vector3< REAL > operator*(vector3< REAL > value, REAL scalar) noexcept;
```

Multiply a vector by a scalar.
Returns: Scaled vector.

```
template <std::floating_point REAL>
vector3< REAL > operator*(REAL scalar, vector3< REAL > value) noexcept;
```

Multiply a scalar by a vector.
Returns: Scaled vector.

```
template <std::floating_point REAL>
vector3< REAL > operator/(vector3< REAL > value, REAL scalar);
```

Divide a vector by a scalar.
Returns: Divided vector.

```
template <std::floating_point REAL>
REAL dot(const vector3< REAL > &left, const vector3< REAL > &right) noexcept;
```

Compute the dot product of two vectors.
Returns: Dot product.

```
template <std::floating_point REAL>
vector3< REAL > cross(const vector3< REAL > &left, const vector3< REAL > &right) noexcept;
```

Compute the cross product of two vectors.
Returns: Right-handed cross product.

```
template <std::floating_point REAL>
vector3< REAL > lerp(const vector3< REAL > &first, const vector3< REAL > &second, REAL fraction)
    noexcept;
```

Linearly interpolate two vectors.
Returns: Interpolated vector.

Appendix B

Rebuilding and extending this manual

B.1 A standalone local build

The manual is intentionally isolated. Its source, generated reference data, build helper, and final PDF are all under `manual/`. It adds no CMake target and requires no change to the project's other documentation.

From the repository root, run:

```
sh manual/build.sh
```

The script uses an already installed C++20 compiler and `pdflatex`. It neither installs packages nor downloads resources. It compiles `pictures/generate_reference.cpp` against `../include`, checks and regenerates the geometry data, runs LaTeX three times to resolve the contents and references, then writes `manual/nurbspath-manual.pdf`. Temporary compiler and TeX products remain under `manual/build/`.

The local TeX installation must already provide TikZ, PGFPlots, Latin Modern, AMS mathematics, geometry, listings, hyperref, booktabs/tabularx/longtable, enumitem, needspace, fancyhdr, microtype, bookmark, and tcolorbox. This edition was built with locally available TeX Live 2026. TikZ pictures are compiled inside the document, without shell escape or externalisation.

B.2 Source layout

Location	Purpose
`main.tex`	Master document and optional author-file hooks.
`preamble.tex`	Typography, colours, TikZ styles, code listings, and page layout.
`sections/00-title.tex`	Cover and snapshot identification.
`sections/01-foundations.tex`	Worlds, units, parameter conventions, and first program.
`sections/02-entities.tex`	Positions, vectors, rays, round surfaces, planes, embedding.
`sections/03-creators.tex`	Value construction and shared-ownership factories.
`sections/04-splines.tex`	Ordinary/array NURBS definitions and curve algorithms.
`sections/05-algorithms.tex`	Numerical utilities, intersections, and distances.
`sections/06-graphics-io.tex`	SVG rendering, persistence, and advisory output.
`sections/07-reference.tex`	API catalogue introduction.
`sections/reference_api.tex`	Generated signature/brief catalogue.
`pictures/`	C++ reference generator, sampled data, and display-value macros.
`tools/generate_api.py`	Optional local Doxygen-to-LaTeX catalogue refresh.

B.3 Your introductory and closing material

The master already contains conditional hooks for two files:

```
\InputIfFileExists{intro.tex}{}{}
...
\InputIfFileExists{external.tex}{}{}
```

Place `intro.tex` in the manual directory to insert your introductory words after the cover and before the contents. Place `external.tex` there to insert your closing material after the appendices. Neither file is created as a placeholder, so there is nothing to overwrite when you import your own files. The document builds while they are absent.

These are included fragments, not separate documents: omit `\documentclass`, `\begin{document}`, and `\end{document}`. You can use the same TikZ styles, `\api`, `manualnote`, and `lstlisting` formatting as the rest of the book. Use `\chapter*{Introduction}` or a similar unnumbered heading if wanted; add it to the contents explicitly with `\addcontentsline` if appropriate.

B.4 Refreshing the API snapshot

A normal build consumes the retained LaTeX catalogue. To refresh it from the current public headers using an already installed Doxygen and Python:

```
cd manual
python3 tools/generate_api.py
sh build.sh
```

The extractor writes a machine-readable coverage inventory to `pictures/api_inventory.json`; it excludes private members and detail helpers and counts public callable overloads. No network

call or package installation is part of this process. The installed Doxygen 1.18 comment lexer can occasionally terminate by signal in this environment; the refresh helper retries signal failures a bounded number of times and never suppresses an ordinary documentation error.

B.5 Checking numerical and visual changes

The C++ reference generator checks interpolation at the supplied stations, rational quarter-circle radii, crossing and tangent contact counts, closest point consistency, array conversion, plane embedding, root discovery, and a linear-system solution before writing coordinates. Display rounding occurs after those checks. It is a reproducible worked-example validation, not a replacement for the library's numbered test suite.

After editing the book, inspect its build log for LaTeX errors, unresolved references, and overfull boxes. Render pages with a local PDF renderer to check diagrams, tables, code, contents, headers, and page breaks. The supplied PDF is the rendered result of the checked-in section files and reference data.